A Lightweight External Memory Engine to Bridge the Capability Gap Between On-Device and Cloud Language Models

Ibrahim Sudheysi Ali

166344

ICS 4C

Supervisor Name

Tiberius Tabulu

An Informatics and Computer Science Project Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

November 2026

Declaration and Approval

I declare that this work has not been previously submitted and approved for the award of a degree by this or any other University. To the best of my knowledge and belief, the project report contains no material previously published or written by another person except where due reference is made in the report itself.

Student Name: Ibrahim Sudheysi Ali

Admission Number: 166344

Student Signature: _____ Date: _____

The project report of Ibrahim Sudheysi Ali has been reviewed and approved by Tiberius Tabulu, School of Computing and Engineering Sciences, Strathmore University.

Supervisor Signature: _____ Date: _____

Abstract

Small language models that run on a laptop keep personal data private, work offline and cost nothing per question, but they know far less than large cloud models and forget everything that falls outside their context window. This study designed, built and evaluated CAMR, a lightweight external memory engine that sits beside a frozen model on an Apple-silicon Mac. CAMR does not store text for the model to read again. It stores the model's own working state: the keys and values that the model writes at every layer while it reads a moment. Related moments are kept together in rooms on the SSD; a question opens the one room it is about, reads it in one burst and lays it in front of the model already read. Three layers share the work: a small core that is always in RAM, the rooms, which move between RAM and the SSD under fixed budgets and go to sleep as words when the disk budget is full, and a vault of plain-text notes that a person can browse. A reach setting decides whether a memory belongs to one session, one project, one agent or everyone.

CAMR was evaluated on the Long Haul, a scripted project history of 2.01 million tokens extended to 4.01 million, with 216 questions graded by rule, against the same models without memory and against Claude Sonnet, Haiku and Opus 5.5 reading as much of the history as fits their one-million-token window. A 4-billion-parameter model with CAMR answered 29 of 73 questions at 2, 3 and 4 million tokens; Claude Sonnet alone answered 32, then 13, then 10. The same model without memory answered 11 to 13 and needed 15.2 GB of RAM; with CAMR the process needed 5.4 to 7.5 GB once the graphics buffer cache was capped. Its first word came in 0.2 to 0.3 seconds, against 13 to 105 seconds for Claude Sonnet and Opus, and the wait of a 1.7-billion-parameter model with CAMR did not grow as its memory grew from 112 thousand to 2 million tokens. With CAMR, the 4B declined 9 of the 10 questions about things never told at 2 million tokens, and all 10 beyond. Past one million tokens, most rooms had been put to sleep to hold a 40 GB disk budget, so most of those answers read their room as words rather than as saved notes: the notes made answers faster, not more often right. CAMR did not help with reasoning across two facts or with building a page from a design decided weeks earlier. An earlier text-note version of CAMR, kept as the baseline, showed on public benchmarks that memory closes knowledge gaps but not reasoning gaps.

Keywords: on-device language models; external memory; key–value cache; retrieval; Apple silicon; long context.

Table of Contents

Chapter 1: Introduction 1
1.1 Background Information 1
1.2 Problem Statement 2
1.3 Objectives 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Research Questions 3
1.5 Justification 3
1.6 Scope 4
1.7 Limitations and Delimitations 4
Chapter 2: Literature Review 6
2.1 Introduction 6
2.2 The State of On-Device Language Models and External Memory 6
2.2.1 The Rise of On-Device Models 6
2.2.2 What the Capability Gap Is Made Of 6
2.2.3 Retrieval as a Substitute for Scale 6
2.2.4 Compressing What the Model Reads 7
2.2.5 Reusing the Model's Own Keys and Values 7
2.2.6 Evaluation and Risks of Persistent Memory 8
2.3 Related Works 9
2.3.1 MemGPT 9
2.3.2 Generative Agents 9
2.3.3 MemoryBank 10
2.3.4 HippoRAG 10
2.3.5 Mem0 11
2.4 Gaps in Related Works 12
2.5 Conceptual Framework 13
Chapter 3: Methodology 15
3.1 Introduction 15
3.2 Research Paradigm 15
3.2.1 Data Acquisition 15
3.2.2 Data Preparation 16
3.2.3 Experimental Conditions 16
3.2.4 Measurement and Validation 16
3.3 Development Approach 16
3.4 System Analysis 17

3.4.1 Functional Requirements 17
3.4.2 Non-Functional Requirements 17
3.5 System Design 18
3.5.1 Architectural Overview 18
3.5.2 Experimental Design 18
3.5.3 Datasets and Task Types 19
3.5.4 Evaluation Metrics 19
3.5.5 Comparison Procedure 20
3.5.6 Threats to Validity 20
3.6 Tools and Techniques 20
3.6.1 Python 20
3.6.2 MLX and Local Models 21
3.6.3 Hosted Inference APIs 21
3.6.4 Embedding Model 21
3.6.5 Storage on the Device 21
3.6.6 Application and Auxiliary Tools 21
3.7 Deliverables 21
3.7.1 The CAMR Memory Engine 21
3.7.2 The Evaluation Harness 21
3.7.3 Experimental Results 22
3.7.4 The Application and System Documentation 22
3.8 Ethical Considerations 22
Chapter 4: System Analysis and Design 23
4.1 Introduction 23
4.2 System Requirements 24
4.2.1 Functional Requirements 24
4.2.2 Non-Functional Requirements 27
4.2.3 Data Requirements 28
4.2.4 Interface and Integration Requirements 29
4.2.5 Requirements Traceability 30
4.3 System Analysis Diagrams 30
4.3.1 Context Diagram 30
4.3.2 Use Case Diagram 33
4.3.3 Evaluation and Data Pipeline 36
4.4 System Design Diagrams 38
4.4.1 Architecture of the Baseline, CAMR v1 38
4.4.2 Architecture of CAMR 39
4.4.3 Sequence Diagram: One Question 43

4.4.4 State Diagram of a Room and Its Moments 44
4.4.5 Logical Schema of the Baseline Store 46
4.4.6 File and Data Design of CAMR 47
4.4.7 Deployment on the Mac 49
4.4.8 User Interface Design 52
4.5 Design Changes Since the Proposal 56
4.6 Chapter Summary 57
Chapter 5: System Implementation, Testing and Evaluation 58
5.1 Introduction 58
5.2 Implementation Environment 59
5.2.1 Hardware and Deployment Environment 59
5.2.2 Software, Services and Configuration 59
5.2.3 Deployment and Reproducibility 60
5.3 Dataset Description 61
5.3.1 The Long Haul 61
5.3.2 Public Suites from the Hugging Face Hub 62
5.3.3 Data of the Earlier Stages 63
5.4 System Implementation 63
5.4.1 The Engine's Modules 63
5.4.2 Write Path, Read Path and Tiers 64
5.4.3 The Vault, the Core, Reach and the Switch 64
5.4.4 Memory for Other Runtimes: the Ollama Pairing 64
5.4.5 The Application 65
5.4.6 The Benchmark Harness and the Contenders 65
5.5 Testing 66
5.5.1 Test Strategy 66
5.5.2 Test Cases and Results 66
5.5.3 Defects Found and Retests 68
5.6 Version Control and Implementation Evidence 68
5.7 Author Oversight and Use of AI-Assisted Tools 68
5.8 Evaluation Results, Discussion and Limitations 69
5.8.1 Results at a Glance 69
5.8.2 The Long Haul: Right Answers as the History Grows 70
5.8.3 Past the Window: 3M and 4M 72
5.8.4 What Was Answered: Knowledge Against Reasoning 72
5.8.5 Larger Readers: 8B and 27B 74
5.8.6 The 1.7B: Its Wording, Not Its Memory 74
5.8.7 Notes Against Words: What the Tensors Buy 74

5.8.8 Speed as Memory Grows 75
5.8.9 RAM: the Cap on the Buffer Cache 76
5.8.10 Which Reach Works Best 77
5.8.11 The Claude Yardstick: Time and Cost 78
5.8.12 The Public Suites 78
5.8.13 The Ollama Pairing on the Suites 79
5.8.14 Multimodal Memory 80
5.8.15 Development Evidence: the Rooms Build on Scripted Lives 81
5.8.16 The Baseline, CAMR v1, on Public Benchmarks 83
5.8.17 What Failed, and Why 83
5.8.18 Limitations, Delimitations and Threats to Validity 84
Chapter 6: Discussion, Conclusions, Recommendations and Future Works 86
6.1 Introduction 86
6.2 Conclusions by Objective 86
6.2.1 Objective i: Techniques and Their Limits on a Device 86
6.2.2 Objective ii: The Design 86
6.2.3 Objective iii: On the Device 87
6.2.4 Objective iv: Evaluation of the Gap 87
6.2.5 Objective v: How Much to Read, and What Each Part Contributes 87
6.2.6 Contribution 88
6.3 Limitations of the Evidence 89
6.4 Recommendations 90
6.5 Future Works 90
6.6 Closing Remarks 91
References 92
Appendix A: Reproducing the Results 96
Appendix B: Decision Log Entries Cited 98

List of Figures

Figure 2.1: The MemGPT virtual context architecture, adapted from Packer et al. (2023) 9

Figure 2.2: The Smallville environment of Generative Agents (Source: Park et al., 2023, project repository)10

Figure 2.3: The MemoryBank framework with memory updating and forgetting (Source: Zhong et al., 2024, project repository) 10

Figure 2.4: HippoRAG offline indexing and online retrieval (Source: Gutierrez et al., 2024, project repository)11

Figure 2.5: The Mem0 memory layer within an agent stack (Source: Mem0 project documentation)12

Figure 2.6: Conceptual framework of the study 13

Figure 4.1: Context diagram of CAMR 31

Figure 4.2: Use case diagram of CAMR 34

Figure 4.3: The evaluation pipeline of the Long Haul 37

Figure 4.4: Architecture of the baseline, CAMR v1 39

Figure 4.5: Component diagram of CAMR, the final engine 41

Figure 4.6: Sequence diagram of one question answered in Chat 44

Figure 4.7: State machines of a moment, a room and a vault note's strength 45

Figure 4.8: Logical schema of the baseline's store 46

Figure 4.9: Files of a CAMR memory, its reaches and its records 48

Figure 4.10: Deployment diagram of CAMR on the Mac 50

Figure 4.11: Starting the app: the model's 36 layers fill as their weights arrive ("loading 14/36"), before the first question 52

Figure 4.12: Chat with memory on and off: a fact told earlier is answered from the room it was kept in ("room 609 · read from the SSD"); with memory off, the same question gets "I don't know" 53

Figure 4.13: The weights view, large beside the chat for a projector: the room an answer reads travels from the memory (rooms, top) into the model's 36 layers (bottom) 54

Figure 4.14: Code: the agent lists the files of a project folder, reads money.py and answers from what it read; the answer is kept in

memory 55

Figure 4.15: The weights view with memory on (left), where the room read lights and flows into the model, and off (right), where the model answers alone and the memory stays dark 56

Figure 5.1: Share of questions answered right as the history grows. CAMR (cyan) holds its level past 1M tokens; the models alone (amber) and Claude alone (grey, dashed) fall once the history outgrows their windows 71

Figure 5.2: Right answers at 2M tokens by family: CAMR (cyan) against Claude alone (grey, hatched) 73

Figure 5.3: The wait, retrieval time, RAM and disk as the 1.7B's memory grows from 1,000 to 17,676 moments 75

Figure 5.4: Process footprint: the models alone (amber) against CAMR (cyan), before and after the cap 76

Figure 5.5: Right answers by reach 77

Figure 5.6: The wait for the first word as the history grows: CAMR's median (cyan) against the model alone (amber) and Claude's first token per request (grey, dashed); note the logarithmic scale 78

List of Tables

Table 2.1: Comparison of representative memory systems for language models 12

Table 3.1: Experimental conditions 18

Table 3.2: Evaluation datasets and the task type each represents 19

Table 4.1: The three stages of CAMR 23

Table 4.2: Functional requirements catalogue 25

Table 4.3: Non-functional requirements catalogue 27

Table 4.4: Data requirements catalogue 28

Table 4.5: Interface and integration requirements catalogue 29

Table 4.6: Traceability matrix 30

Table 4.7: Files of a CAMR memory 48

Table 4.8: What every answer records 48

Table 4.9: Changes from the proposed design 56

Table 5.1: Hardware used (from each record's environment field) 59

Table 5.2: Software and models (versions as tested) 60

Table 5.3: What the Long Haul plants and asks 61

Table 5.4: The cuts and their checkpoints 62

Table 5.5: The verdicts 62

Table 5.6: The public suites as run 62

Table 5.7: The engine's main modules (v2/camr_engine/) 63

Table 5.8: Contenders on the Long Haul 65

Table 5.9: Test cases and results 66

Table 5.10: Defects found and retests 68

Table 5.11: Scorecard: each claim, its evidence and whether it was met 69

Table 5.12: Right answers on the Long Haul (questions asked: 34, 40, 69 and 73; 3M and 4M ask the 2M
questions) 70

Table 5.13: Where the 4B's room came from at each cut (summary.all.tiers and status in the records) 72

Table 5.14: Right answers at 2M by family (10 questions each; for "never said", declining is right) 72

Table 5.15: Why the 1.7B missed facts at 2M (E-37) 74
Table 5.16: The wait as memory grows (engine:qwen3-1.7b; E-38) 75
Table 5.17: The 4B at 3M, before and after the 1 GB cap (E-41) 76
Table 5.18: Which reach works best (engine:qwen3-4b, 256K tokens, 40 questions; E-42) 77
Table 5.19: The Claude yardstick on the Long Haul (from the records) 78
Table 5.20: Right answers on the public suites 78
Table 5.21: BABILong by story length (25 items each; qa1 to qa5) 79
Table 5.22: Multimodal memory: right answers by kind of file (seed 1: 29 planted facts and 8 never-said questions; seed 2: the same counts) 80
Table 5.23: Desks compared on the development lives at day 83, MacBook Air (E-16) 81
Table 5.24: The proof lives at week 12, three lives added (E-20) 82
Table 5.25: Errands on the proof lives, last day, 1.7B reader (E-25) 82
Table 5.26: Time to the first word by memory on the desk, median of 5 (E-20) 82
Table 5.27: The baseline's main results on public benchmarks (CAMR v1) 83
Table 5.28: Weaknesses found 84
Table 6.1: Alignment from objectives to evidence and conclusions 88
Table 6.2: Recommendations 90
Table 6.3: Prioritised future works 90
Table A.1: The records behind Chapter 5 (v2/records/bench/2026-11-04/) 96
Table B.1: Decision log entries cited in this report 98

List of Abbreviations

AI - Artificial Intelligence

API - Application Programming Interface

CAMR - Capability-Adaptive Memory Retrieval, the memory engine of this study

CPU - Central Processing Unit

DSR - Design Science Research

GB - Gigabyte (10^9 bytes); GiB - Gibibyte (2^30 bytes)

GPU - Graphics Processing Unit

HTTP - Hypertext Transfer Protocol

JSON - JavaScript Object Notation

KV - Key–Value (the keys and values a transformer layer keeps for each token it has read)

LLM - Large Language Model

LRU - Least Recently Used

MCP - Model Context Protocol

MLX - Apple's array framework for machine learning on Apple silicon

QA - Question Answering

RAG - Retrieval-Augmented Generation

RAM - Random Access Memory

RoPE - Rotary Position Embedding

SLM - Small Language Model

SSD - Solid-State Drive

TTFT - Time to First Token (the wait for the first word)

UML - Unified Modeling Language

Chapter 1: **Introduction**

1.1 **Background Information**

Language models have become the foundation of modern conversational software, powering assistants that answer questions, summarise documents and help with everyday tasks. The most capable of these models contain tens or hundreds of billions of parameters and run as cloud services on specialised hardware. Their capability comes at a cost: every query travels over a network, personal data leaves the user's device, the service fails without connectivity, and each request carries a price. Empirical work on scaling has shown that capability grows predictably with model size and training data (Kaplan et al., 2020), which is precisely why the strongest models are also the largest and the least practical to run locally.

A parallel movement has formed around small language models, typically between one and eight billion parameters, that run directly on consumer hardware such as laptops and phones. Recent small models such as Qwen2.5 (A. Yang et al., 2024), Qwen3 (A. Yang et al., 2025), Llama 3.2 (Grattafiori et al., 2024), Phi-3 (Abdin et al., 2024), and the on-device foundation models that Apple ships on its consumer chips (Gunter et al., 2024) show that useful language ability fits in the memory of an ordinary computer. On-device deployment offers real advantages: answers are generated locally with low latency, private data never leaves the machine, the assistant keeps working offline, and the marginal cost of a query is zero. A recent survey confirms both the rapid progress of small models and their persistent weaknesses relative to their larger counterparts (Lu et al., 2024).

The most consistent of those weaknesses is knowledge. A small model has not memorised as many facts as a large one. Mallen et al. (2023) showed this directly: language models struggle with less popular, long-tail facts, retrieval of external text helps significantly in exactly those cases, and making models bigger mainly improves recall of popular facts while barely improving the long tail. If much of the gap between small and large models is missing knowledge rather than missing reasoning, then supplying the right knowledge at the right time, from outside the model, can recover a meaningful share of the gap without touching the model at all.

A second weakness matters as soon as an assistant is used for long work. A model only sees what is in its context window, and on a laptop the window is limited by memory: the keys and values a transformer keeps for every token it has read grow with the history. Cloud models now read up to a million tokens at once, while a small local model with a 64,000-token window already needed 15.2 GB of RAM in this study (Section 5.8.2). A project that runs for months produces far more than either window holds, so both kinds of model forget its beginning.

This study designed, built and evaluated CAMR (Capability-Adaptive Memory Retrieval), a lightweight external memory engine that sits beside a frozen language model on a consumer laptop. CAMR was built in two versions. The first, kept in this report as the baseline (CAMR v1), stored short text notes in an embedded database and gave the model the few that matched each question; it was

evaluated on public question-answering benchmarks. The engine reported here keeps the model's own working state instead: while the model reads a moment, it writes its keys and values at every layer, and CAMR saves them to the SSD in rooms of related moments. A question opens one room, reads it in one burst and lays it in front of the model already read, so the model continues as if it had just read those moments. The model's weights are never changed. The study then measured, separately for different kinds of question, how much of the gap between a small on-device model and a large cloud model this memory closes, and at what cost in time, RAM and disk.

1.2 **Problem Statement**

Users who want a private, offline, low-cost assistant must accept a small on-device model, and with it a marked drop in capability, most visibly on questions that require knowledge the small model has not memorised and on long work whose history no longer fits in its context window. The two standard remedies do not fit the on-device setting. Replacing the small model with a larger one defeats the purpose, because the larger model no longer fits on the device. Fine-tuning the small model on new knowledge requires compute that ordinary users do not have, must be repeated whenever knowledge changes, and risks degrading the model's existing abilities.

Retrieval-augmented approaches offer a third path: leave the model frozen and supply relevant text at question time. The research literature has established the ingredients separately. Retrieval can substitute for model scale on knowledge-heavy tasks (Lewis et al., 2020; Borgeaud et al., 2022; Izacard et al., 2023). Memory systems can accumulate and organise information across long interactions (Packer et al., 2023; Zhong et al., 2024; Gutierrez et al., 2024). Prompts can be compressed so that less text carries the same meaning (Jiang et al., 2023), and a model's keys and values can be reused instead of recomputed (Gim et al., 2024). What no existing work provides is a measured answer to the question that matters for on-device deployment: when these ingredients are combined into a memory engine beside a frozen small model on a real consumer device, what fraction of the small-versus-large capability gap is actually recovered, on which kinds of tasks, and at what cost in time, RAM and storage as the memory grows? Without that answer, designers of on-device assistants cannot know whether memory engineering is a worthwhile substitute for model scale, or where its limits lie.

1.3 **Objectives**

This section states the general objective of the study and breaks it down into the specific objectives that guided the work.

1.3.1 **General Objective**

The general objective of this study was to design, implement and experimentally evaluate a lightweight external memory engine for a frozen small language model on a consumer laptop, in order to measure how much of the capability gap between on-device and cloud language models such an engine bridges, by task type, with the time and memory cost of every answer.

1.3.2 **Specific Objectives**

i. To investigate existing memory, retrieval and compression techniques for language models and establish their applicability and limitations in the on-device setting.

ii. To design a lightweight memory engine for a frozen small language model, comprising a write path that turns what the user tells it into stored memory, a read path that finds the memory a question needs, and a budget that controls how much memory the model reads.

iii. To implement the designed engine on a consumer laptop with no dedicated graphics card, so that the model and its memory run entirely on the device and the memory can grow on local storage.

iv. To evaluate the implemented engine experimentally against the same model without memory and against a large cloud model, separately for knowledge-bound and reasoning-bound tasks, together with the time to the first word, the RAM and the storage of every answer.

v. To determine how much memory the model should read, and what each component of the engine contributes, through controlled comparisons in which one choice is varied at a time.

1.4 **Research Questions**

The study sought to answer the following research questions, one for each specific objective:

i. What memory, retrieval and compression techniques exist for language models, and what are their limitations in the on-device setting?

ii. How should a lightweight engine keep and find memory for a frozen small model so that the model can use it?

iii. Can such an engine run on a consumer laptop with a memory that grows without making the model slower or heavier?

iv. How much of the capability gap between a small on-device model and a large cloud model does the engine bridge, does it help more on knowledge-bound than on reasoning-bound tasks, and at what cost?

v. How much memory should the model read, and what does each component of the engine contribute?

1.5 **Justification**

The study matters for three audiences. For users, especially in settings where connectivity is unreliable or data privacy is a serious concern, a more capable on-device assistant means useful AI that works offline, keeps personal data local and costs nothing per query. For developers of on-device assistants, the study provides what the literature lacked: a measured, component-by-component

account of how far memory engineering can substitute for model scale and for a long context window, allowing informed decisions about where to invest effort. For the research community, the study contributes an evaluation method, a scripted long project graded by rule in which every answer records what it cost, against which future memory systems can be compared.

The study was also deliberately scoped to be feasible as a final-year undergraduate project. The language models themselves were reused, not trained, so the original work lies in the design of the engine and the design of the measurement, both of which were achievable on a single consumer laptop with modest API credits for the large reference models.

1.6 **Scope**

The deployment target was a consumer laptop with no dedicated graphics card. Development began on a MacBook Air M4 with 16 GB of unified memory, and every reported measurement of the engine was made on a MacBook Pro M5 Pro with 24 GB. Both use Apple silicon, whose integrated graphics processor runs the models through Apple's MLX framework (Hannun et al., 2023); the baseline ran on CPU-only machines with 16 GB. The small models studied were open-weight Qwen3 models of 1.7, 4 and 8 billion parameters, run locally and frozen; a 27-billion-parameter model was run on the same Mac as a larger local comparison, and the Ollama runtime was paired with the engine to test memory supplied as words. The large reference models were Claude Haiku 5.5, Sonnet 5.5 and Opus 5.5, reached through a cloud API and never run locally; they answered the same questions alone and served only as the yardstick. The baseline's reference was Kimi K3.

Evaluation used English material of three kinds: a scripted project history of 2 to 4 million tokens with questions graded by rule (the Long Haul), public benchmarks from the Hugging Face Hub (MATH-500, AIME, HARP and BABILong), and, for the baseline, the public question-answering benchmarks PopQA, HotpotQA, 2WikiMultiHopQA and GSM8K. Together they separate knowledge-bound from reasoning-bound performance. The system delivered is the memory engine, its benchmarks and their records, and a native Mac application with two parts, Chat and Code, that shows the engine at work. It is a research prototype, not a commercial product.

1.7 **Limitations and Delimitations**

The limitations of the study are the constraints it operated under. Timings and memory figures were measured on one Mac and may differ on other hardware, so device results are indicative rather than universal. The cloud reference models are commercial services whose behaviour can change between versions; the study pinned the model identifiers, recorded them in every record and logged every call. The main test is a scripted project, built to be hard in known ways; it is an approximation of real work, and its findings generalise to live use only to the degree that the script represents it.

The delimitations are boundaries the study deliberately drew. No language model was trained or fine-tuned in the engine; all weights remained frozen, since the research question is precisely what memory achieves without touching the model (the baseline trained two adapters only as a

comparison). The engine was not ported to mobile phones or to computers other than Apple-silicon Macs; the laptop target preserves the on-device claim while avoiding toolchain work that adds engineering effort without research value. A graph-based associative layer in the style of Gutierrez et al. (2024) was treated as optional; the baseline implemented a light form of it as entity bridging. No study with real users was carried out. Finally, the study is limited to English text; a first test of memory for documents, images and video is reported, but multilingual memory is left to future work.

Chapter 2: **Literature Review**

2.1 **Introduction**

This chapter reviews the research that the study builds on. It first surveys the current state of the field: the divide between small and large language models, the evidence on what the capability gap consists of, retrieval as a substitute for scale, the compression of what a model reads, the reuse of a model's own keys and values, the benchmarks that evaluate memory in long interaction, and the security risks of persistent memory. It then examines specific related systems that implement memory for language models, identifies the gaps those systems leave, and closes with the conceptual framework that guided the study.

2.2 **The State of On-Device Language Models and External Memory**

This section reviews the current state of the field: the rise of small on-device models, the composition of the capability gap, retrieval as a substitute for model scale, the compression and reuse of model inputs, and the evaluation and security of persistent memory.

2.2.1 **The Rise of On-Device Models**

Scaling studies established that language model capability grows smoothly with parameter count and training data (Kaplan et al., 2020), which drove the field toward ever larger models served from the cloud. A counter-movement followed. Model families such as Qwen2.5 (A. Yang et al., 2024), Qwen3 (A. Yang et al., 2025), Llama 3.2 (Grattafiori et al., 2024), Phi-3 (Abdin et al., 2024) and Gemma 2 (Gemma Team, 2024) publish strong open-weight models at one to eight billion parameters, small enough to run in a few gigabytes of memory on an ordinary laptop. Apple has gone further and ships a roughly three-billion-parameter foundation model directly on its consumer devices, with a larger server model handling requests that exceed the on-device model's ability (Gunter et al., 2024). Apple's MLX framework runs such models on the integrated graphics processor of Apple silicon, in memory shared with the CPU (Hannun et al., 2023). A survey by Lu et al. (2024) documents both the speed of this progress and the consistent finding that small models remain clearly weaker than large ones, particularly on knowledge-intensive tasks.

2.2.2 **What the Capability Gap Is Made Of**

The most important evidence for this study concerns the composition of the gap. Mallen et al. (2023) probed language models of many sizes on factual questions whose subjects ranged from very popular to very obscure, using a dataset they constructed for the purpose, PopQA. Three findings stand out. First, models of every size struggle on long-tail facts, and small models fail on them severely. Second, supplying retrieved text helps substantially on exactly those questions. Third, scaling the model up mainly improves recall of popular facts and barely improves the long tail at all. In other words, a large part of what users buy with model scale is memorised knowledge, and memorised knowledge is the one ingredient that can also be supplied from outside the model. Reasoning ability, by contrast,

appears to be more deeply tied to scale, which is why this study treated knowledge-bound and reasoning-bound tasks separately and predicted different outcomes for each.

A long context window does not remove the need for memory. Liu et al. (2024) showed that models use information at the start and end of a long context far better than information in its middle, so text that is present is not necessarily used. Kuratov et al. (2024) built BABILong, which hides simple facts in book-length text, and found that models use only a fraction of their nominal window well. Both results suggest that what a model is given to read should be small and relevant, not merely long.

2.2.3 **Retrieval as a Substitute for Scale**

A line of research spanning several years has demonstrated that letting a model read relevant external text at answer time can substitute for parameters. Retrieval-augmented generation showed that a model paired with a dense retriever over a document store outperforms much larger purely parametric models on knowledge-intensive tasks (Lewis et al., 2020), building on dense passage retrieval, which finds relevant text by comparing embedding vectors rather than keywords (Karpukhin et al., 2020). Borgeaud et al. (2022) made the substitution explicit: their RETRO model, augmented with retrieval from a trillion-token database, matched the performance of models with twenty-five times more parameters on language modelling benchmarks. Izacard et al. (2023) showed with Atlas that a retrieval-augmented model of eleven billion parameters could outperform a model fifty times its size on knowledge tasks. Together these results establish the premise this study depends on: parameters spent on memorising facts can, to a substantial degree, be replaced by retrieval, and the saving is largest precisely where small models are weakest. What these systems do not address is the on-device setting, where the retrieval corpus, the index and the model must all fit on a consumer machine and every token read costs measurable time.

2.2.4 **Compressing What the Model Reads**

Every token a model reads costs time on a device, so the density of stored memory matters as much as its relevance. Research on prompt compression has shown that the text supplied to a model carries substantial redundancy. Jiang et al. (2023) demonstrated with LLMLingua that prompts can be compressed many times over, by removing low-information tokens, with little loss in the quality of the model's output. A complementary line compresses context into learned representations: Chevalier et al. (2023) taught models to summarise long contexts into compact vectors that stand in for the original text, and Ge et al. (2024) trained an in-context autoencoder that condenses a context window into a short sequence of memory slots. The lesson the baseline took was that memory should be stored densely, as short notes rather than verbatim transcripts, so that a fixed token budget carries the most useful information. Compression had so far been applied to prompts and contexts at inference time, not to the design of a persistent memory store, and never with the question of how it interacts with closing the capability gap on a device.

2.2.5 **Reusing the Model's Own Keys and Values**

A transformer layer keeps, for every token it has read, a key and a value for each attention head; later tokens attend to them (Vaswani et al., 2017). This key–value cache is normally thrown away when a request ends, and building it is most of the cost of reading a long prompt. Several lines of work keep it instead. Transformer-XL carried the previous segment's hidden states into the next so that a model could attend beyond its segment (Dai et al., 2019). Memorizing Transformers stored past keys and values in an external memory and let one layer attend to the nearest of them, improving language modelling over long documents (Y. Wu et al., 2022). Serving systems page the cache in blocks so that many requests share memory efficiently (Kwon et al., 2023). Prompt Cache reuses the attention states of prompt modules that recur across requests (Gim et al., 2024), and CacheBlend combines the caches of separately processed retrieved passages, recomputing a small share of tokens so that the passages can attend to each other (Yao et al., 2025).

Two facts from this work shaped the engine. First, a saved cache is exact: laid back in front of the model, it gives the same result as reading the text again, at a fraction of the time. Second, caches made separately do not combine for free, because passages written apart never attended to each other; this is the problem CacheBlend works around. Keys also record each token's position as a rotation (Su et al., 2024), so a cache moved to a new position must be turned. None of these systems uses a saved cache as a persistent personal memory on a laptop, and none measures what such a memory costs in storage as it grows.

2.2.6 **Evaluation and Risks of Persistent Memory**

Benchmarks for conversational memory have matured recently and informed this study's evaluation design. Maharana et al. (2024) introduced LoCoMo, a benchmark of very long multi-session conversations, and found that models, even with long context windows or retrieval, lag far behind human recall of conversation history. D. Wu et al. (2024) introduced LongMemEval, five hundred questions embedded in long user–assistant histories testing information extraction, multi-session reasoning, temporal reasoning, knowledge updates and abstention, and found that commercial assistants suffer accuracy drops of around thirty percent when information must be remembered across sustained interaction. They also decompose memory design into indexing, retrieval and reading stages, a decomposition the engine mirrors in its write path, read path and budget. These benchmarks establish that memory in long interaction is an unsolved problem even for large cloud systems; neither, however, measures memory as an instrument for closing the small-versus-large capability gap, which is this study's concern.

A memory store that accepts writes from interaction is also an attack surface, and recent security research documents the risk concretely. Chen et al. (2024) showed with AgentPoison that an agent's long-term memory or retrieval store can be poisoned with a small number of crafted records, so that queries containing an optimised trigger retrieve the malicious records and steer the agent's behaviour while benign queries are left untouched. Dong et al. (2025) went further with MINJA, demonstrating that an attacker with ordinary query access, and no privileged write access at all, can inject malicious

records into an agent's memory purely through crafted interactions. These findings motivated two design choices: every stored memory carries its source and date, and the baseline screened each note before storing it. A full security evaluation was beyond the study's scope.

2.3 **Related Works**

This section examines specific systems that implement memory for language models, from operating-system-inspired context management to brain-inspired associative retrieval and production memory layers. Table 2.1 summarises the comparison.

2.3.1 **MemGPT**

Packer et al. (2023) framed the memory problem in operating-system terms. The model's limited context window is treated as fast main memory, an external store as disk, and the system pages information in and out as the conversation demands, allowing a fixed-context model to sustain interactions far longer than its window. The framing is influential because it makes explicit that what the model reads at each step is a policy decision, not a given, which is exactly the decision this study's budget controls; the engine's movement of rooms between RAM and the SSD follows the same idea at the level of saved tensors. MemGPT manages context length, however; it does not target capability gain, and it does not account for token cost or device latency. Figure 2.1 illustrates the two-tier architecture.

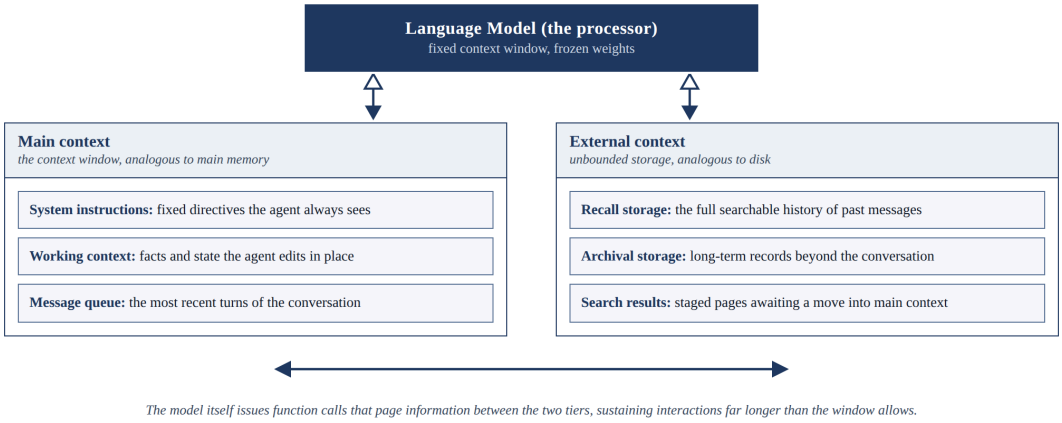


Figure 2.1: The MemGPT virtual context architecture, adapted from Packer et al. (2023)

2.3.2 **Generative Agents**

Park et al. (2023) gave simulated agents a memory stream of timestamped observations and retrieved from it by combining three scores: relevance to the current situation, recency and importance. Their generative agents produced believable long-running behaviour in a simulated town, and their three-part retrieval score was adopted by the baseline's retrieval policy. The system was designed for simulation rather than question answering, and its memory records are verbose natural-language entries, which makes it an instructive design source but not an answer to the efficiency questions a device imposes. Figure 2.2 shows the simulated town in which the agents live, work and remember.



Figure 2.2: The Smallville environment of Generative Agents (Source: Park et al., 2023, project repository)

2.3.3 **MemoryBank**

Zhong et al. (2024) added a model of forgetting. MemoryBank strengthens or decays stored memories over time in the spirit of the Ebbinghaus forgetting curve, so that what the system retains reflects both how often a memory is used and how old it is. The contribution is a principled answer to what should be kept, complementing the question of what should be retrieved; the engine's vault applies the same idea, letting a note's strength fade each day it is unused. Like the systems above, MemoryBank is evaluated on the quality of long-term companionship behaviour rather than on capability gain, token cost or latency. Figure 2.3 reproduces the framework figure from the project's repository.

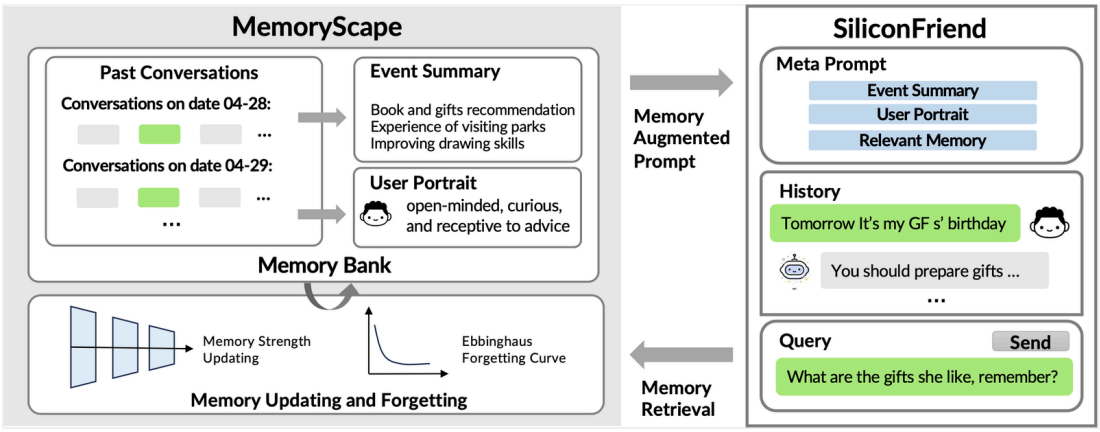


Figure 2.3: The MemoryBank framework with memory updating and forgetting (Source: Zhong et al., 2024, project repository)

2.3.4 **HippoRAG**

Plain similarity search retrieves passages that resemble the question, but some questions require connecting facts that do not resemble the question at all. Cognitive science offers a model for this in

spreading activation, where recalling one concept activates linked concepts (Collins & Loftus, 1975), and in the hippocampal indexing theory, where a compact index links the distributed traces of an experience so that touching one part can reactivate the whole (Teyler & DiScenna, 1986). Gutierrez et al. (2024) operationalised these ideas in HippoRAG: a language model extracts entities from stored passages into a knowledge graph that acts as the index, and at question time the entities in the query seed a Personalized PageRank walk over the graph, spreading relevance to connected passages, including multi-hop neighbours that similarity search misses. Figure 2.4 illustrates the pipeline. HippoRAG outperformed strong retrieval baselines on multi-hop question answering by up to twenty percent while being many times cheaper and faster than iterative retrieval methods. Its relevance to this study is twofold: it demonstrates the value of associative structure in memory, and its computational profile, graph construction by a language model and a graph walk per query, illustrates the kind of cost that must be justified against a device budget. The engine's rooms are a lighter version of the same index: a compact grouping of related moments that one question reactivates together.

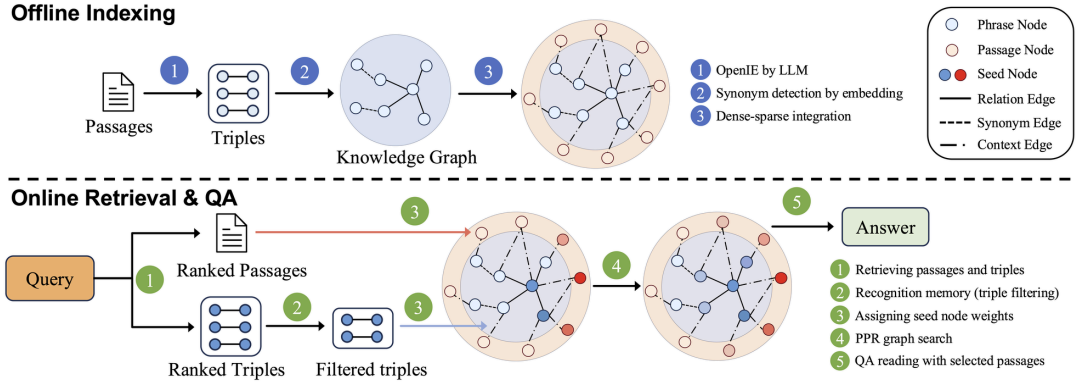


Figure 2.4: HippoRAG offline indexing and online retrieval (Source: Gutierrez et al., 2024, project repository)

2.3.5 **Mem0**

Production systems have begun packaging these ideas. Mem0 extracts and consolidates salient facts from conversations into a managed store and reports large token savings relative to feeding a model its full history (Chhikara et al., 2025). Its existence is evidence that the engineering direction is commercially credible. As a cloud service, however, it does not answer the on-device question, and like the research systems it does not measure gap closing per task type. Figure 2.5, drawn from the project's documentation, shows where the memory layer sits in an agent stack, reading at question time and writing after each response.

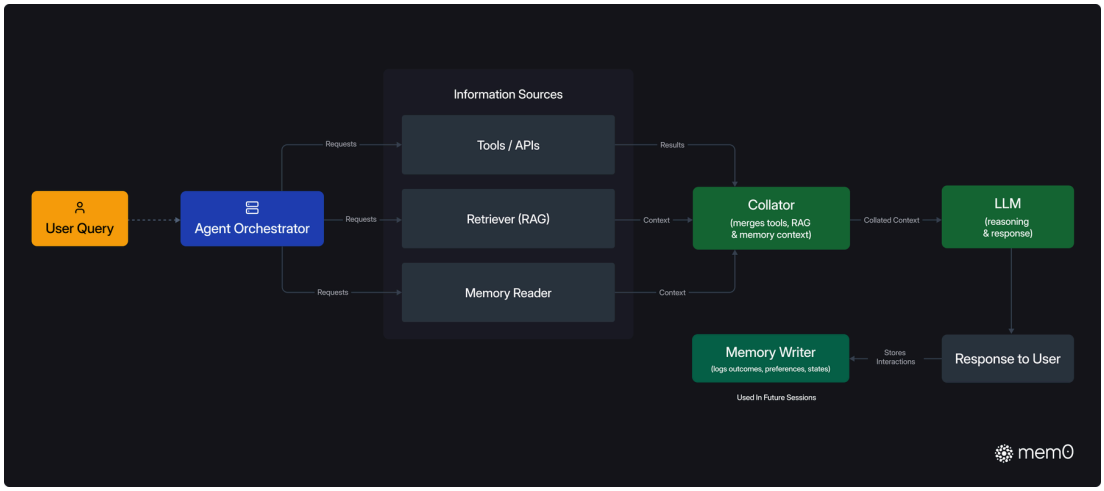


Figure 2.5: The Mem0 memory layer within an agent stack (Source: Mem0 project documentation)

Table 2.1: Comparison of representative memory systems for language models

System	**Core idea**	**Strength**	**Limitation for the on-device setting**
MemGPT (Packer et al., 2023)	Pages context in and out of the window like an operating system	Sustains very long interactions on a fixed window	Manages context length; does not target capability gain or token cost
Generative Agents (Park et al., 2023)	Memory stream scored by relevance, recency and importance	Simple, effective retrieval scoring	Designed for simulation; verbose natural-language records
MemoryBank (Zhong et al., 2024)	Memory strengthening and decay over time	Principled forgetting	No accounting of token cost or device latency
HippoRAG (Gutierrez et al., 2024)	Knowledge-graph index with Personalized PageRank retrieval	Strong multi-hop recall in a single retrieval step	Graph construction and walk are heavy for a device budget
Mem0 (Chhikara et al., 2025)	Extraction and consolidation of salient facts into a managed store	Production-ready; large token savings reported	Cloud service; gap closing per task type not measured
Memorizing Transformers (Y. Wu et al., 2022)	Past keys and values in an external memory, read by k-nearest-neighbour attention	Exact reuse of the model's own state over long documents	Needs a trained model; memory held in accelerator memory, not on a laptop's SSD

2.4 **Gaps in Related Works**

The reviewed literature establishes each ingredient separately. Retrieval can substitute for model scale on knowledge-heavy tasks (Lewis et al., 2020; Borgeaud et al., 2022; Izacard et al., 2023), and it helps most on exactly the long-tail knowledge where small models fail (Mallen et al., 2023). Memory systems can accumulate, organise, score and forget information across long interactions (Packer et al., 2023; Park et al., 2023; Zhong et al., 2024; Gutierrez et al., 2024; Chhikara et al., 2025). Text supplied to a model can be heavily compressed with little loss (Jiang et al., 2023; Chevalier et al., 2023; Ge et al., 2024), and a model's own keys and values can be saved and reused (Y. Wu et al., 2022; Gim et al.,

2024; Yao et al., 2025). Benchmarks exist that expose how poorly current systems remember long interactions (Maharana et al., 2024; D. Wu et al., 2024; Kuratov et al., 2024).

What was missing is the combination and the measurement. No existing work assembles these ingredients into a memory engine beside a frozen small model on a consumer device and then answers the deployment question directly: what fraction of the small-versus-large capability gap does the engine close, separately for knowledge-bound and reasoning-bound tasks, and at what cost in time, RAM and storage on the device as the memory grows past any context window? Existing memory systems are evaluated on recall or task success, not on gap closing; key–value reuse is evaluated as a serving optimisation, not as a persistent memory; and no study reports honest per-answer cost accounting on consumer hardware. This intersection defines the gap this study addressed.

2.5 **Conceptual Framework**

Figure 2.6 presents the conceptual framework that structured the study, as it was set out at the proposal stage. The independent variable is the configuration of the memory engine: how what the user tells it is kept (the write policy), how a question finds what it needs (the retrieval policy), and how much memory the model reads (the budget). The process is question answering on the device: the engine recalls relevant memory, assembles it within the budget, and the frozen model generates an answer from the question and the assembled memory. The dependent variables are the share of the capability gap closed, the accuracy gained per token of memory supplied, and the end-to-end latency on the device. Task type, knowledge-bound against reasoning-bound, moderates the relationship: substantial gap closing was predicted on knowledge-bound questions and little on reasoning-bound ones. The language model itself is held constant throughout, weights frozen, so that every measured gain is attributable to the memory engine alone.

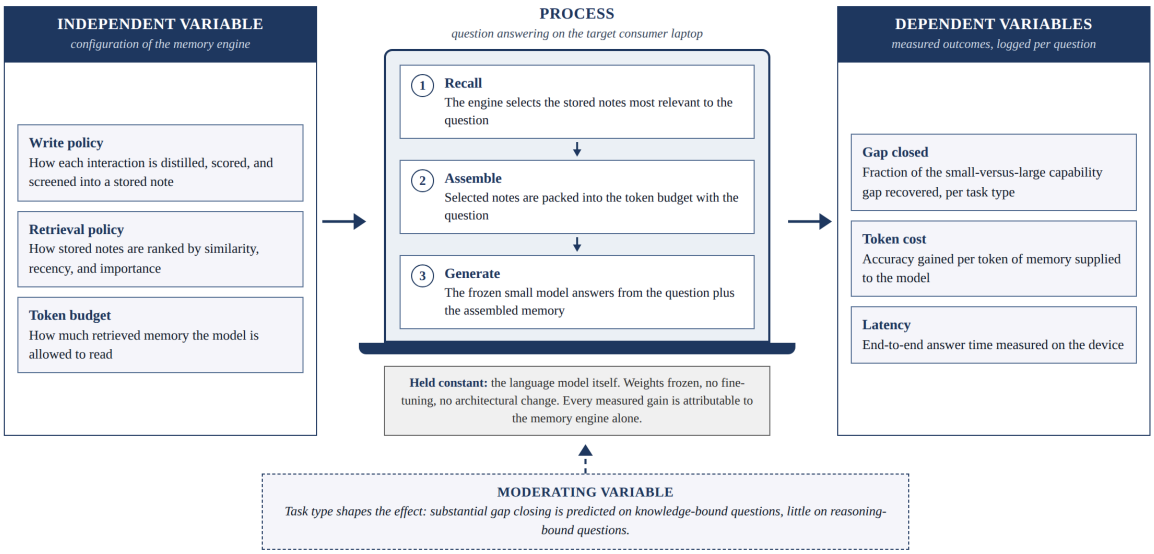


Figure 2.6: Conceptual framework of the study

The framework held for the engine as built, with one change in what each variable means. In the final engine the write policy keeps a moment as the model's notes in a room, the retrieval policy chooses one room and the closest moments within a reach, and the budget is the size of a room plus the moments re-read. The dependent variables gained two measures that matter once memory grows: the RAM of the whole process and the storage the memory takes on the SSD.

Chapter 3: **Methodology**

3.1 **Introduction**

This chapter describes how the study was carried out. It states the research paradigm that guided the work, the development approach that was followed, the analysis of what the system had to do, the design of both the system and the experiments that evaluated it, the tools and techniques that were used, and the deliverables the study produced. Throughout, the guiding principle was that the measurement apparatus was built and validated before the system it measures, so that every later design decision could be judged as a controlled experiment. Chapter 4 gives the full requirements and design, and Chapter 5 the implementation and results.

3.2 **Research Paradigm**

The study adopted a pragmatist research philosophy, combining design science research with controlled experimentation. Design science research is appropriate because the study created and evaluated a novel artefact, the memory engine, and the discipline's guidelines require that such an artefact be evaluated rigorously against explicit measures of utility (Hevner et al., 2004). The study followed the design science research methodology of Peffers et al. (2007), proceeding through problem identification, definition of solution objectives, design and development, demonstration, evaluation and communication. The evaluation took the form of controlled experiments organised as a research pipeline of four stages, described in the subsections that follow.

3.2.1 **Data Acquisition**

Evaluation data came from three sources, chosen so that knowledge-bound and reasoning-bound performance could be separated.

i. **Public question-answering benchmarks** for the baseline: PopQA for single-hop long-tail facts (Mallen et al., 2023), HotpotQA and 2WikiMultiHopQA for multi-hop facts (Z. Yang et al., 2018; Ho et al., 2020) and GSM8K for mathematical reasoning (Cobbe et al., 2021), with the document collections that accompany the factual benchmarks.

ii. **A scripted long project, the Long Haul,** written for this study because no public set tests a memory that grows over months. It is generated from a seed, so the same seed always gives the same script, and every planted fact has one answer a rule can check (Section 5.3.1).

iii. **Public suites from the Hugging Face Hub** for the engine's comparison with the cloud yardstick: MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024), AIME 2025 and 2026 (Balunović et al., 2025), HARP correction pairs (Yue et al., 2024) and BABILong (Kuratov et al., 2024).

All public datasets are released for research use and were downloaded from their official distributions. Earlier development of the engine also used three scripted twelve-week lives of invented people

(Section 5.8.15).

3.2.2 **Data Preparation**

From each public benchmark, a fixed random sample of questions was drawn once, with a fixed seed, and reused across all conditions so that every comparison is paired. The baseline's memory was populated from the supporting document collections only, never from the question–answer pairs, so the experiment measured the engine's ability to surface relevant knowledge rather than to look up a planted answer. Distractor documents were included, so that retrieval had to discriminate, not merely fetch.

The Long Haul was prepared the same way. It was cut at fixed sizes (64,000, 256,000, 1 million and 2 million tokens, and after an extension 3 and 4 million). At each cut a checkpoint draws a stratified sample of the questions that can be asked there, ten from each family with a fixed seed, so every contender answers exactly the same 216 questions (E-30; decisions are cited by their number in the project's decision log, summarised in Appendix B). Questions are asked only after the history has been told, and asking never teaches: the engine's questions are read-only, so a question asked early does not help the same question asked later (E-31).

3.2.3 **Experimental Conditions**

The baseline ran three conditions on every question: a floor, in which the small local model answered alone; a ceiling, in which the large cloud model answered alone; and a treatment, in which the small model answered with the memory engine. The engine's evaluation kept the same logic but named the conditions by what each contender can see (Table 3.1): the model alone with the newest part of the history that fits its window ("bare"), the same model with the engine, and the cloud model alone with the newest part that fits its one-million-token window. Every contender read the same instructions, so that only what each one could see differed (E-32).

3.2.4 **Measurement and Validation**

Every answer in every condition was logged with the model, the engine's settings, the tokens read and written, the time to the first word, the total time, the retrieval time of each step, where the memory came from (RAM, SSD or words) and the RAM and disk in use. Answers were graded by rule, so the same answer always gets the same verdict and anyone can grade again. Each run wrote a dated record naming the machine, how busy it was and a fingerprint of the code that made it. Validity was protected by the paired design, by fixed seeds, by one measuring job at a time on the graphics processor (E-35), and by the threat mitigations in Section 3.5.6.

3.3 **Development Approach**

The system was developed incrementally, in small working stages, each evaluated before the next began. An incremental approach suited the study for two reasons. First, the central research instrument, the evaluation harness, had to exist before the engine, because every component of the engine was judged by its measurements; building measurement first removed the largest project risk,

which is discovering late that results cannot be produced. Second, the engine was designed as a chain of separable components, so adding them one at a time turned development into the comparisons that the fifth objective required.

Development ran in five increments.

i. **The baseline (CAMR v1).** The harness for the public benchmarks was built first and validated by a baseline gap measurement with no memory; then the text-note engine was added one component at a time (similarity retrieval and a token budget, then gating, entity bridging, verified answering and learning by lookup), each producing a measured row. Its results are summarised in Section 5.8.16.

ii. **Step 0, the ruler.** Before the new engine existed, three scripted lives, their quiz and the baselines that would judge it were built and measured on the MacBook Air (E-01 to E-13).

iii. **Step 1, rooms and notes.** The engine's first build stored the model's keys and values in rooms on the SSD; the room design was chosen on development lives, and pass marks were written before the held-out lives were run (E-14 to E-20).

iv. **Step 2, the event log.** Each answer's attention over the moments on the desk was recorded so that an answer could be replayed and inspected (E-21).

v. **The final engine and its proof.** The core, the tiers and sleep, the vault, reach, metrics and the Ollama pairing were added, and the engine was measured on the Long Haul against the Claude yardstick (E-29 to E-43).

3.4 System Analysis

This section analyses what the system had to do. The analysis studied the requirements that the research questions impose, since in this study the experiment itself is a primary stakeholder: every requirement traces to a measurement the study had to make. Users of the application are the second stakeholder. Chapter 4 lists every requirement with its source and verification method.

3.4.1 Functional Requirements

The system had to keep what it is told as stored memory, with the date and source of each item; store memory durably on the device; find, for an incoming question, the memory it needs; assemble that memory within a set budget and give it, with the question, to a frozen local model; let the user turn memory off and choose whose memory a question reads; and log, for every answered question, the model, the engine's settings, the tokens supplied and generated, the time of each step and the answer, so that every experiment can be reproduced from its records. The evaluation harness had to generate or load each test set deterministically, run each contender on identical questions, grade each answer by rule and write a record after every checkpoint.

3.4.2 Non-Functional Requirements

The system had to run entirely on the target laptop, with the sole exception of the cloud reference models, which are reached through an API and are not part of the engine. Finding memory had to add only a small fraction of the answer's time, and the time to the first word and the RAM in use had not to grow as the memory grew. All stored data had to remain on the device, and the engine had to work without a network once its models were downloaded. Spending on the cloud reference had to be capped and logged call by call. The whole experimental pipeline had to be runnable from documented commands, so that results can be regenerated and verified.

3.5 **System Design**

This section describes the design of the memory engine and, with equal weight, the design of the experiments that evaluated it.

3.5.1 **Architectural Overview**

The engine has three parts arranged around a frozen local language model. The write path runs when the user tells it something: the moment is embedded, placed in the room of its closest earlier moment if that moment is close enough and the room has space, and read once by the model with the room's earlier moments in view; the keys and values the model writes while reading are appended to the room's file on the SSD. The store is a folder of files: one file of notes per room, an index of every moment's words, day and place, a matrix of embeddings, and a vault of plain-text notes. The read path runs when a question arrives: the question is embedded, the moment closest to it is found, and that moment's room is laid in front of the model already read, after a small core of standing facts; the model then reads the four closest moments as words, the date and the question, and answers. Rooms used recently stay in RAM; others are read from the SSD in one burst; when the room files outgrow their disk budget, the weakest rooms go to sleep and are read as words. The model's weights and inference code are never changed. The baseline's architecture, a single SQLite file with a ranked, gated and bridged read path, is described in Chapter 4 for comparison.

3.5.2 **Experimental Design**

The central quantity of the baseline's evaluation was the fraction of the capability gap closed, computed for each task type by Equation 3.1, where S is the accuracy score of a condition on that task type.

$$\text{gap closed} = \frac{S_{\text{treatment}} - S_{\text{floor}}}{S_{\text{ceiling}} - S_{\text{floor}}} \tag{3.1}$$

A value of zero means the engine added nothing and a value of one means the small model with memory matched the large model. The engine's evaluation reports right answers out of the questions asked at each size of history, for each contender, because the ceiling itself changes with the size of the history: a cloud model that sees the whole history at 256,000 tokens sees only its last part at 2 million. Table 3.1 summarises the conditions.

Table 3.1: Experimental conditions

Condition	Model	Memory	Purpose
Floor (baseline) or bare (engine)	Small local model (0.5–3B for the baseline; Qwen3 1.7B, 4B, 8B for the engine)	None: the newest part of the history that fits its window	Lower anchor
Ceiling (baseline) or yardstick (engine)	Kimi K3 (baseline); Claude Haiku, Sonnet and Opus 5.5 (engine)	None: the newest part that fits its window	Upper anchor; the yardstick is not a contender with memory
Treatment	The same small model as the floor	CAMR v1 (baseline) or CAMR (engine), settings varied per experiment	Measures what each configuration adds
Memory as words	Qwen 27B; Qwen3 1.7B through Ollama	The engine's memory, chosen the same way, pasted in as text	Separates what the memory knows from how it is delivered

One design guard was enforced throughout: the questions were held out of what built the memory, so the experiment measured the engine's ability to surface relevant knowledge rather than to look up a planted answer. In the Long Haul a probe is planted early and asked late, and its right answer is resolved for the point at which it is asked.

3.5.3 **Datasets and Task Types**

The datasets were assigned to task types as in Table 3.2. A null result on a reasoning task was treated as a positive finding, since it would show that the engine does what it claims, supplying knowledge, and not what it does not claim.

Table 3.2: Evaluation datasets and the task type each represents

Dataset	Task type	Why it was included
PopQA (Mallen et al., 2023)	Knowledge-bound, single hop	Long-tail facts where small models fail severely (baseline)
HotpotQA (Z. Yang et al., 2018); 2WikiMultiHopQA (Ho et al., 2020)	Knowledge-bound, multi-hop	Answers combine facts across stored passages (baseline)
GSM8K (Cobbe et al., 2021)	Reasoning-bound (control)	Memory is predicted to help little (baseline)
The Long Haul (this study)	Both: facts, changed facts, look-alikes, two hops, corrected maths, code, a page, things never said	A memory that grows past every window; each family isolates one way of failing
BABILong (Kuratov et al., 2024)	Knowledge-bound, long context	Facts hidden in book text from 4,000 to 1 million tokens
HARP correction pairs (Yue et al., 2024)	Reasoning with a worked correction in memory	Whether a remembered worked solution carries to a new problem
MATH-500 (Lightman et al., 2024); AIME (Balunović et al., 2025)	Reasoning-bound, no memory	What each model can do on its own

3.5.4 **Evaluation Metrics**

Answer quality was scored with each benchmark's own rule: exact match and token-level F1 for the baseline's question-answering sets, the final answer for the mathematics sets, and, for the Long Haul, five verdicts given by rule (Table 5.5). An answer about something never told is right only if it declines; a guess is counted as a false memory, separately from facts known, so that declining cannot raise a score. Efficiency was reported as the time to the first word, the total time, the retrieval time of each step, the tokens read, the peak RAM, the disk taken by the memory and, for the cloud models, the dollars spent at list price. Medians and the 90th percentile are reported, since a few long answers would distort a mean.

3.5.5 **Comparison Procedure**

Each component of the engine was evaluated by changing it alone and measuring the result, holding everything else fixed. For the baseline the components were the write policy, recency and importance in the ranking, gating, entity bridging and the token budget. For the engine they were how notes are written and laid out (one moment at a time or a room at a time; one room or three), the size of a room and the number of moments re-read, the instructions the model reads, memory as notes against the same memory as words, the reach (session, project, global, project with global) and the cap on the graphics buffer cache. Each comparison is reported as a row stating the change in right answers and the change in time and memory.

3.5.6 **Threats to Validity**

The study guarded against the following threats. Answer leakage was prevented by building memory only from what the history tells, never from the questions. Prompt sensitivity was reduced by giving every contender the same instructions, pinned byte for byte by a test (E-32); the one exception, the 1.7B model's own wording, is reported both ways and was not chosen after seeing results on the test questions (E-37). Sampling error was addressed by paired comparisons on identical questions; the Long Haul's samples are small (34 to 73 questions per checkpoint), so differences of one or two answers are read as level. Timing noise was reduced by running one measuring job at a time, recording the machine's load with every record, and reporting medians. Instability of the cloud reference was handled by pinning model identifiers and recording the model that answered each request; no fallback model was ever used. Scorer error in grading by rule was checked by keeping every raw answer in the records so that any verdict can be checked by hand.

3.6 **Tools and Techniques**

This section describes the tools that were used to build the engine and the harness, and the role each played.

3.6.1 **Python**

The engine, the harness and the analysis were implemented in Python 3.12, the standard language of the research ecosystem the study draws on, with NumPy for vectors and the scientific Python stack for analysis and figures.

3.6.2 **MLX and Local Models**

The engine runs its models through MLX on the Apple-silicon graphics processor (Hannun et al., 2023), because it must read and write the model's key–value cache inside its own process. The models were 4-bit Qwen3 1.7B, Qwen3 4B Instruct 2507 and Qwen3 8B (A. Yang et al., 2025), with Qwen 27B (`Qwen3.8-27B-4bit`) as the larger local comparison. The baseline served Qwen2.5, Llama 3.2 and Phi models through Ollama, which was also used for the engine's pairing test.

3.6.3 **Hosted Inference APIs**

The cloud yardstick was Claude Haiku 5.5, Sonnet 5.5 and Opus 5.5, reached through the Anthropic API with pinned model identifiers, streaming, and a ledger that logs every call's tokens and cost against a hard cap of $60 for a night of runs. The baseline's ceiling was Kimi K3 through the Moonshot API.

3.6.4 **Embedding Model**

Moments are embedded by Qwen3-Embedding-0.6B (Zhang et al., 2025), run through MLX, which serves every contender so that differences between contenders are differences between memories (E-02). The baseline used the compact BGE embedder (Xiao et al., 2024).

3.6.5 **Storage on the Device**

The baseline stored notes, vectors and logs in one SQLite file with a vector-search extension, as proposed. The engine instead stores one append-only file per room on the SSD, read and written with the operating system's file cache bypassed so that every read is a real SSD read, beside a JSON index, a NumPy embedding matrix and a folder of Markdown notes. Chapter 4 (Table 4.9) gives the reason for the change.

3.6.6 **Application and Auxiliary Tools**

The application's front end is a native macOS application written in SwiftUI, which starts the engine as a local process and talks to it over the loopback address. Version control used Git and GitHub, with the tests that need no MLX run by GitHub Actions on every push; pytest ran the test suite; a headless Chromium checked generated web pages. All results were written as structured JSON records, so every figure and table in this report can be regenerated from them.

3.7 **Deliverables**

The study produced the following deliverables.

3.7.1 **The CAMR Memory Engine**

A documented, installable Python package implementing the write path, the store, the read path, the three layers, reach and metrics described in Chapter 4, runnable entirely on an Apple-silicon Mac, with a pairing that gives Ollama memory as words.

3.7.2 **The Evaluation Harness**

The Long Haul generator and grader, the runners for the engine, the bare models, Ollama and the Claude yardstick, the Hugging Face suites, and the baseline's benchmark pipeline, each reproducible from documented commands.

3.7.3 **Experimental Results**

The measured results of the engine and the baseline, with the dated records that support every number.

3.7.4 **The Application and System Documentation**

The native Mac application with its Chat and Code parts, the user guide, the decision log and this report, including the instructions sufficient for an independent reader to reproduce the results (Appendix A).

3.8 **Ethical Considerations**

The study involved no human participants and no personal data. The baseline used established public benchmarks released for research use and an invented company. The engine was evaluated on scripted histories of invented people and an invented business, with Kenyan names and places; the only text sent to a cloud service was this scripted material, sent to the Claude yardstick under the provider's terms. All local models are open-weight models run on the Mac. Results are reported as they came out, including negative results and requirements that were not met. Cloud spending was metered call by call and capped. No user study was run, so no ethics approval was required; one would be needed before any study with real users (Chapter 6).

Chapter 4: **System Analysis and Design**

4.1 **Introduction**

Chapter 3 adopted a pragmatist philosophy that combines design science research with controlled experimentation (Hevner et al., 2004; Peffers et al., 2007), and an incremental approach in which the measurement apparatus is built before the artefact it measures. This chapter applies that paradigm to CAMR (Capability-Adaptive Memory Retrieval). CAMR is an AI artefact judged by experiment, not a transactional information system, so the chapter uses a justified hybrid of views: a structured-analysis context diagram for the boundary; UML use case, activity, component, sequence, state and deployment diagrams for actors, the evaluation, the parts, one question, the life cycle of memory and the hardware; and storage views for the files and the baseline's database. The diagrams are drawn in standard notation (UML 2.5 and Gane–Sarson) from generator scripts kept with this report, so they can be redrawn and checked against the code.

CAMR was built in three stages (Table 4.1). The baseline, CAMR v1, stores short text notes in one SQLite file beside a frozen model and was evaluated on public benchmarks. The engine's first build (the rooms build, 2–3 November 2026) replaced text with the model's own keys and values, saved in rooms on the SSD, and was evaluated on scripted twelve-week lives. The final engine (4 November 2026) added a core that stays in RAM, tiers that move rooms between RAM and the SSD and put them to sleep, a vault of linked plain-text notes, reach, per-answer metrics and a pairing with Ollama, and was evaluated on the Long Haul against the Claude yardstick. This chapter describes the final engine and keeps the baseline's design where the evaluation still uses it. Decisions from the project's decision log are cited as E-nn (Appendix B).

Section 4.2 states and traces the requirements. Section 4.3 presents the analysis diagrams: the context diagram (Figure 4.1), the use case diagram (Figure 4.2) and the evaluation pipeline (Figure 4.3). Section 4.4 presents the design: the baseline's architecture (Figure 4.4), the engine's architecture (Figure 4.5), one question (Figure 4.6), the life of a room (Figure 4.7), the baseline's schema (Figure 4.8), the engine's files (Figure 4.9), the deployment on the Mac (Figure 4.10) and the application. Section 4.5 justifies the changes from the proposal and Section 4.6 summarises.

Table 4.1: The three stages of CAMR

Aspect	**CAMR v1: the baseline**	**The rooms build (2–3 Nov)**	**CAMR: the final engine (4 Nov)**
A memory is	A text note with embedding, importance, timestamps and source	A dated moment: its words, its embedding and the reader's keys and values at every layer	The same moment, also kept as a linked Markdown note in a vault
Store	One SQLite file with sqlite-vec	One append-only file per room on the SSD, a JSON index, a NumPy vector file	The same, plus tiers (RAM, SSD, asleep), a vault and one folder per reach

Read path	k nearest notes, gate, entity bridging, token budget	One room as notes, plus the four closest moments as words	The same desk after a core of standing facts; the room comes from RAM, the SSD or, when asleep, as words
Frozen reader	0.5B–3B models through Ollama; 22 models in the size sweep	Qwen3-1.7B through MLX	Qwen3 1.7B, 4B or 8B through MLX; Qwen 27B and Ollama read the memory as words
Embedder	bge-small-en-v1.5	Qwen3-Embedding-0.6B	Qwen3-Embedding-0.6B
Cloud model	Kimi K3: ceiling and escalation	None	Claude Haiku, Sonnet and Opus 5.5, as a yardstick only
Evaluation	PopQA, HotpotQA, 2WikiMultiHopQA, GSM8K; gap closed	Three scripted 12-week lives; weekly quiz; errands	The Long Haul (2–4M tokens); Hugging Face suites; reach; speed and RAM as memory grows
Interfaces	CLI, CAMR Personal web app, inspector, n8n gateway	One local web page	A native Mac app (Chat and Code), a local web app, the Ollama pairing, an OpenAI-compatible endpoint, a Python library
Machines	CPU-only, 4-vCPU 16 GB containers	MacBook Air M4 16 GB, then MacBook Pro M5 Pro 24 GB	MacBook Pro M5 Pro 24 GB

4.2 **System Requirements**

The system requirements identified for this project are presented in the following subsections. They were derived by working backwards from the objectives (Section 1.3.2) and the measurements the evaluation needs (Section 3.2.4), then revised against what building the three stages showed. Following ISO/IEC/IEEE 29148:2018, each has an identifier, states one thing, and names its source and verification method.

- **Priority:** MoSCoW (Must, Should, Could).
- **Source:** objective (i–v); the stage it applies to (v1 for the baseline, E for the engine); and where the requirement or threshold is recorded: the proposal (P §), the decision log (E-nn), the baseline's findings log (Fnn) or the code.
- **Verification:** inspection (I), demonstration (D), automated test (T) or measurement (M).

No threshold was invented. Every number comes from the proposal, a design constant in the code, or a pass mark written in the decision log before the run that tested it (E-19); otherwise the requirement is a comparison or a behaviour that can be shown. The stakeholders are the user, who needs private, offline and honest answers; the researcher, who needs reproducible measurements; and the supervisor and examiners, who need traceable evidence.

4.2.1 **Functional Requirements**

The functional requirements fall into five groups, listed in full in Table 4.2:

- **The baseline's paths (FR-01 to FR-06).** Documents become screened, embedded notes; a question's nearest notes are ranked, gated, bridged and packed into

the model's prompt.

- **Keeping memory (FR-07 to FR-11).** A moment is read once by the frozen model, placed in a room and appended to the room's file; it is also written to the vault; rooms move between tiers under budgets.
- **Answering (FR-12 to FR-17).** A question's room and closest moments are laid on the desk after the core, within the reaches the session reads; memory can be switched off; every answer is measured and logged.
- **Interfaces (FR-18 to FR-21).** Chat, Code, the live view of the model at work, and the pairing that gives other runtimes memory as words.
- **The proof (FR-22 to FR-26).** The Long Haul is built from a seed, cut and sampled; every contender is told and asked the same; answers are graded by rule; the cloud yardstick is metered; every run writes a dated record.

Table 4.2: Functional requirements catalogue

ID	Requirement	Priority	Source	Verification
FR-01	The baseline shall turn each document into notes under a configurable write policy (verbatim or structured).	Must	ii; v1; P §3.5.5	T
FR-02	The baseline shall reject empty, duplicate and instruction-like notes and record each rejection with its source.	Must	ii; v1; P §3.3	T
FR-03	The baseline shall retrieve the k notes most similar to a question and rank them by similarity alone or by Equation 4.1.	Must	ii; v1; P §3.5.1	T
FR-04	The baseline shall admit candidates by the gate of Equation 4.2 and add notes about entities named in admitted notes.	Must	ii; v1; F1, F1b	T, M
FR-05	The baseline shall pack admitted notes into a token budget and never exceed it.	Must	ii, v; v1; P §3.5.1	T
FR-06	The baseline shall run floor, ceiling and treatment conditions on one fixed sample and compute the gap closed (Equation 3.1).	Must	iv; v1; P §3.5.2	T
FR-07	The engine shall remember a moment by having the frozen reader read it once, with its room's earlier moments in view, and saving the keys and values of every layer.	Must	ii; E; E-15	T, M
FR-08	The engine shall place each new moment in a room by Equation 4.3.	Must	ii; E; E-18	T
FR-09	The engine shall append a moment's notes to its room's file and keep its words, day, room and embedding in an index.	Must	ii, iii; E; `rooms.py`	T
FR-10	The engine shall write each moment as a Markdown note in a vault, with links (doors) to notes it shares a room or an answer with, and a strength that fades each day it is unused (Equation 4.5).	Should	ii; E; E-29; `vault.py`	T

FR-11	The engine shall keep recently used rooms in RAM under a RAM budget, and put the weakest rooms to sleep, keeping their words, when the room files outgrow a disk budget.	Must	iii; E; E-29; `swap.py`	T, M
FR-12	For each question, the engine shall lay on the desk the room holding the moment closest to the question, after the header and the core, and then let the reader read the four closest moments as words, in date order.	Must	ii, v; E; E-16	M
FR-13	The engine shall read a room from RAM when it is hot, from the SSD in one burst when it is awake, and as words (or woken) when it is asleep.	Must	iii; E; `swap.py`	T
FR-14	The engine shall build a core of at most 256 tokens from vault notes tagged `core` and keep it in RAM as notes.	Could	ii; E; `core.py`	T
FR-15	A session shall read several reaches (session, project, agent, global) and write to one; a question shall choose the best room over every reach it reads.	Must	v; E; E-42; `reach.py`	T, M
FR-16	A session shall let memory be switched off, so that the plain model answers with the same prompt and no notes.	Must	iv; E; `reach.py`	T
FR-17	The engine shall record for every answer its time to the first word, total time, retrieval time per step, tier, bytes read, tokens in context, history held, peak RAM and disk (Table 4.8).	Must	iv; E; `metrics.py`	T
FR-18	The application shall offer Chat, in which a user tells and asks with memory on or off, and Code, a coding agent on the local model whose every shell command waits for the user's approval.	Must	iii; E; E-50	D
FR-19	The application shall show, as the engine measures them, the model's layers loading, the room an answer reads and where it came from, and each layer's activity for each word.	Should	ii; E; `live.py`	D, T
FR-20	A pairing proxy shall give Ollama, and any OpenAI-compatible local server, memory as words: the closest notes in front of the last user message.	Should	iii; E; E-33, E-47	T
FR-21	The engine shall log each remember, answer, sleep and wake as one line, with each written word's attention over the moments on the desk.	Should	ii; E; E-21	T
FR-22	The Long Haul shall be generated deterministically from its seed and cut at fixed sizes, each cut asking a stratified sample of 10 questions per family (all 3 for the page).	Must	iv; E; E-30	T
FR-23	Every contender shall be told the history one event at a time and asked the same questions with the same instructions; asking shall never teach.	Must	iv; E; E-31, E-32	T, I
FR-24	Every answer shall be graded by rule into one of five verdicts (Table 5.5).	Must	iv; E; `haul/grade.py`	T

FR-25	Every call to the cloud yardstick shall be logged with its tokens and cost, and refused if its worst case would pass the night's cap.	Must	iv; E; `bench/claude.py`, `bench/cost.py`	T
FR-26	Every run shall write a dated record after every checkpoint, naming the machine, its load, the code fingerprint and every answer.	Must	iv; E; E-12, E-40	T, I

4.2.2 **Non-Functional Requirements**

The non-functional requirements use the ISO/IEC 25010:2023 characteristics relevant here (performance efficiency, reliability, security, maintainability, flexibility, interaction capability) and add model quality and honesty for the AI components. Requirements measured as not met are kept as written so that Chapters 5 and 6 report them honestly.

Table 4.3: Non-functional requirements catalogue

ID	Category	Requirement	Priority	Source	Verification
NFR-01	Performance: time	With 2,000 words in the room on the desk, read from the SSD with the cache bypassed, the median time to the first word shall be under 0.5 s.	Must	iii, v; E; E-19 mark 1	M
NFR-02	Performance: time	At 2,000 words, answering from notes shall be at least 10 times sooner than reading the same desk as words.	Should	iii; E; E-19 mark 3	M
NFR-03	Performance: capacity	As the memory grows, the median time to the first word and the peak RAM shall not climb.	Must	iii, iv; E; E-20, E-38	M
NFR-04	Performance: resources	The room files shall stay within a disk budget (40 GB in the benchmarks) and the hot rooms within a RAM budget (2 GB in the benchmarks; 512 MiB by default).	Must	iii; E; `swap.py`, `bench/local.py`	M
NFR-05	Performance: resources	MLX's cache of freed graphics buffers shall be capped (1 GB by default), and every record shall log the process's real footprint.	Must	iii; E; E-40, E-41	M
NFR-06	Performance: time	Retrieval shall take at most 10% of answer time on the target device.	Should	iii, v; v1; F8	M
NFR-07	Model quality	Within one room, notes shall know at least 95% as many facts as the same desk read as words, with at most 3 extra false memories of 45.	Must	ii, iv; E; E-19 mark 2	M
NFR-08	Honesty	Answers about things never told shall be graded separately; a decline is right and a guess is a false memory.	Must	iv; E; E-08, E-30	T
NFR-09	Reliability	Local contenders shall decode greedily with thinking off, so that re-runs give the same verdicts.	Must	iv; E; E-10, E-41	I, M

NFR-10	Security: privacy	All local inference and memory shall stay on the Mac once the models are downloaded; only the yardstick's benchmark runs send scripted text to a cloud service.	Must	iii; E; P §1.6	I
NFR-11	Security: integrity	The engine's server and proxies shall listen on 127.0.0.1 only, answer only requests that name the local host, and refuse requests from web pages; the app's server shall require the token made at launch.	Must	iii; E; `app/server.py`, `pair/ollama.py`, `Engine.swift`	T
NFR-12	Security: integrity	A reach folder shall be opened by one process at a time, and rooms written for one model shall not open with another.	Must	ii; E; `reach.py`, `rooms.py`	T
NFR-13	Reliability: measurement	Only one measuring job shall use the graphics processor at a time, and the app shall pause while a benchmark measures.	Must	iv; E; E-35	I
NFR-14	Cost	The yardstick's spending shall stay under $60 a night, with $2 of headroom held for the worst case of the next call.	Must	iv; E; E-29; `bench/claude.py`	T, M
NFR-15	Maintainability: testability	Every reported number shall be regenerable from records that carry a fingerprint of the code that made them.	Must	iv; both; P §3.7.2; E-12	T, I
NFR-16	Flexibility: adaptability	The tests that need no MLX shall run on Linux on every push.	Should	iv; E; `tests.yml`	T

4.2.3 **Data Requirements**

Neither stage trains the language model, so the separation of development and test data applies to the engine's settings and to the questions. In the baseline, settings were tuned on validation questions and confirmed once on held-out ones (F13, F15). In the rooms build, the development lives (seed 0) were used to choose the room design and the proof lives (seed 1) were run only after the pass marks were written (E-19). In the final engine, no setting was tuned on the Long Haul: the room design came from the lives, and the instructions were fixed before any Long Haul result (E-32, E-37).

The Long Haul is synthetic by design. A memory that grows over months cannot be tested on a public question set, while a script knows its right answer on every day, including after a fact changes. It uses Kenyan names and places, which suits the intended users but limits the results to one invented project. The public suites are the real-data complement. No personal data enters any evaluation. Table 4.4 lists the data requirements.

Table 4.4: Data requirements catalogue

ID	Requirement	Priority	Source	Verification

DR-01	The baseline shall use the official PopQA, HotpotQA, 2WikiMultiHopQA and GSM8K files, and its memory shall come only from supporting and distractor documents.	Must	iv; v1; P §3.2	I, T
DR-02	The Long Haul for seed 1 shall hold 17,786 events and 2,010,094 tokens over 110 working days, extended to 4.01M tokens with noise only, every probe unchanged.	Must	iv; E; E-30, E-39	T
DR-03	Each checkpoint shall sample 10 askable probes per family with seed 0; a cut where nothing new is askable shall reuse the sample before it.	Must	iv; E; E-30, E-39	T
DR-04	Each moment shall carry its date and the Long Haul's day number, as `[YYYY-MM-DD, day N] text`.	Must	iv; E; `bench/local.py`	I
DR-05	Each moment shall be stored as words in the index, an embedding row, notes in its room's file and a vault note.	Must	ii; E; `rooms.py`, `vault.py`	T
DR-06	Room files shall only grow while awake; sleep shall delete a room's notes but never its words.	Must	ii; E; `swap.py`	T
DR-07	Every record shall name the machine, chip, memory, power, load, library versions, commit and a hash of the code.	Must	iv; E; E-12, E-17	I, T
DR-08	Personal memory shall stay in folders under `~/.camr/` on the user's Mac.	Must	iii; E; `reach.py`	I

4.2.4 **Interface and Integration Requirements**

The interfaces are kept narrow so that each can be tested: the app reaches the engine only through a small JSON API on the loopback address, the models are loaded in-process through MLX, and the cloud yardstick is reached only by the benchmark runners (Table 4.5).

Table 4.5: Interface and integration requirements catalogue

ID	Requirement	Priority	Source	Verification
IR-01	The baseline shall reach local models only through Ollama on the loopback address, and the cloud model under one pinned, logged identifier.	Must	iii, iv; v1; P §3.6	T
IR-02	The engine shall load its models through MLX and refuse a reader whose layers do not all keep keys and values with plain rotary positions.	Must	iii; E; E-26, E-44; `notes.py`	I
IR-03	The engine shall read and write room files with the file cache bypassed (`F_NOCACHE`), in 1 MB pieces eight at a time.	Must	iii; E; E-14	T, I
IR-04	The Mac app shall start the engine as a local process on a free port with a token only it knows, and use only `POST /api/chat/message`, `GET /api/live` and the engine's other JSON routes.	Must	iii; E; `Engine.swift`	D
IR-05	The engine's server shall return failures as JSON (403, 404, or 500 with the error) instead of dropping the connection.	Should	iii; E; `server.py`	I

IR-06	The Ollama pairing shall speak Ollama's own HTTP API on 127.0.0.1:11435 and forward everything else byte for byte to 11434.	Should	iii; E; E-33	T
IR-07	The OpenAI-compatible endpoint shall serve `/v1/` routes on 127.0.0.1:11436 in front of Ollama, LM Studio or mlx_lm.server.	Could	iii; E; E-47	T
IR-08	The yardstick runner shall stream every request, turn the SDK's retries off and never ask for a fallback model.	Must	iv; E; `bench/claude.py`	I
IR-09	The suites shall be loaded from the Hugging Face Hub once and read from a local folder afterwards.	Must	iv; E; `bench/suites`	I

4.2.5 **Requirements Traceability**

Table 4.6 traces each objective to its requirements, design elements and evidence. Objective i is met mainly by Chapter 2, but it also shaped the system: the reviewed techniques were built as switchable choices so that their value could be measured.

Table 4.6: Traceability matrix

Objective	**Requirements**	**Design elements**	**Verification evidence**
i. Techniques and limits	FR-03, FR-04, FR-07, FR-12	Baseline read path (Figure 4.4); room design (Figure 4.5)	F1, F1b; E-15, E-16
ii. Design	FR-07 to FR-14, FR-21, DR-05, DR-06, NFR-07	Memory, Rooms, Notebook, Tiers, Vault, Core, Event log (Figures 4.5 to 4.9)	Tests of rooms, notes, tiers, vault and events; E-16, E-18, E-20, E-21
iii. On the device	FR-11, FR-13, FR-18 to FR-20, NFR-01 to NFR-05, NFR-10 to NFR-13, IR-02 to IR-04	Tiers; deployment (Figure 4.10); the app	Speed and RAM records; E-38, E-40, E-41
iv. Evaluation	FR-06, FR-16, FR-22 to FR-26, DR-01 to DR-04, DR-07, NFR-08, NFR-09, NFR-14, NFR-15	Evaluation pipeline (Figure 4.3)	Long Haul and suite records of 4 November 2026; E-30 to E-43
v. Budget and components	FR-05, FR-08, FR-12, FR-15	Rooms of 512 tokens, join 0.80, four closest moments; reach	E-18 (room size); E-37 (wording); E-42 (reach); F8 (baseline budget)

4.3 **System Analysis Diagrams**

4.3.1 **Context Diagram**

The context diagram treats CAMR as one process and shows only what crosses its boundary. It fixes the scope of Section 1.6, in which the model and the memory run on the user's Mac, and shows which flows leave it (NFR-10, DR-08). It uses Gane–Sarson notation: CAMR is process 0, each shaded box is an external entity and each labelled arrow is a data flow; flows that exist only in the baseline are dashed.

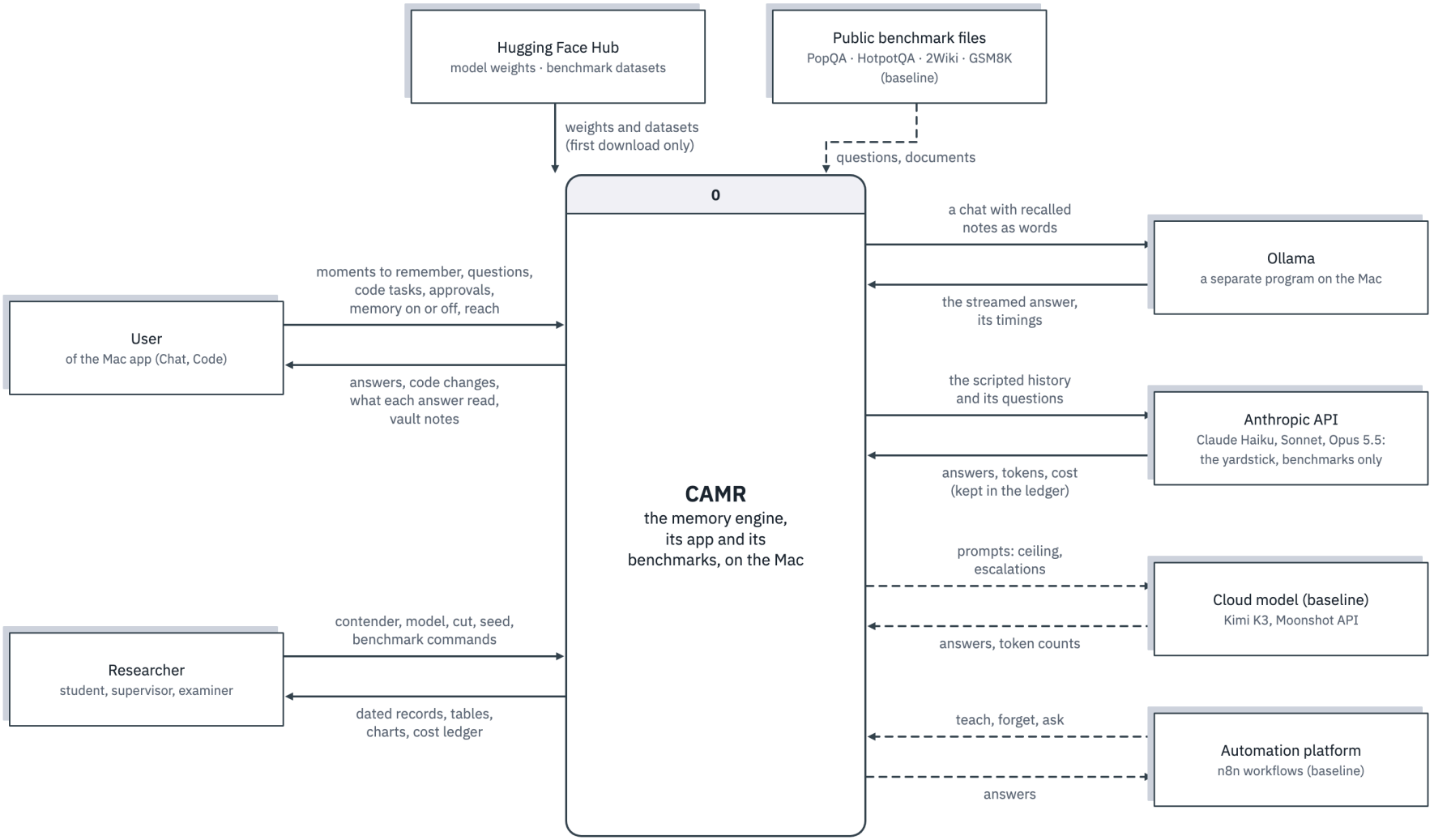


Figure 4.1: Context diagram of CAMR

The user's moments and questions flow into CAMR and their answers flow back, and nothing continues to an outside party. Two flows leave the Mac, and neither carries personal data. The Hugging Face Hub supplies model weights and the public suites once; afterwards they are read from the local cache. The Anthropic API receives the scripted Long Haul and the suites' questions, and only when the researcher runs the yardstick. Ollama is outside the boundary because it is a separate program that the pairing reaches over the loopback address; the engine's own models run inside its process. The baseline's flows to Kimi K3 and n8n are dashed. The diagram therefore shows CAMR in use as a closed system on the Mac, the privacy property the problem statement asks for.

4.3.2 **Use Case Diagram**

The use case diagram shows who uses CAMR and for what (Figure 4.2). Its actors are people and external systems only: the User, the Researcher, the Ollama command line and the Claude API. Rooms, the embedder and the event log are internal parts, not actors. «include» and «extend» relations are dashed arrows, and generalisation is a hollow triangle.

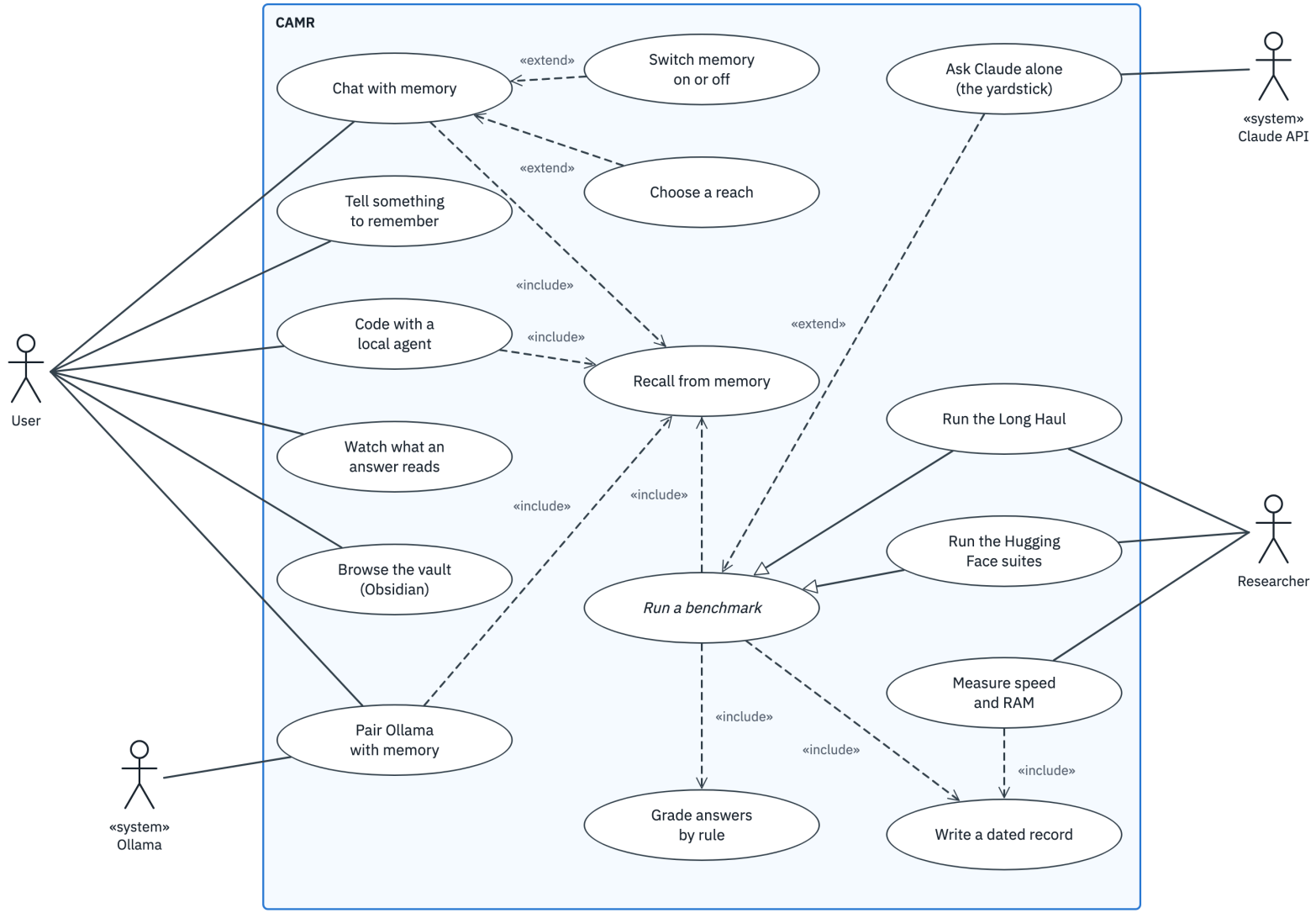


Figure 4.2: Use case diagram of CAMR

"Recall from memory" is included by every use case that answers anything: chatting, coding with the agent, the Ollama pairing and the abstract "Run a benchmark", which the Long Haul and the suites specialise. One read path therefore serves chat, agentic work and evaluation, so what is measured is what the user gets. Switching memory off and choosing a reach extend chatting. Asking Claude alone extends a benchmark: the yardstick answers the same questions with no memory. Watching what an answer reads, and browsing the vault, are a user's use cases, not only research tools: they show a user when an answer rested on the wrong moment.

4.3.3 **Evaluation and Data Pipeline**

The study's contribution is a measurement, so the pipeline that produces each number is analysed as carefully as the engine. Figure 4.3 shows the Long Haul's pipeline as a UML activity diagram.

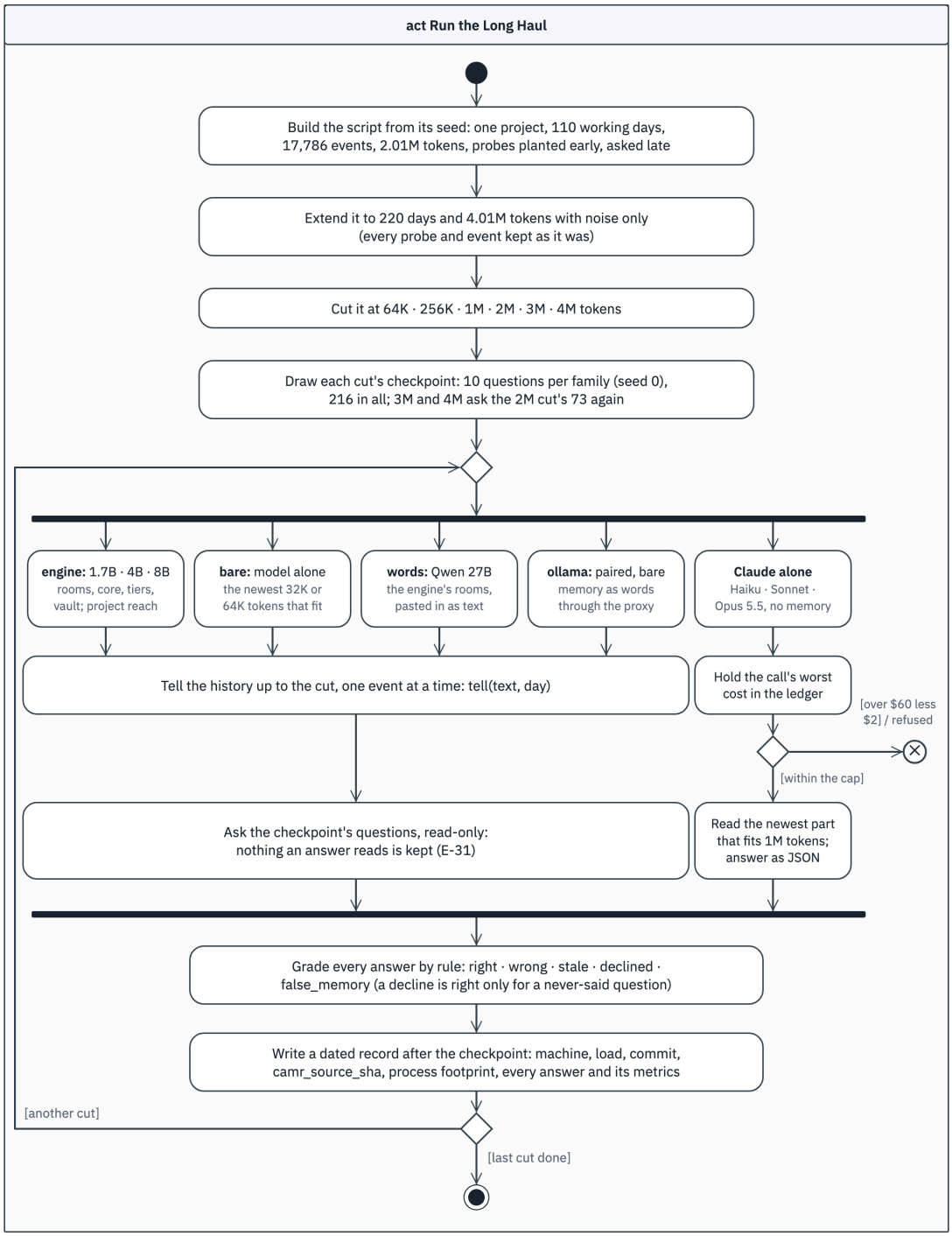


Figure 4.3: The evaluation pipeline of the Long Haul

The script is built once from its seed and cut at fixed sizes; at each cut, a checkpoint samples the questions that can be asked there. The pipeline then forks into one branch per contender. Each local contender is told the history one event at a time and then asked the checkpoint's questions, read-only. The yardstick instead reads the newest part of the history that fits its window, with every request held against the cost ledger before it is sent. The branches join at grading, which applies the same rules to every answer, and every checkpoint writes a dated record before the next cut is told. What is held

constant is what makes the scores comparable: the same events, the same questions, the same instructions and the same grader for every contender (E-32). What differs is only what each contender can see.

4.4 **System Design Diagrams**

4.4.1 **Architecture of the Baseline, CAMR v1**

CAMR v1 has a write path and a read path that meet at one SQLite file (Figure 4.4). Ranking follows Park et al. (2023), with similarity and importance normalised over the candidates, and recency counted in question steps. The gate then decides how much of the ranked list the model sees.

$$\text{score}(n) = w_s(n) + w_r \gamma^{\Delta t(n)} + w_i(n) \tag{4.1}$$

Equation 4.1 is the composite score of note n (defaults w_s = 0.60, w_r = 0.15, w_i = 0.25, $\gamma = 0.99$; similarity only sets w_s = 1, w_r = w_i = 0).

$$n \text{ is admitted} \iff s_{max} \geq \tau \text{ and } s(n) \geq s_{max} - \delta \tag{4.2}$$

Equation 4.2 is the gate, where s_max is the best candidate's similarity (defaults $\tau = 0.50$, $\delta = 0.15$).

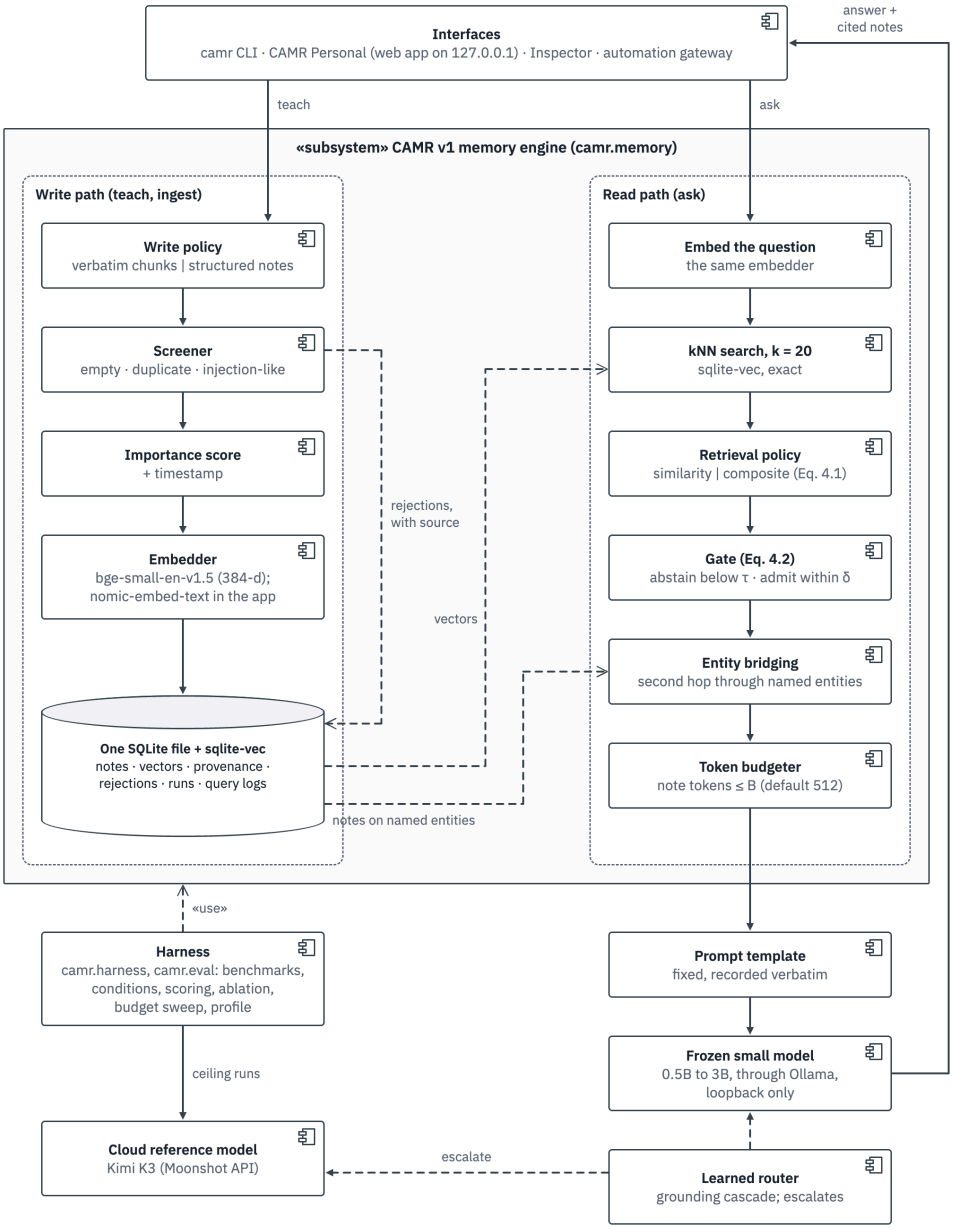


Figure 4.4: Architecture of the baseline, CAMR v1

The gate and entity bridging made the read path capability-adaptive: on 2WikiMultiHopQA, bridging raised the share of questions with both gold paragraphs in context from 35.8% to 78.4% with 7% fewer tokens (F1b). The composite score lowered that share (F1), so similarity became the default ranking (Section 4.5). The model was reached only through a loopback runner and the cloud only in ceiling conditions or on escalation.

4.4.2 **Architecture of CAMR**

CAMR changes what a memory is. Instead of text the model must re-read for every question, each moment is kept as the notes the model wrote while reading it: the keys and values of every layer, which a transformer normally discards after a prompt. Laid in front of the model, they let it continue

as if it had just read the moment. Notes are large (Equation 4.4), so they live on the SSD in rooms of related moments that are written as one continuous page and fetched in one read. A moment joins a room by Equation 4.3.

$$\text{room}(m)=\text{room}\left(\underset{k:\, s_{mk}\geq J,\, T_{\text{room}(k)}+t_m\leq T_{\max}}{\operatorname{argmax}} s_{mk}\right),\ \text{else a new room} \tag{4.3}$$

In Equation 4.3, m is the new moment with t_m tokens, s_mk is its similarity to an earlier moment k, and T is a room's tokens (J = 0.80, T_max = 512; E-18).

$$b_{\text{token}}=L\times 2\times H_{kv}\times d_h\times 2 \text{ bytes} \tag{4.4}$$

Equation 4.4 gives the notes per token, for L layers, H_kv key–value heads, head size d_h and 16-bit values: 28 × 2 × 8 × 128 × 2 = 114,688 bytes for Qwen3-1.7B and 36 × 2 × 8 × 128 × 2 = 147,456 bytes for Qwen3 4B and 8B (from the records' `kv_shape`). A full room of 512 tokens is therefore 56 MiB for the 1.7B, about 29,000 times its text.

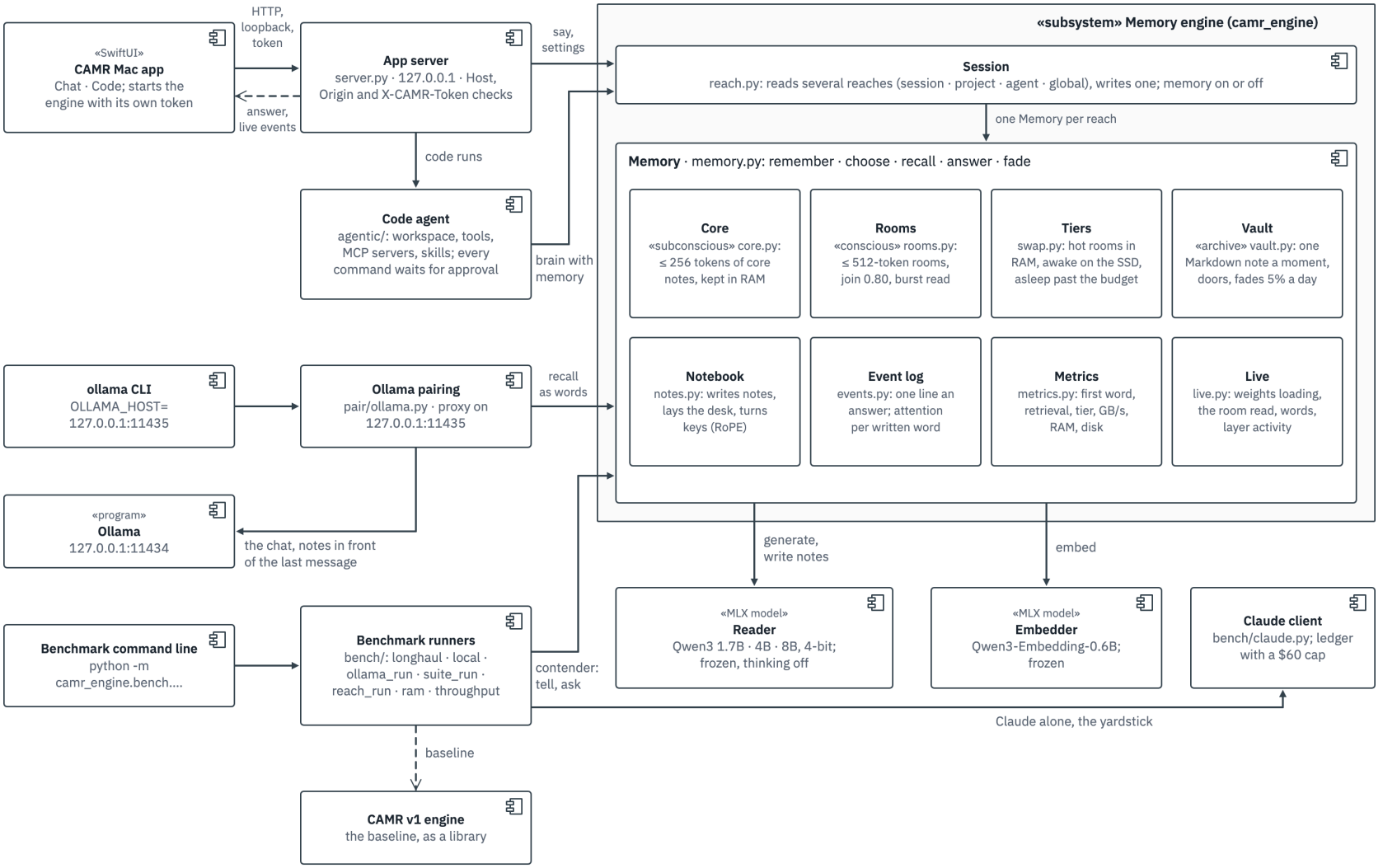


Figure 4.5: Component diagram of CAMR, the final engine

Each part has one job (Figure 4.5). Rooms owns the room files and the index and knows nothing about models. The Notebook turns tokens into notes and notes into a desk, and is the only part that touches the reader's cache. The Tiers decide where each room's notes are: hot in RAM, awake on the SSD, or asleep as words. The Vault keeps every moment as a Markdown note that a person can open in Obsidian, with links between notes that are used together and a strength that fades (Equation 4.5). The Core holds a few standing facts as notes in RAM. The Event log watches attention without changing it, Metrics times every step, and Live reports what a person can watch. Memory composes them into remember, choose, recall, answer and fade, and a Session composes one Memory per reach. The app, the Code agent, the pairing and every benchmark runner call the same Session or Memory, so the measured read path is the one the user meets.

$$s_n(d)=s_n(d_0)\times 0.95^{d-d_0}+u_n \tag{4.5}$$

In Equation 4.5, a vault note's strength s_n on day d is its strength on the day d_0 it was last brought up to date, reduced by 5% for each day unused, plus 1.0 for each of its u_n uses since then; a note at strength 1 that goes unused for about 45 days falls below 0.1 and counts as faded. Pinned notes never fade, and nothing is ever deleted by the engine. A room is as strong as its strongest note, and when the room files outgrow the disk budget the weakest room sleeps first.

Two measurements on the MacBook Air shaped this design. Moments written alone and laid side by side lost the thread: with 30 unrelated moments beside the right one, the reader found 2 of 46 facts from such notes against 32 from the words (E-15). So a room is written as one page, each moment read with the room's earlier moments in view. And one room plus the four closest moments as words knew 158 facts, against 120 for the room alone and 135 for the moments alone (E-16). A single room sits where it was written, so its keys need no turning; with the core before it, a room sits further along the desk and its keys are turned by the core's length (Su et al., 2024).

4.4.3 **Sequence Diagram: One Question**

Figure 4.6 follows one question asked in Chat. Lifelines run left to right and time downwards; solid arrows with filled heads are calls, dashed arrows with open heads are returns, a looping arrow is work done inside one part, and a bar shows while a part is busy. It realises FR-12, FR-13, FR-17 and FR-19 and shows where the wait of NFR-01 and NFR-03 is spent.

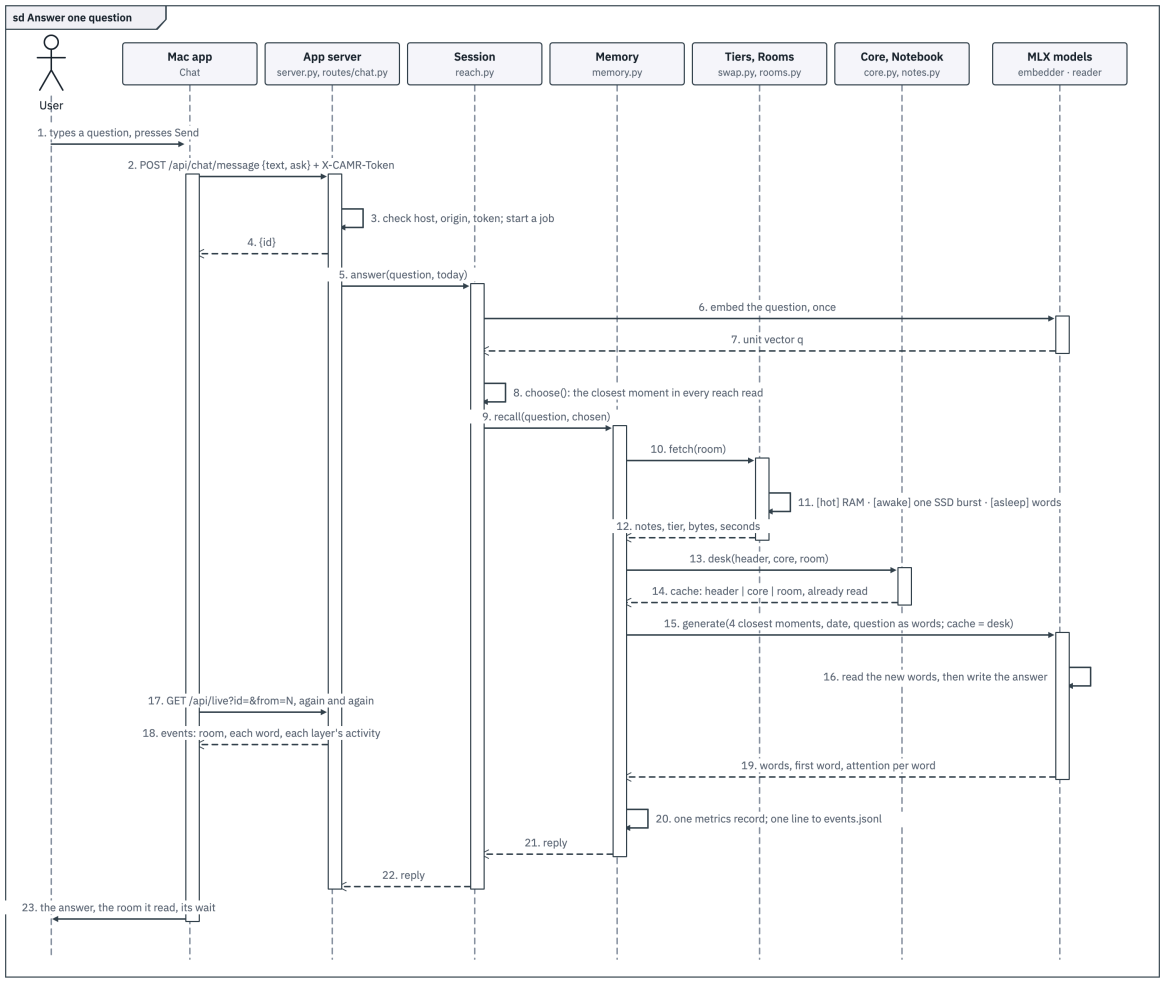


Figure 4.6: Sequence diagram of one question answered in Chat

The order of steps keeps the wait short. The question is embedded once, and the closest moment is chosen over every reach the session reads (steps 6 to 8). The room is then fetched before the reader writes anything: from RAM when hot, in one SSD burst when awake, or as words when asleep (steps 10 to 12). The header and the core are already notes in RAM, and the room arrives already read, so at question time the reader reads only the four closest moments, the date and the question as words (step 15). While the answer is written, the app polls the live events and draws each word and each layer's activity (steps 17 and 18). Every answer ends with one metrics record and one line in the event log (step 20). On the Long Haul at 2M tokens this path took 18 ms to choose and fetch (retrieval p50) and gave the 1.7B's first word in 0.08 s (E-38).

4.4.4 **State Diagram of a Room and Its Moments**

A moment and its room have coupled life cycles: placing a moment changes its room, a question moves a room onto the desk, and the disk budget can put a room to sleep. Figure 4.7 shows the three state machines; its guards are Equations 4.3, 4.6 and 4.5.

$$\text{share}_j = \frac{\sum_w a_{wj}}{\sum_w \sum_k a_{wk}}, j \text{ is used} \Longleftrightarrow \text{share}_j \geq 0.10 \tag{4.6}$$

Equation 4.6 is the share of the desk's attention drawn by moment j over an answer, where a_wj is the attention from written word w to j's tokens, averaged over every head of layers 12, 16, 20 and 24 (E-21).

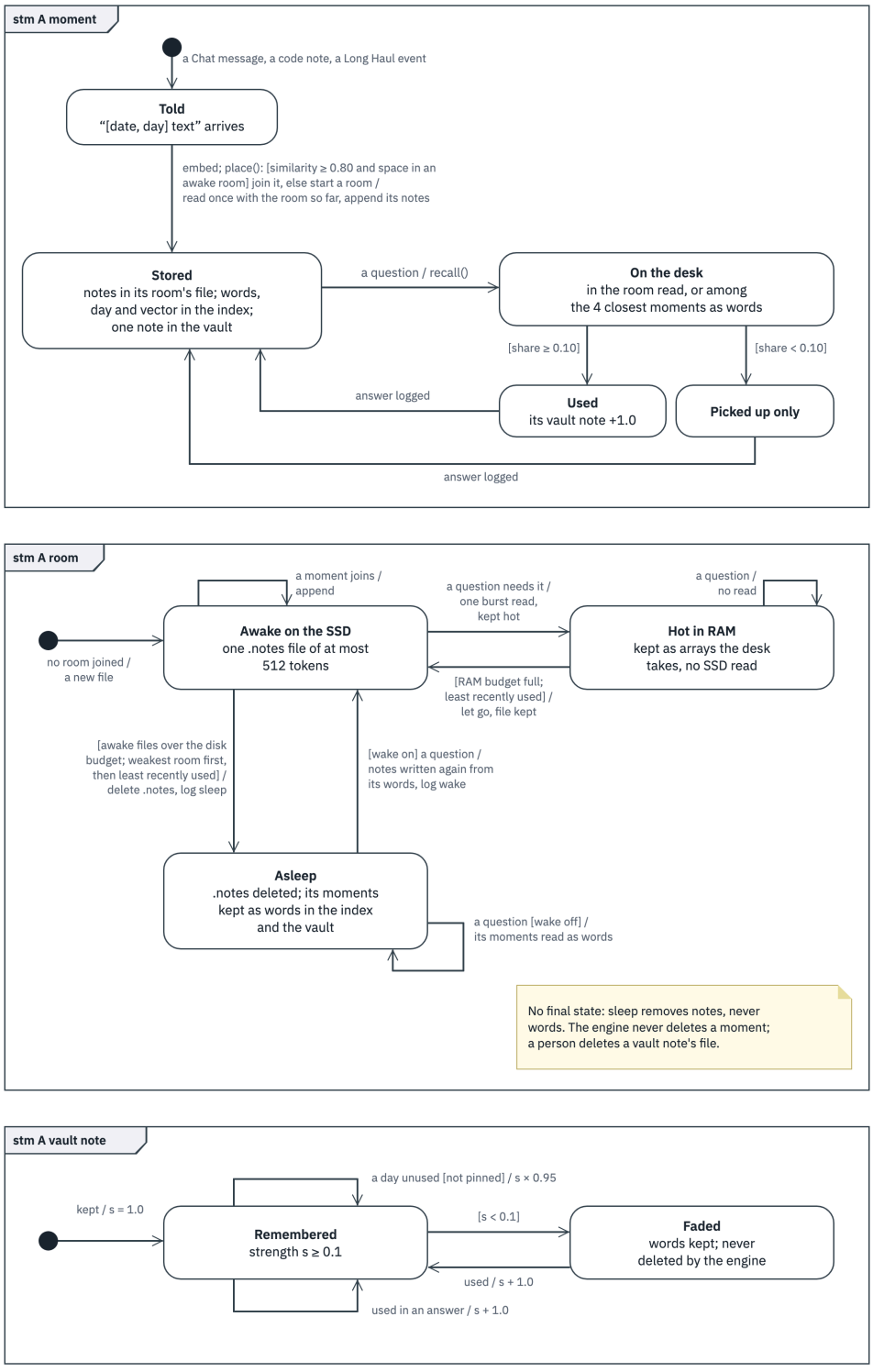


Figure 4.7: State machines of a moment, a room and a vault note's strength

A moment has no final state: once stored it only cycles between Stored and On the desk, where it is either used or only picked up. A room is awake on the SSD, hot in RAM after a read, and back to awake when it is the least recently used room and the RAM budget is full; letting a hot room go writes nothing, since its file is still on the SSD. When the awake room files outgrow the disk budget, the weakest room goes to sleep: its notes file is deleted and its moments stay as words in the index and the vault. A question about an asleep room reads its moments as words, or wakes it when waking is on, which costs one reading of the room by the model. Sleep therefore keeps the SSD within budget without destroying anything: at the end of the 2M-token Long Haul, 4,471 of 5,391 rooms were asleep (E-38).

4.4.5 **Logical Schema of the Baseline Store**

The baseline uses an embedded relational database, so a logical schema is the right view. Figure 4.8 shows its tables with their keys and cardinalities.

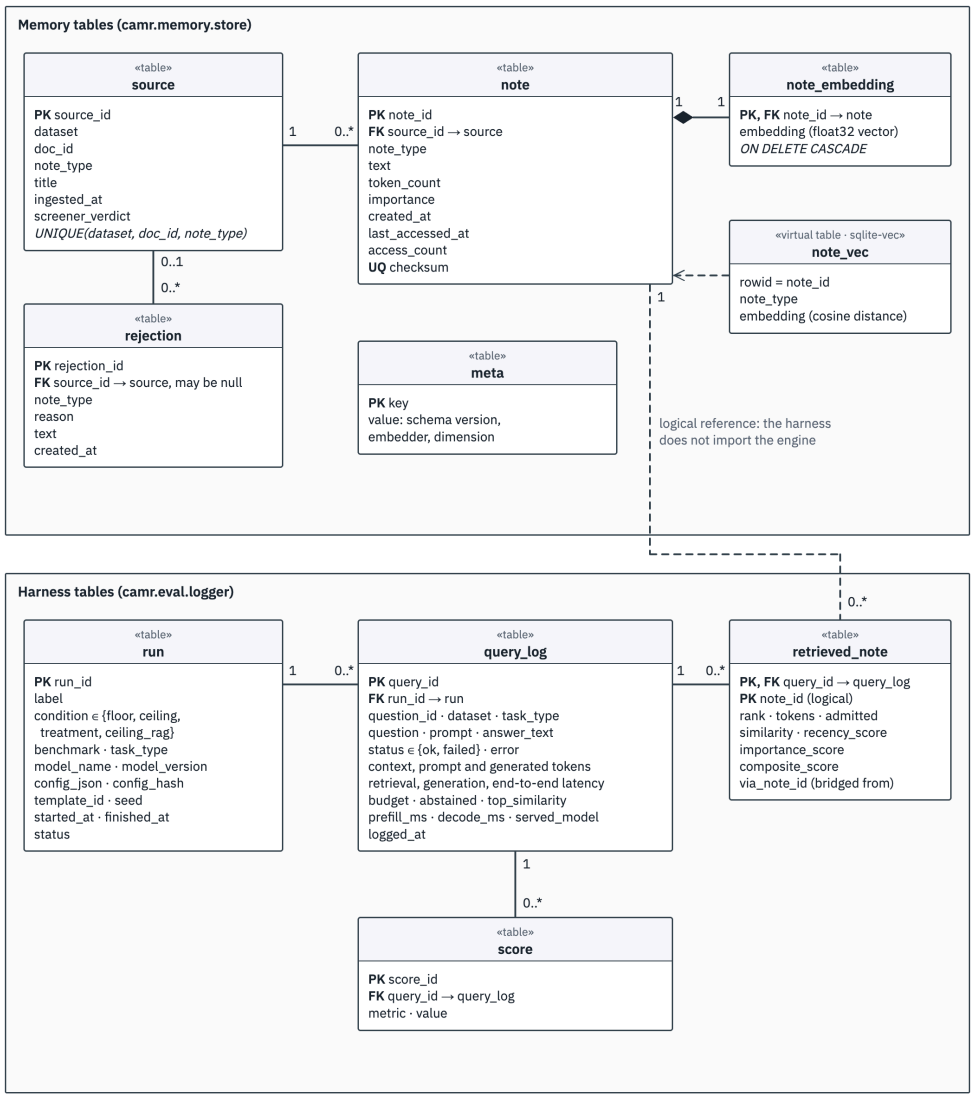


Figure 4.8: Logical schema of the baseline's store

The integrity rules carry the baseline's guarantees. Checksums are unique, and a source is unique per dataset, document and note type, so each write policy is its own note population in one file. A note and its embedding are committed together with a cascading delete, so no note lacks its vector. `meta` records the schema version, embedder and dimension, and a store refuses an embedder it was not built with.

4.4.6 **File and Data Design of CAMR**

CAMR has no transactional database. Each memory is one folder in which every file has one writer and one format, and each reach is a folder of its own under `~/.camr/` (Figure 4.9, Table 4.7).

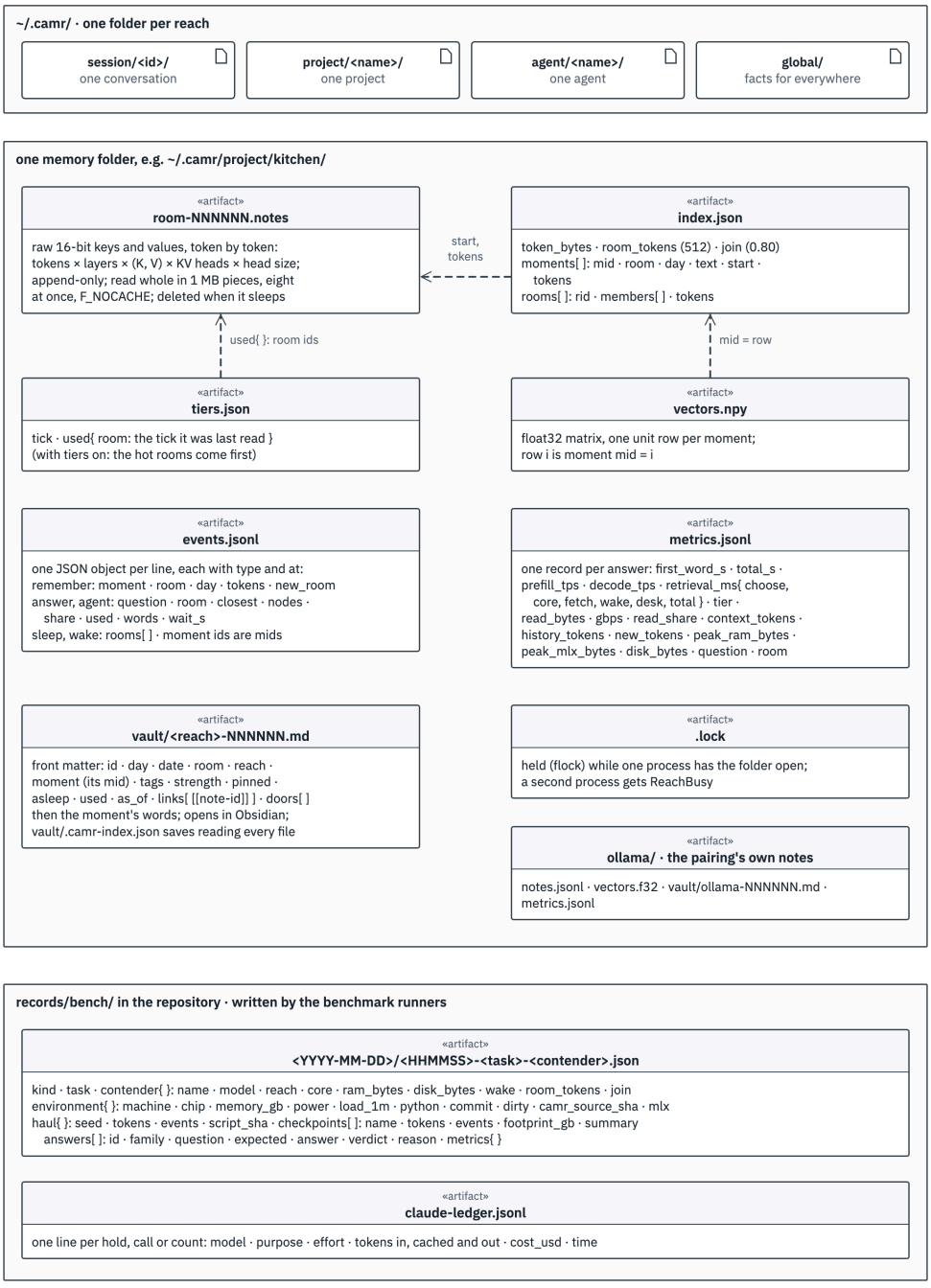


Figure 4.9: Files of a CAMR memory, its reaches and its records

Table 4.7: Files of a CAMR memory

File	Format	Written	Read
`room-NNNNNN.notes`	Raw 16-bit values, token by token (tokens × layers × keys and values × heads × head size)	Appended at each remember; deleted when the room sleeps	Whole, at recall, cache bypassed
`index.json`	Every moment's words, day, room and byte range; every room's members and tokens	After each moment in a shared reach; at save	When the memory opens; for the words a desk re-reads
`vectors.npy`	float32 matrix, one unit row per moment	With the index	On opening; one product per question
`vault/*.md`	One Markdown note per moment: front matter (id, day, room, reach, strength, asleep, links, doors), then its words	At each remember and each fade	By a person (Obsidian); by the core and by sleep
`events.jsonl`	One JSON line per moment kept, answer given, room put to sleep or woken	At every event	The app's views
`metrics.jsonl`	One JSON line per answer (Table 4.8)	At every answer	Summaries; the app
`tiers.json`	When each room was last used	With the index	When the memory opens
`.lock`	A lock held while a process has the folder open	On opening a reach	By a second process, which is refused

Files suit a workload with two operations: append a moment's notes to one file, and read one whole file. Each room is a contiguous array read in one burst, with no lookup and no database page cache in between, and bypassing the file cache makes every recall a real SSD read: 6 to 12 GB/s on the MacBook Pro when a room is not hot (Section 5.8.8). The readable part of memory sits in a small index and the vault, which can be inspected without touching the notes. Files refer to each other by integer position: a moment's id is its row in `vectors.npy`, and its byte range in the room comes from the index. The notes' size per token, recorded in the index, stops another model from opening the memory.

Table 4.8: What every answer records

What	Field	How it is counted
Time to the first word	`first_word_s`	From the question to the first word: choosing, reading the room, laying the desk and reading the words all count
Total time	`total_s`	From the question to the last word
Prefill and decode speed	`prefill_tps`, `decode_tps`	Tokens read as words per second before the first word; tokens written per second
Retrieval time	`retrieval_ms`	Per step: choose, core, fetch (the SSD read), wake, desk, and total
Where the room came from	`tier`	RAM, SSD or vault (asleep)
Bandwidth	`read_bytes`, `gbps`	Bytes read from the SSD for this answer, and how fast

Context and history	`context_tokens`, `history_tokens`	Every token the model saw; every token the memory holds over every reach read
Peak RAM and footprint	`peak_ram_bytes`, `peak_mlx_bytes`, `footprint_gb`	The process's peak resident memory and MLX's own peak; in the benchmarks, the process's physical footprint per checkpoint
Disk	`disk_bytes`	The memory folders on disk: room files, indexes, logs and vaults

4.4.7 **Deployment on the Mac**

Figure 4.10 shows where each part of CAMR runs: one Mac, its execution environments, the files on its SSD, and three outside parties used once, for benchmarks only, or for testing.

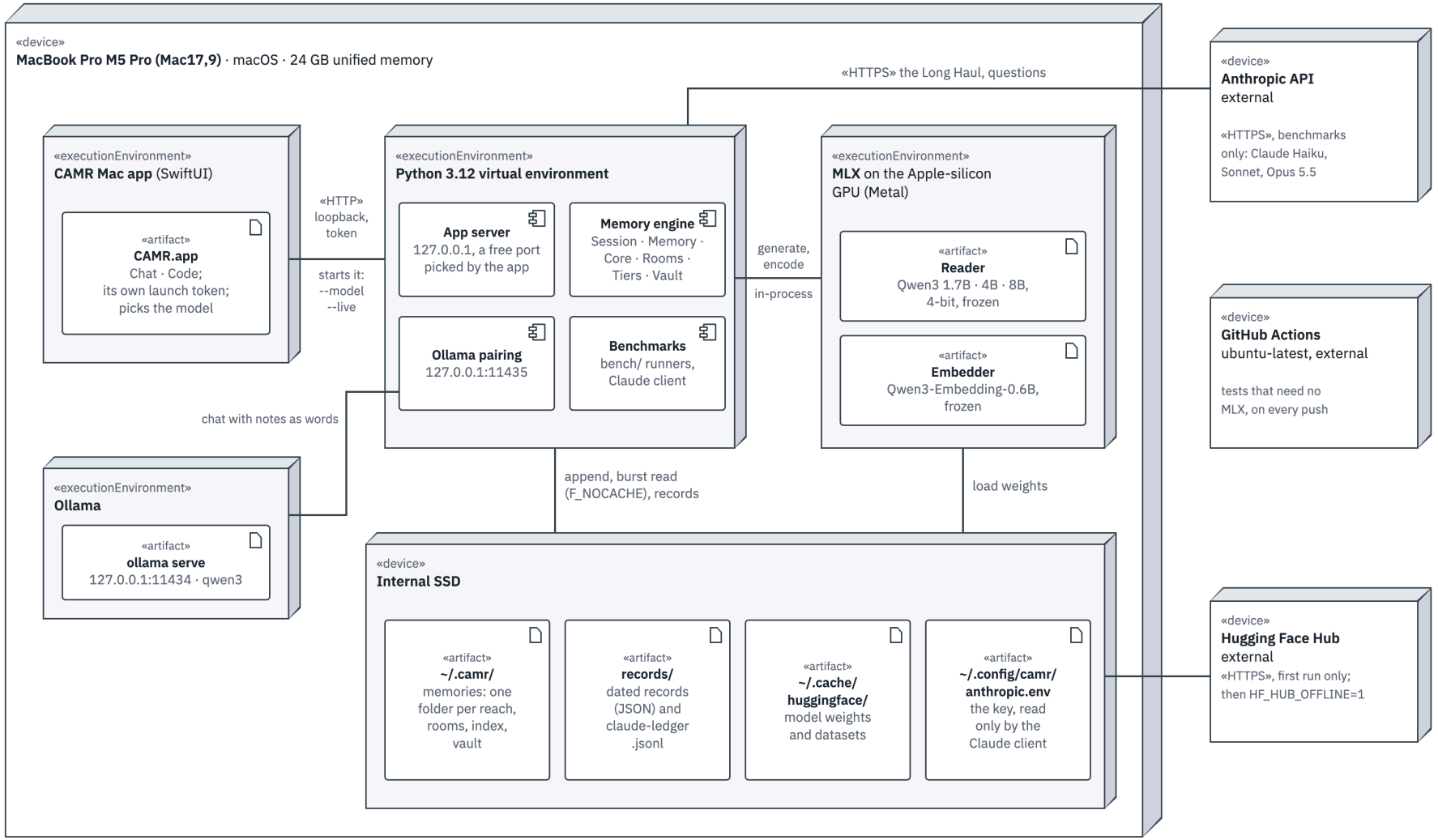


Figure 4.10: Deployment diagram of CAMR on the Mac

The Mac app is a SwiftUI process that starts the engine as a second, Python process with the chosen model and a token made at launch, and talks to it only over the loopback address (IR-04). The engine holds the reader and the embedder in unified memory through MLX on the graphics processor; each model has its own memory folder, because notes belong to the model that wrote them. Ollama, when paired, is a third process that the proxy reaches on its own port. The Hugging Face Hub is contacted only when a model or suite is missing. The Anthropic API is contacted only by the yardstick's runner, with its key read from a private file and never logged. GitHub Actions runs the tests that need no MLX on every push (NFR-16).

4.4.8 **User Interface Design**

The application is a native Mac app written in SwiftUI, with two parts, Chat and Code (E-50). An earlier Swarm screen, which ran several agents with their own memories, was removed so that the app does two things well. The visual direction ("Instrument") draws memory as an instrument would: rooms as tiles, the model's layers as rows, and colour used only to mean something, with memory in cyan, the model in amber and the cloud yardstick always grey (E-34, E-49). A panel beside the conversation, the weights view, draws only what the engine measures (FR-19). Figures 4.11 to 4.15 show the app with Qwen3 4B.

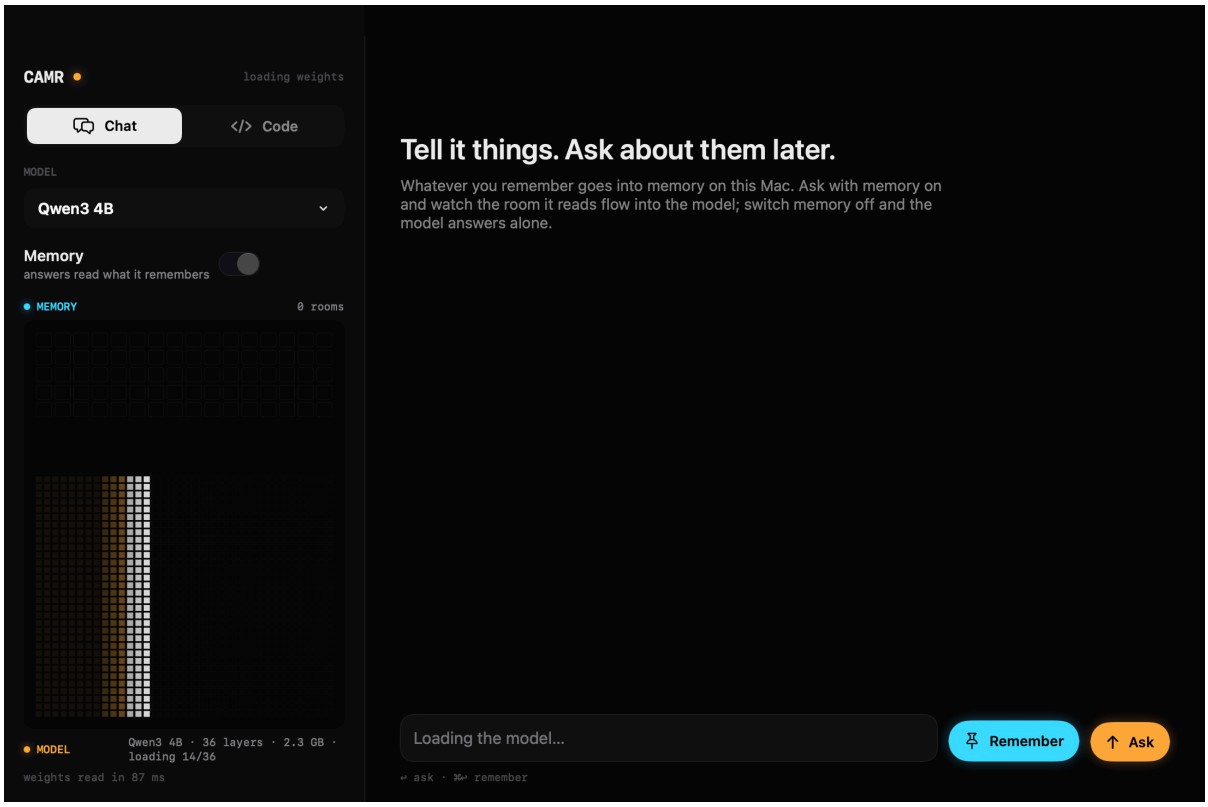


Figure 4.11: Starting the app: the model's 36 layers fill as their weights arrive ("loading 14/36"), before the first question

When the app starts, it starts the engine with the chosen model and draws the model's layers filling as their weights load (Figure 4.11). The memory panel above them shows how many rooms the memory

holds.

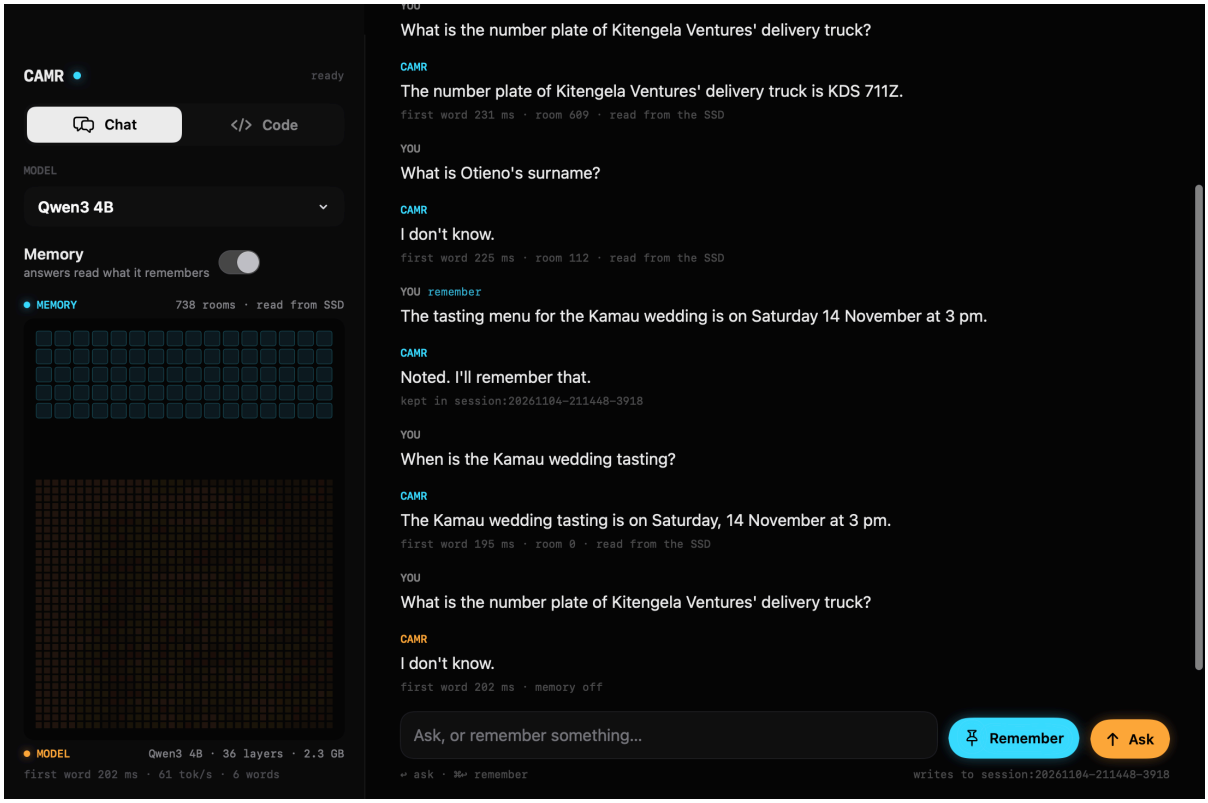


Figure 4.12: Chat with memory on and off: a fact told earlier is answered from the room it was kept in ("room 609 · read from the SSD"); with memory off, the same question gets "I don't know"

In Chat (Figure 4.12), the user tells the model something with Remember and asks with Ask. Under each answer the app shows the time to the first word, the room read and whether it came from RAM or the SSD; under each remembered fact, the reach it was kept in. A switch in the window turns memory off, so the same question can be asked of the model alone.

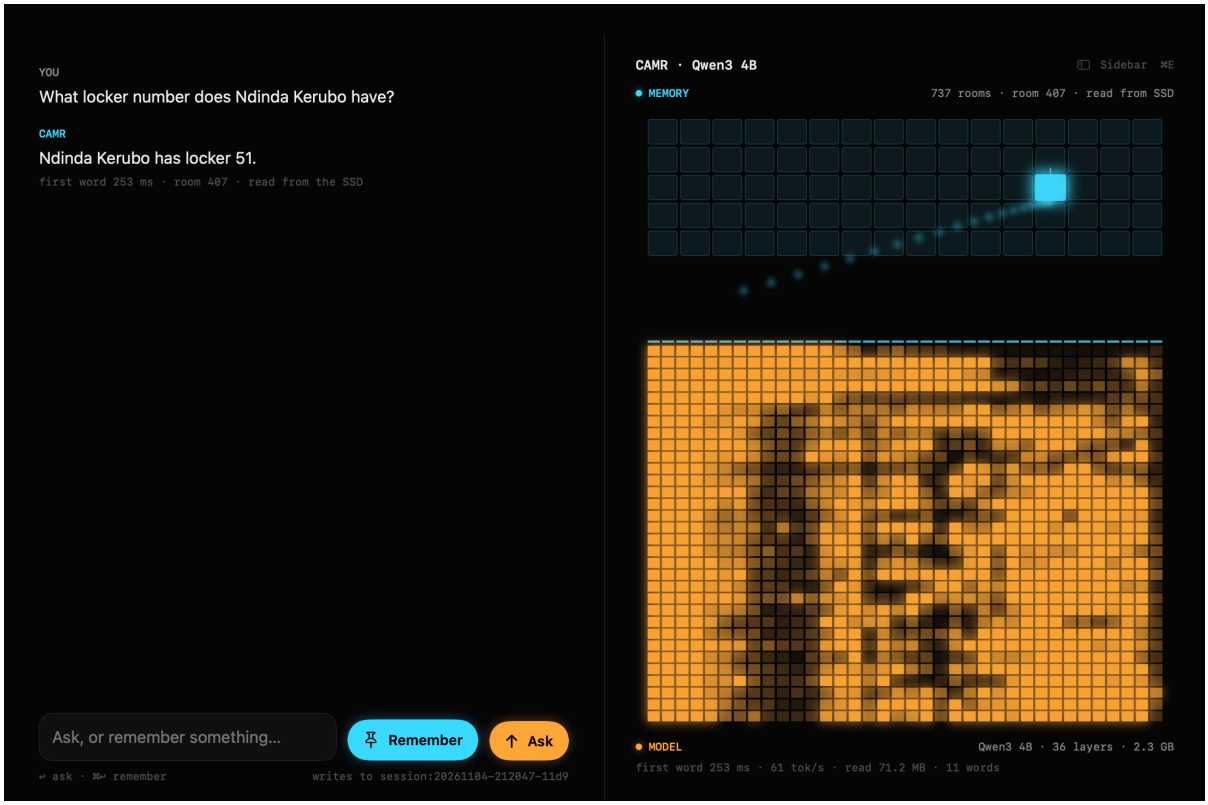


Figure 4.13: The weights view, large beside the chat for a projector: the room an answer reads travels from the memory (rooms, top) into the model's 36 layers (bottom)

The weights view can be shown large beside the chat (Figure 4.13). While the model answers, the room it reads is marked and travels into the model, and each layer's row lights with how strongly that layer fired for each word: 32 measured numbers per layer per word, sent by the engine only while the app listens, so the benchmarks pay nothing for it. With no events, nothing moves.

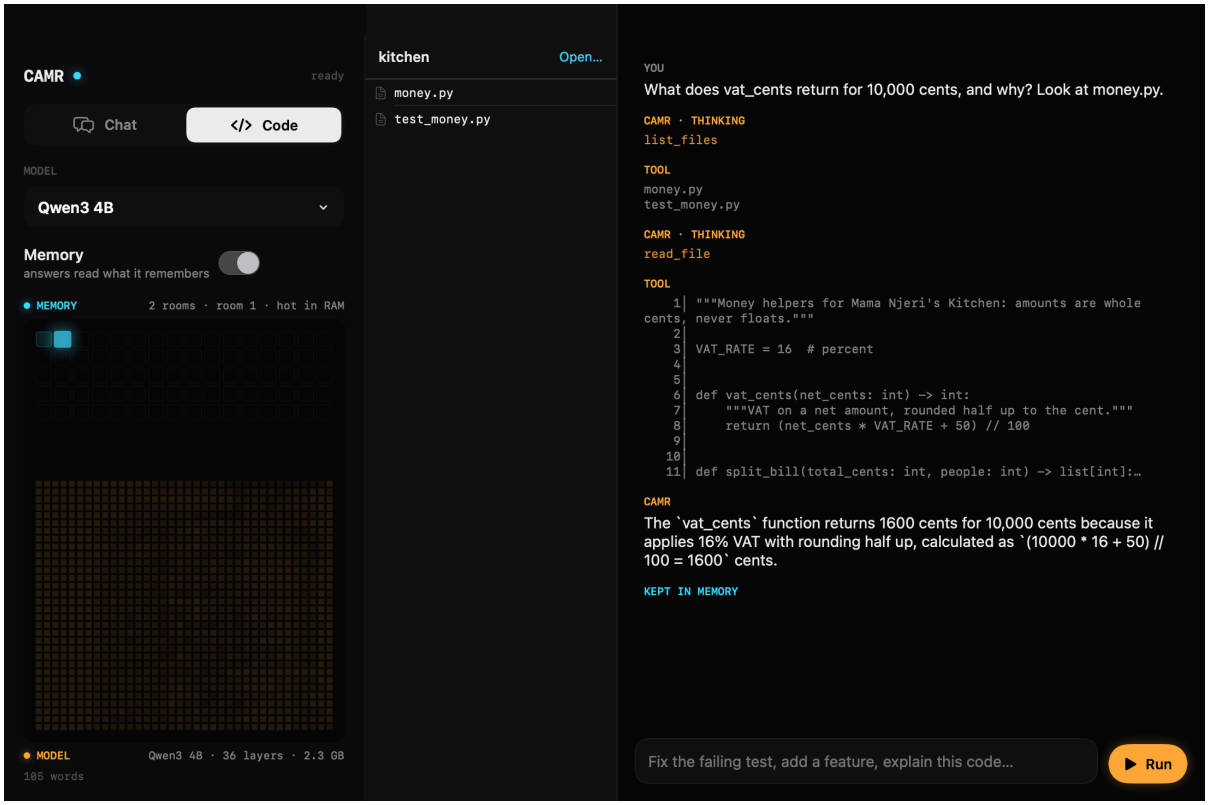


Figure 4.14: Code: the agent lists the files of a project folder, reads `money.py` and answers from what it read; the answer is kept in memory

Code is a coding agent on the same local model (Figure 4.14). The user picks a folder; the agent can list, search, read and edit files and run commands there, and every shell command waits for an explicit Allow. Its turns, tool calls and the files it opened are shown as it works, and with memory on what it learns about the project is kept for the next question.

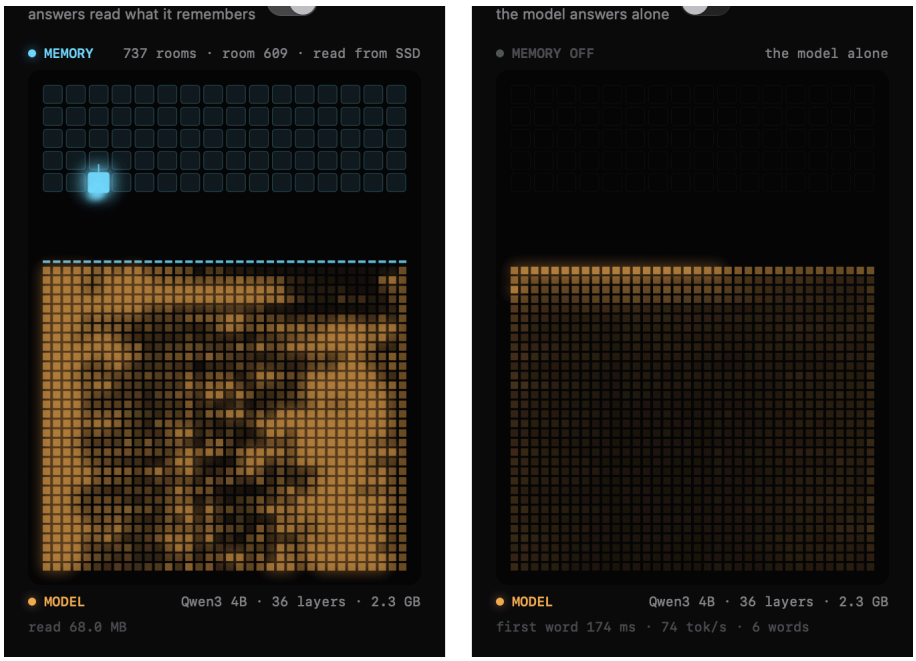


Figure 4.15: The weights view with memory on (left), where the room read lights and flows into the model, and off (right), where the model answers alone and the memory stays dark

Figure 4.15 sets the two states side by side. The difference is the design's main idea made visible: with memory on, an answer reads one room from the SSD or RAM; with it off, nothing outside the model is read.

4.5 **Design Changes Since the Proposal**

Building and measuring the three stages changed several choices that the proposal had fixed in advance. Table 4.9 states each change and its evidence, so that the diagrams, the code and the report describe the same system.

Table 4.9: Changes from the proposed design

Proposed	**Built**	**Reason and evidence**
An embedded database: SQLite with a vector extension in one file (P §3.6.5; objective iii)	The baseline as proposed; the engine stores one append-only file per room on the SSD, an index, a vector file and a vault	The engine stores the model's notes, 114,688 to 147,456 bytes per token (Equation 4.4), and only appends and reads whole files; bypassing the cache gives honest SSD timings (E-14)
Short structured notes of the text (P §3.5.1)	The model's own keys and values, read back as if just read	Reading notes back skips re-reading the text; within one room, notes answered as the words did (164 against 163 facts, E-20)
A 16 GB laptop with no dedicated graphics card, CPU inference (P §1.6)	The baseline on CPU-only 16 GB machines; the engine on a MacBook Air M4 (16 GB), then a MacBook Pro M5 Pro (24 GB)	MLX needs Apple silicon and uses its integrated graphics processor, not a dedicated card; the Pro was adopted for speed, with the Air's results kept (E-17)
Qwen2.5, Llama 3.2, Phi-3 through Ollama; a BGE embedder (P §3.6)	The baseline as proposed; the engine on Qwen3 1.7B, 4B and 8B and Qwen3-Embedding through MLX	The engine must read and write the reader's cache in its own process; one embedder serves every contender (E-02)
Ranking by similarity, recency and importance (P §3.5.1)	The baseline defaults to similarity, gate and bridging; the engine chooses the closest moment's room	The composite lowered full-support recall by 1.2 points on 2WikiMultiHopQA and 5.4 on HotpotQA (F1)
A fixed token budget (P §3.5.1)	Rooms capped at 512 tokens plus four moments re-read	Larger rooms knew fewer facts at every setting tried (E-18)
A pinned hosted reference model as the ceiling (P §3.6.3)	Kimi K3 for the baseline; Claude Haiku, Sonnet and Opus 5.5 as a yardstick for the engine	A cloud model with a one-million-token window sees the whole history early and only its end later, so it is reported beside the engine, not as a fixed ceiling
Public benchmarks only; "not an end-user application" (P §1.6)	The Long Haul, the Hugging Face suites, and a Mac app with Chat and Code	The claim is about a memory that grows over months, which no public set tests; the final project also needs a usable release

Parts of the design were planned but not built, because the remaining time went to the proof. They are future work, and no requirement above depends on them: naps that run on their own while the Mac is charging; distilling what was learnt, checked by a quiz before it is kept; settling unsure facts at recall;

reading several rooms together; and the native engine (notes rather than words) behind the pairing and the OpenAI-compatible endpoint.

4.6 **Chapter Summary**

This chapter specified CAMR in three stages, using structured, UML and storage views suited to an experimental AI artefact. It stated 26 functional, 16 non-functional, 8 data and 9 interface requirements, each with a source and a verification method, without invented thresholds, and traced them to the objectives. The analysis showed CAMR in use as a closed system on the Mac. The design described the baseline's paths over one SQLite file and the engine's memory of saved notes in rooms on the SSD, read one room at a time after a small core, with the four closest moments re-read as words, rooms moving between RAM, the SSD and sleep under fixed budgets, and a vault a person can read. The main changes from the proposal, the model's own notes instead of text, files on the SSD instead of a database, and an Apple-silicon Mac instead of a CPU-only laptop, were justified from measurements. Chapter 5 reports the implementation and the evaluation against these requirements.

Chapter 5: **System Implementation, Testing and Evaluation**

5.1 **Introduction**

Chapter 5 at a glance. What CAMR is. A language model writes text by predicting the next word, and it only knows what it absorbed in training and what is in front of it now. A model small enough for a laptop knows little and can hold only a short history in front of it. CAMR is a memory placed beside the model. Whatever the user tells it is read once by the model, and the notes the model writes to itself while reading (its keys and values) are kept on the Mac's SSD. For each question CAMR opens the one room of notes the question is about and lays it in front of the model, already read. The model itself is never changed.

What was tested. One scripted software project of 2 to 4 million tokens (the Long Haul) with 216 questions graded by rule; Qwen3 models of 1.7, 4 and 8 billion parameters with and without CAMR; a 27-billion-parameter model and Ollama reading CAMR's memory as words; Claude Haiku, Sonnet and Opus 5.5 answering alone as the yardstick; four public suites; four ways of sharing memory; and the time and RAM of every answer as memory grew.

What was found.

- At 2 million tokens, a 4B model with CAMR answered 29 of 73 questions; Claude Sonnet alone answered 32 and Opus 31. At 3 and 4 million tokens CAMR still answered 29, while Sonnet fell to 13 and then 10.
- Without CAMR, the same 4B answered 11 to 13 and needed a 15.2 GB process; with CAMR the process needed 5.4 to 7.5 GB once a graphics buffer cache was capped.
- The first word came in 0.2 to 0.3 seconds with CAMR, against 13 to 105 seconds for Sonnet and Opus.
- As memory grew from 112 thousand to 2 million tokens, the wait for the first word stayed at about 0.1 s and the RAM at about 4.3 GB (1.7B model); the SSD was held at its 40 GB budget by putting old rooms to sleep.
- Past 1M tokens most rooms were asleep, so most answers read their room as words; the saved notes bought speed where rooms were awake, not accuracy.
- CAMR helped most on facts told far back and declined almost every question about things never told. It did not help with joining two facts, or with building a page from a design decided weeks earlier; no contender, Claude included, managed that.
- Project memory was the best way to share memory (26 of 40); mixing in a second project cost 2 answers.

How to read the numbers. Most results are counts of right answers out of a set of questions, graded by rule. *Right* means the expected answer; for a question about something never told, a decline is right. *Declined* means the model said it did not know something it had been told. *First word* is the wait from the question to the first word of the answer. *Peak RAM* and *footprint* are the memory the whole process

> used. Every number comes from a dated record in the repository (`v2/records/bench/2026-11-04/`) or from the decision log (E-nn).

This chapter reports what was built, how it was tested, and what the measurements showed, including where the system failed. Section 5.2 describes the environment and Section 5.3 the data. Section 5.4 describes the implementation and the application. Section 5.5 reports testing, Sections 5.6 and 5.7 the evidence and the use of AI tools, and Section 5.8 the evaluation results, discussion and limitations.

5.2 Implementation Environment

> **In plain words.** CAMR runs on one Apple-silicon Mac. The model, the memory and the app stay on that Mac; the internet is used to download the models and public datasets once, and, in the benchmarks only, to ask the Claude yardstick.

5.2.1 Hardware and Deployment Environment

CAMR is a local prototype: a repository, an installer and an application that run on the user's own Mac. Nothing was deployed for public access. Development began on a MacBook Air M4 with 16 GB, the laptop class of the proposal's scope (Section 1.6). From 3 November 2026 every contender was run on a MacBook Pro M5 Pro, and every result of the final engine comes from that machine (Table 5.1).

Table 5.1: Hardware used (from each record's `environment` field)

Item	Specification	Purpose	Evidence
Evaluation and app machine	MacBook Pro Mac17,9, Apple M5 Pro, 24 GB unified memory, internal SSD, macOS 27.2	Every Long Haul, suite, reach and speed run; the app	`environment` of the records; E-17
First development machine	MacBook Air M4 (Mac16,12), 16 GB, fanless	Step 0 baselines and the room design (E-14 to E-16)	`records/2026-11-02/`; E-17
Minimum to run	An Apple-silicon Mac; MLX needs one. M1 to M3 Macs and 8 GB Macs were not tested	The installer refuses other machines	`scripts/install.sh`
Specialised hardware	None dedicated. MLX uses the chip's integrated graphics processor	The reader's keys and values are read and written inside the process	Table 4.9
Network	Model weights and suites downloaded once; the Anthropic API for the yardstick only	Private, offline use (NFR-10)	`bench/claude.py`; `HF_HUB_OFFLINE`

The records note the power source and the one-minute load when each run started; the 4B's 3M and 4M rerun, for example, ran on AC power at a load of 4.33. One measuring job used the graphics processor at a time, run from a queue that logs each job's start and end (E-35).

5.2.2 Software, Services and Configuration

Table 5.2 lists the software and models as tested; their versions come from each record's `environment` and `contender` fields.

Table 5.2: Software and models (versions as tested)

Component	Version	Role
Python; NumPy	3.12.12; 2.5.4	The engine, the app's engine process, the harness
MLX / mlx-lm	0.32.4 / 0.32.2	Run the models on the graphics processor and expose the key–value cache (Hannun et al., 2023)
Readers (frozen, 4-bit)	`Qwen3-1.7B-4bit`, `Qwen3-4B-Instruct-2507-4bit`, `Qwen3-8B-4bit` (A. Yang et al., 2025)	The engine's contenders
Larger local model	`Qwen3.8-27B-4bit`	Reads CAMR's memory as words (its linear-attention layers keep no keys and values)
Embedder (frozen)	`Qwen3-Embedding-0.6B-4bit-DWQ` (Zhang et al., 2025)	Every contender
Ollama	0.13.5, `qwen3:1.7b` (Q4_K_M)	The pairing: memory as words
Anthropic API	`claude-haiku-5-5`, `claude-sonnet-5-5`, `claude-opus-5-5`; effort "medium", adaptive thinking	The yardstick, alone
SwiftUI app	macOS 14 or later, built with the Swift command-line tools	The native front end (Chat, Code)
CAMR v1 as a library	camr 0.4.0 with sqlite-vec 0.1.9	The baseline contender on the scripted lives

The engine's settings are constants in the code, recorded in each record's `contender` field: rooms of at most 512 tokens, a join threshold of 0.80, four closest moments re-read, 2 GB of RAM for hot rooms and 40 GB of disk for room files in the benchmarks, the graphics buffer cache capped at 1 GB, and Qwen3's thinking off for every local model. Answers get room by family: 1,200 tokens for a page, 600 for code, 400 for maths and 120 for the rest.

5.2.3 **Deployment and Reproducibility**

The engine installs with one script on an Apple-silicon Mac, which checks the machine, creates a Python 3.12 environment, installs the engine and its models, and runs a health check that teaches a throwaway memory two dated facts and asks for one back. A `camr` command then starts the app, the Ollama pairing or the OpenAI-compatible endpoint (E-48). The Mac app is built from its Swift package and starts the engine itself. Every benchmark can be rerun with the commands in Appendix A, and every run writes a dated record to `records/bench/<date>/` after every checkpoint, naming the machine, its load and a hash of the code that made it (`camr_source_sha`). Three limits apply: records written before the rename on the evening of 4 November keep the old field names; no run was repeated for variance; and the Claude yardstick's answers depend on the provider's service on that day.

5.3 **Dataset Description**

> **In plain words.** The main test is an invented software project, told event by event over 110 working days and then 110 more, with questions about things said weeks earlier. It is fixed by its seed, so anyone can build the same one. Four public test sets add mathematics and long-text reading.

5.3.1 **The Long Haul**

The Long Haul is the study's own hard test (E-30). It is one scripted project: a developer building the website and back office of "Mama Njeri's Kitchen", a catering business in Nairobi, over 110 working days. It mixes chat, meetings, decisions, code, documents, logs, a mathematics course and noise. The script for seed 1 has 17,786 events and 2,010,094 tokens (counted with Qwen3-1.7B's tokenizer), from 22 September 2025 to 27 February 2026. For the 3M and 4M cuts it was extended by 110 more working days of noise only, from the same generators with random streams of their own; the first 110 days and every probe were kept exactly as they were, checked event for event (E-39).

Probes are planted early and asked late, and each has one answer a rule can check (Table 5.3). One example of an invented rule: the delivery fee is 1% rounded up to the next 5 shillings, at least 10, and a flat 25 for exact multiples of 1,000, so 12,000 gives 25 and 7,321 gives 75. A model cannot know such a rule from training; it can only remember it.

Table 5.3: What the Long Haul plants and asks

Family	**Probes**	**Planted**	**Asked**	**Graded**
Facts	400	Facts at random depths	Far past every window	Exact
Changed facts	80 facts, 160 questions	Each changed 2 to 4 times, with dates	The value now, and the value on a past date	Exact; an old value is stale
Two hops	40	Two halves, weeks apart	A question that needs both	Exact
Corrected maths	40 problems, 120 questions	A hard problem, a first answer with the tempting wrong method, then a step-by-step correction; some corrections state a rule invented for the project	The same problem; the method with new numbers; the invented rule on a new case	Exact number; the uncorrected method's number is stale
The page that grows	3	A design system in week 1; the main colour changes on day 61	"Build the catering page" on day 110	HTML and CSS checks
Code that must fit	20	Five helper modules with exact names, units and quirks	New code that calls them	Unit tests run on the answer, in a sandbox
Look-alikes	30	Two Wanjirus, prices one shilling apart, near-identical repository names	Aimed at one of each pair	Exact
Never said	60	Nothing	Questions on things never told	A decline is right; a guess is a false memory

The script is cut at six sizes. At each cut, a checkpoint takes a stratified sample of the probes askable there, 10 from each family (seed 0), each resolved to its right answer at that point (Table 5.4). Early cuts have fewer questions because some families are not askable yet. The 3M and 4M cuts ask the 2M cut's 73 questions again, so any change in score there is the weight of the longer history, not new questions (E-39).

Table 5.4: The cuts and their checkpoints

Cut	Tokens told	Events told	Last day	Questions
64K	63,860	551	7	34
256K	255,952	2,343	16	40
1M	999,988	8,760	56	69
2M	1,999,743	17,699	110	73
3M	2,999,988	26,244	after day 110	73 (the 2M questions)
4M	4,000,000	34,936	after day 110	73 (the 2M questions)

Every answer receives one of five verdicts (Table 5.5). The same answer always gets the same verdict, grading costs nothing, and anyone can grade again from the raw answers kept in each record.

Table 5.5: The verdicts

Verdict	Meaning
right	The expected answer; for a never-told question, a decline that does not then guess
wrong	Something else, a look-alike's value, or code or a page that fails its checks
stale	A value an update replaced, the uncorrected method's number, or the old main colour
declined	Says it does not know, but the answer was told
false memory	Answers a question about something never told, or declines and then guesses

Synthetic-data risk. A scripted project cannot establish real-world performance on its own. Each fact is told once or a few times, the questions follow templates, and there is one project. The Long Haul compares memories fairly on such a history, built to be hard in known ways; the public suites and the baseline's public benchmarks are the real-data complement.

5.3.2 Public Suites from the Hugging Face Hub

Four public suites were loaded, sampled with a seed and graded by their own rules (Table 5.6). They were run on the Claude yardstick and on Ollama, paired and bare; the engine's own MLX contenders were not run on them, because their instructions are the Long Haul's and a suite needs its own (E-36).

Table 5.6: The public suites as run

Suite	Source	What it tests	How it was run
BABILong	RMT-team/babilong (Kuratov et al., 2024)	bAbI facts hidden in book text	qa1 to qa5 at 4k, 16k, 64k and 128k tokens, 100 items; the story told in chunks of about 512 tokens, then its question

MATH-500	HuggingFaceH4/MATH-500 (Lightman et al., 2024)	Competition problems	50 problems of levels 3 to 5, no memory; graded by the last boxed answer's value
AIME	MathArena aime_2025 and aime_2026 (Balunović et al., 2025)	30 problems a year, each a whole number 0 to 999	60 problems, no memory; exact
HARP pairs	aadityasingh/HARP (Yue et al., 2024)	Problem A and its worked solution as context, then a new problem B of the same subject and level	60 pairs; memory on: A and its solution told, 30K tokens of Long Haul noise, then B and A asked; memory off: B and A with nothing told

5.3.3 **Data of the Earlier Stages**

The rooms build was developed and judged on three scripted twelve-week lives of invented people (a university student, a dairy-cooperative manager and a software developer), each with about 330 notes and the same 100 questions every week: 45 single facts, 15 updated facts, 10 facts told across two notes, 15 dates and 15 things never told (E-05). Development lives (seed 0) were used for every design choice, and held-out proof lives (seed 1) were run only after the pass marks were written (E-19). The baseline used PopQA, HotpotQA, 2WikiMultiHopQA and GSM8K with fixed samples (seed 13), a store built from supporting and distractor paragraphs only, and a 200-question held-out set for its final claims.

5.4 **System Implementation**

> **In plain words.** The engine is a Python package of a few thousand lines. A moment is read once by the model and its notes are appended to a room file; a question picks one room, reads it from RAM or the SSD, and the model reads only a few lines more before it answers.

5.4.1 **The Engine's Modules**

The implementation follows Figures 4.5 and 4.6. Table 5.7 lists the engine's main modules with their sizes on 4 November.

Table 5.7: The engine's main modules (`v2/camr_engine/`)

Module (lines)	**What it does**	**Requirements**
`rooms.py` (137)	Places a moment by Equation 4.3; appends its notes to the room file; fetches a whole room in 1 MB pieces, eight at once, with the cache bypassed; cuts a room file back to its indexed length on opening (E-27)	FR-08, FR-09, IR-03
`notes.py` (101)	Turns tokens into notes and notes into a desk; refuses scaled rotary positions; turns keys to a new position	FR-07, IR-02
`memory.py` (406)	Remember, choose, recall, answer and fade; composes the parts below	FR-07, FR-12
`swap.py` (301)	The tiers: hot rooms in RAM under a budget (least recently used first out), awake rooms on the SSD, sleep past the disk budget, waking	FR-11, FR-13, NFR-04
`vault.py` (613)	One Markdown note per moment; doors between notes; strength fading 5% a day; reads a person's edits back	FR-10

`core.py` (92)	The core: at most 256 tokens of notes tagged `core`, kept in RAM	FR-14
`reach.py` (318)	Sessions over several reaches; one lock per reach folder; the memory switch	FR-15, FR-16, NFR-12
`metrics.py` (169)	One record per answer (Table 4.8); medians and 90th percentiles	FR-17
`events.py` (104); `live.py` (185)	The event log with attention per written word; the live events the app watches	FR-19, FR-21
`models.py` (245)	Loads the models through MLX, layer by layer; greedy decoding, thinking off; caps the graphics buffer cache	NFR-05, NFR-09
`pair/` (1,039)	The Ollama pairing proxy and the OpenAI-compatible endpoint, with their security checks and metrics	FR-20, IR-06, IR-07
`app/` and `agentic/`	The engine's server and its JSON routes, the Code agent's tools, workspace and approval gate	FR-18, IR-04, IR-05, NFR-11
`bench/`	The Long Haul generator and grader, the local, Ollama and Claude runners, the suites, the reach and RAM runners, the cost ledger	FR-22 to FR-26

5.4.2 **Write Path, Read Path and Tiers**

Two implementation choices from the rooms build matter for every result. Each room is written as one continuous page, so a new moment is read with its room's earlier moments in view (E-15). A single room is laid on the desk without turning its keys, which cut laying the desk from 55 ms to 15 ms at 2,000 words (E-18). The final engine added three things to that path. Every moment carries its date and day number, as `[YYYY-MM-DD, day N] text`, since the Long Haul asks about days by number. Rooms move between tiers: in the benchmarks, 2 GB of RAM held the hot rooms and 40 GB of disk the awake room files, and at the end of the 2M-token history 920 rooms were awake and 4,471 asleep (E-38). A question whose room is asleep reads that room's moments as words, which costs reading them as plain text would; waking was off in the benchmarks. Finally, the benchmark's questions were read-only: the hot rooms, the least-recently-used clock and the vault strengths were restored after every answer, so that asking never teaches (E-31).

5.4.3 **The Vault, the Core, Reach and the Switch**

Every moment is also written to the vault as a Markdown note whose front matter holds its id, day, date, room, reach, strength and links. The vault opens in Obsidian as it is, and a note edited by hand is read again because its file changed. In the benchmarks the core was on but empty, since every moment was told untagged. A `Session` reads several reaches and writes to one; each reach is a folder of its own under `~/.camr/`, locked while a process has it open, and a question picks the best room over every reach it reads. With the switch off, the model answers alone with the same prompt and no notes, and the answer still gets a metrics record, so on and off compare directly.

5.4.4 **Memory for Other Runtimes: the Ollama Pairing**

Ollama runs its own model and lets no outside code put keys and values into it, so the engine gives it memory as words. A proxy on 127.0.0.1:11435 speaks Ollama's own API; before a chat is forwarded to the real Ollama, the proxy recalls the closest notes on the last user message and puts them, at most 8 notes and 512 tokens, at the front of that message (E-33). After the answer has streamed back, the exchange is remembered on a background worker. The same code serves an OpenAI-compatible endpoint on 127.0.0.1:11436 in front of Ollama, LM Studio or mlx_lm.server (E-47). Through these doors memory goes in as words; the engine's notes need the engine.

5.4.5 **The Application**

The application described in Section 4.4.8 is a Swift package of seven source files (about 1,500 lines) that builds into `CAMR.app`, signed for the Mac it was built on. It starts the engine as a local process with the chosen model (Qwen3 1.7B, 4B or 8B), a free port and a token only it knows, waits for the engine's line saying it holds that port before it sends the token, and then talks to it over the loopback address. Each model has its own memory folder, since notes belong to the model that wrote them. The screenshots in Figures 4.11 to 4.15 were captured from the running app with Qwen3 4B and a real memory of 737 rooms.

5.4.6 **The Benchmark Harness and the Contenders**

The harness drives every contender through the same protocol: it is told the history one event at a time (`tell(text, day)`) and asked each question (`ask(question, max_tokens, family)`). Table 5.8 names the contenders as the records do.

Table 5.8: Contenders on the Long Haul

Name	**What it is**
`engine:qwen3-1.7b`, `engine:qwen3-4b`, `engine:qwen3-8b`	CAMR: one project reach's memory with the vault, the core, the tiers and the metrics on, driven as a `Session` drives it; `+own` gives the 1.7B the engine's own instructions
`words:qwen-27b`	CAMR's rooms, chosen the same way, pasted in as text, for the 27B model
`bare:<model>`	No memory: the newest part of the history that fits (32K tokens for the 1.7B and 8B, 64K for the 4B), so that its keys and values fit in 24 GB
`ollama:qwen3:1.7b`; `ollama-bare:qwen3:1.7b`	Ollama through the pairing (memory as words); plain Ollama with the newest 32,768 tokens
`claude-haiku-5-5`, `claude-sonnet-5-5`, `claude-opus-5-5`	Claude alone: the newest part of the history that fits its one-million-token window, with room for the questions and the reply

The yardstick runner sends a checkpoint's questions in one streamed request that asks for the answers as JSON, with the history cached where the last request ended. A reply cut off by its own token limit is not graded wrong: the questions it never reached are asked again, alone, with twice the room. No fallback model is ever asked for, so each record names the model that answered. Before each call, its

worst-case cost is held in a ledger, and the call is refused if it would pass the night's $60 cap less $2 of headroom.

5.5 **Testing**

> **In plain words.** Automated tests checked each part; benchmark runs checked the whole system against requirements written beforehand; and defects found by the measurements were fixed and the runs repeated. Two requirements were not met and are kept as written.

5.5.1 **Test Strategy**

Testing used five levels:

i. **Unit and integration tests.** White-box pytest tests in 45 files cover the rooms, notes, tiers, vault, core, reach, metrics, events, pairing, the app's routes and security, the Code agent, and every part of the benchmark harness (the Long Haul's generator and grader, the runners, the suites and the cost ledger). On 4 November, 730 tests passed.

ii. **System tests.** The benchmark runs were judged against the non-functional requirements of Table 4.3, whose thresholds were written before the runs that tested them (E-19, E-40).

iii. **Security tests.** Requests with a foreign host, a foreign origin, no token or a non-JSON body were sent to the engine's server and the proxies.

iv. **Deployment test.** The installer and the health check were run, and the Mac app was built and run with the 4B model.

v. **Demonstration.** Chat and Code were used with a real memory, and the screens were captured from the running app (Section 5.4.5).

User acceptance testing and an accessibility audit were not done.

5.5.2 **Test Cases and Results**

Table 5.9 gives each test case with its requirement, its expected and actual result and its verdict.

Table 5.9: Test cases and results

Test ID	Traceability	Scenario	Expected result	Actual result and evidence	Verdict
T-01	All FR	The automated suite	Every test passes	730 passed (4 November, before Swarm was removed)	Pass
T-02	NFR-01	Speed bench, 2,000 words in the room, cache bypassed, five tries	Median first word under 0.5 s	0.233 s (E-20)	Pass
T-03	NFR-02	The same bench, notes against words	At least 10 times sooner	4.6 times (E-20)	Fail (kept)

T-04	NFR-03	The 1.7B told the whole Long Haul, measured at 1K, 10K and 17.7K moments	First word and RAM do not climb	First word 0.11, 0.15, 0.08 s; RAM 4.26, 4.28, 4.23 GB (E-38)	Pass
T-05	NFR-04	The same run	Disk within 40 GB; hot rooms within 2 GB	Disk 40.0–40.1 GB, held by sleep; hot rooms 1.95–1.99 GB	Pass
T-06	NFR-05	The 4B at 3M, with and without the 1 GB cap	Footprint falls; answers unchanged	19.2 → 7.5 GB; identical verdicts (E-41)	Pass after fix (D-02)
T-07	NFR-06	The 4B at 3M	Retrieval at most 10% of answer time	Retrieval p50 16 ms of a 0.39 s median answer (4%)	Pass for the 4B
T-08	NFR-07	Proof lives, week 12: notes against the same desk as words	≥ 95% of the facts; ≤ 3 extra false memories	164 against 163 facts; 25 against 24 false memories (E-20)	Pass
T-09	NFR-08	Never-said questions at every cut	A decline is graded right, a guess false memory	Graded as specified (grading tests); the 4B declined 9 of 10 at 2M and 10 of 10 at 3M and 4M	Pass
T-10	NFR-09	The 4B's 3M checkpoint run twice	Identical verdicts	29 of 73 both times, every verdict the same (E-41)	Pass
T-11	NFR-11	Negative requests to the server and proxies	Only the app's own requests are served	The app's 12 security tests and the pairing's 35 tests, which include its security checks, pass	Pass
T-12	NFR-12	A second process opens a reach folder	Refused	Refused with `ReachBusy` (reach tests)	Pass
T-13	NFR-14	The night's yardstick runs	Spend under $60	$28.71, every call in the ledger	Pass
T-14	NFR-15	Inspection of the records of 4 November	Machine, load, versions, commit and code hash in each	Present in every record	Pass
T-15	FR-18, FR-19	The Mac app with Qwen3 4B	Chat and Code work with memory on and off; the weights view moves only with measured events	Figures 4.12 to 4.15	Pass (demonstration)
T-16	FR-20	A fact told through the OpenAI-compatible endpoint, asked back through the real Ollama	The fact comes back	"Pilipili", with 187 characters of notes (E-47)	Pass
T-17	FR-22	The Long Haul built twice from seed 1, and extended	Identical scripts; the first 110 days unchanged by the extension	Generator tests pass; checked event for event (E-39)	Pass

T-18	Users	User acceptance; accessibility audit		Not run	Not tested

5.5.3 **Defects Found and Retests**

The measurements found six defects in the system or in its measurement (Table 5.10). Each was fixed or bounded, and the affected runs were repeated or reported both ways.

Table 5.10: Defects found and retests

ID	Defect and how it was found	Correction	Retest
D-01	The 1.7B declined facts sitting on its desk at 1M and 2M; diagnosis ruled out retrieval and sleep and found the yardstick's wording (E-37)	The 1.7B is run with both wordings; the others keep the yardstick's, as planned	Both reported (Section 5.8.6)
D-02	The engine's process held 19.2 GB, not the 5.7 GB the records showed: MLX kept freed graphics buffers up to the whole limit and swap filled (E-40)	The buffer cache capped at 1 GB; every record now logs the process's footprint	7.5 GB, identical answers (E-41)
D-03	Ollama, bare, BABILong: 58 of 100 answers thrown out as "thinking" with no thinking in them (E-43)	An empty think block is cut off the answer; real thinking is still an error	The 58 resumed (Section 5.8.13)
D-04	Ollama, HARP: at the shared 400-token limit, 82 of the bare 1.7B's 120 answers and 95 of the paired one's were cut off mid-working (E-43)	The same room to work in words as Claude has to think (8,192 tokens); the 400-token records kept	Rerun (Section 5.8.13)
D-05	Ollama's `qwen3:4b` had become the Thinking build (E-33)	The 4B runs as `qwen3:4b-instruct-2507-q4_K_M`	Pass
D-06	A crash between appending a moment's notes and saving the index would misplace later moments (E-27)	Opening a memory cuts each room file to its indexed length; a test covers it	Pass

5.6 **Version Control and Implementation Evidence**

The project lives in one Git repository on GitHub (https://github.com/Satasdin/CAMR), which holds the baseline at its root and the engine in `v2/`, with their full histories. The repository is private, so the supervisor and examiners must be given access. The final engine, its records and this report are on the branch `camr-build`. The repository holds the README, the user guide and the decision log; the installer, the scripts and the tests; every benchmark record and the Claude ledger; the Mac app's Swift package; and the generator scripts of this report's figures. `.gitignore` excludes the virtual environment, model weights, datasets and `work/`, where contenders' memories are rebuilt on every run. Each benchmark job of the night committed its records as it finished, so the history shows when each number was made.

5.7 **Author Oversight and Use of AI-Assisted Tools**

Claude (Anthropic), used through Claude Code, assisted with this project. Under the student's direction it wrote and revised the engine, the app, the harness and their tests; ran the experiments; and

drafted the documentation, including the decision log, the user guide and drafts of this report. The student set the problem, the objectives, the scope and the direction of the work, and made the calls recorded as his own, such as making the engine the project (E-29), pushing the Long Haul past 2 million tokens (E-39), asking for the RAM to be optimised (E-40) and removing Swarm from the app (E-50). The output was checked by running the automated tests, by the records, which name the machine and the code behind every number, and by reading the outputs themselves. Requirements were written before the runs that tested them, and the requirements that were not met are reported as measured. Claude also served, separately, as the yardstick in the benchmarks; that role is unrelated to its role as a tool. The student made the final decisions and takes full responsibility for the work. Student to confirm this statement and add the model versions used.

5.8 Evaluation Results, Discussion and Limitations

> **In plain words.** A 4B model with CAMR answers a long project's questions about as well as Claude Sonnet alone at 2 million tokens, and better past Claude's window, at a fraction of the wait and in a process of 5 to 8 GB. It stays as fast as memory grows. It is weakest at joining facts and at following a design over weeks.

5.8.1 Results at a Glance

Table 5.11 gives each claim of the study a verdict, with the section that supports it.

Table 5.11: Scorecard: each claim, its evidence and whether it was met

Claim	Evidence	Result	Verdict
A small model with CAMR answers like a cloud model on a long history	Long Haul at 2M (Section 5.8.2)	4B + CAMR 29 of 73; Sonnet alone 32, Opus 31, Haiku 10	Nearly met at 2M
Past the cloud model's window, memory wins	Long Haul at 3M and 4M (Section 5.8.3)	4B + CAMR 29 and 29; Sonnet 13 and 10; Haiku 10 and 10	Met
Memory beats the same model without it	Long Haul, every cut	4B: 25/26/25/29 against 18/14/20/11; 8B: 26/26/24/27 against 15/11/15/11	Met
Lighter than the model reading its own window	Footprint (Section 5.8.9)	5.4–7.5 GB against 15.2 GB (4B); 8.1–8.8 GB against 13.6 GB (8B)	Met, after the cap (D-02)
No slower as memory grows	Speed as memory grows (Section 5.8.8)	First word 0.11 → 0.08 s and RAM 4.26 → 4.23 GB from 112K to 2.01M tokens	Met
Faster than the cloud	First word (Section 5.8.11)	0.2–0.3 s against 13–105 s for Sonnet and Opus	Met
Honest about what was never said	Never-said family	4B declined 9 of 10 at 2M and 10 of 10 past it; Claude 10 of 10	Met for the 4B; not for the 1.7B with its own wording

Memory recovers knowledge, not reasoning	Families at 2M (Section 5.8.4)	Facts 6 of 10 (Sonnet and Opus 4, Haiku 0); two hops 0; the growing page 0 for everyone	Met as predicted
Notes and words know the same within a room	Rooms build (Section 5.8.15)	164 against 163 facts	Met
Notes are at least 10 times sooner than words	Speed bench (Section 5.8.15)	4.6 times at 2,000 words	Not met
The best reach	Reach (Section 5.8.10)	Project 26 of 40; global 24; session 11	Project reach
Files remembered: PDFs, images, charts, video	Multimodal memory (Section 5.8.14)	29 of 29 planted facts with CAMR, for the vision model and the 4B; 0 for the vision model alone	Met, on a small test

5.8.2 **The Long Haul: Right Answers as the History Grows**

> **In plain words.** Up to a million tokens Claude Sonnet sees most of the history and answers the most. At 2 million the 4B with CAMR is three answers behind it. Without memory, the same 4B falls to 11.

Table 5.12 gives every contender's right answers at every cut, with its median wait for the first word and its peak RAM, and Figure 5.1 plots them as shares, since the cuts ask different numbers of questions.

Table 5.12: Right answers on the Long Haul (questions asked: 34, 40, 69 and 73; 3M and 4M ask the 2M questions)

Contender	**64K**	**256K**	**1M**	**2M**	**3M**	**4M**	**First word p50**	**Process RAM**
CAMR, Qwen3 4B	**25**	**26**	**25**	**29**	**29**	**29**	0.21–0.31 s	7.5 GB (capped, at 3M)
Qwen3 4B alone (newest 64K)	18	14	20	11	12	13	0.32–0.38 s	15.2 GB
CAMR, Qwen3 8B	26	26	24	27			0.32–0.54 s	8.1–8.8 GB
Qwen3 8B alone (newest 32K)	15	11	15	11			0.22 s	13.6 GB
Qwen 27B, CAMR's memory as words	24	21	24	27			2.2–2.4 s	16–17 GB
CAMR, Qwen3 1.7B, its own wording	19	20	21	21			0.10–0.14 s	4.5 GB†
CAMR, Qwen3 1.7B, the yardstick's wording	14	20	17	15			0.10–0.14 s	4.5 GB†
Qwen3 1.7B alone (newest 32K)	8	13	14	7			0.13 s	7.5 GB†
Ollama 1.7B with the pairing (words)	16	17	16	16			0.38–0.40 s	3.3 GB†

Ollama 1.7B alone (newest 32K)	8	12	10	7				5.7 GB†
Claude Haiku 5.5 alone	22	21	17	10	10	10	2–43 s	
Claude Sonnet 5.5 alone	28	35	43	32	13	10	13–70 s	
Claude Opus 5.5 alone	24	33	42	31			19–105 s	

Process RAM is the process's physical footprint where it was recorded; the figures marked † are peaks measured before the buffer-cache cap (MLX's own peak for the engine, the peak resident memory for Ollama), not process footprints, and may understate the process (Section 5.8.9). The Claude rows have no RAM figure, since Claude runs in the cloud.

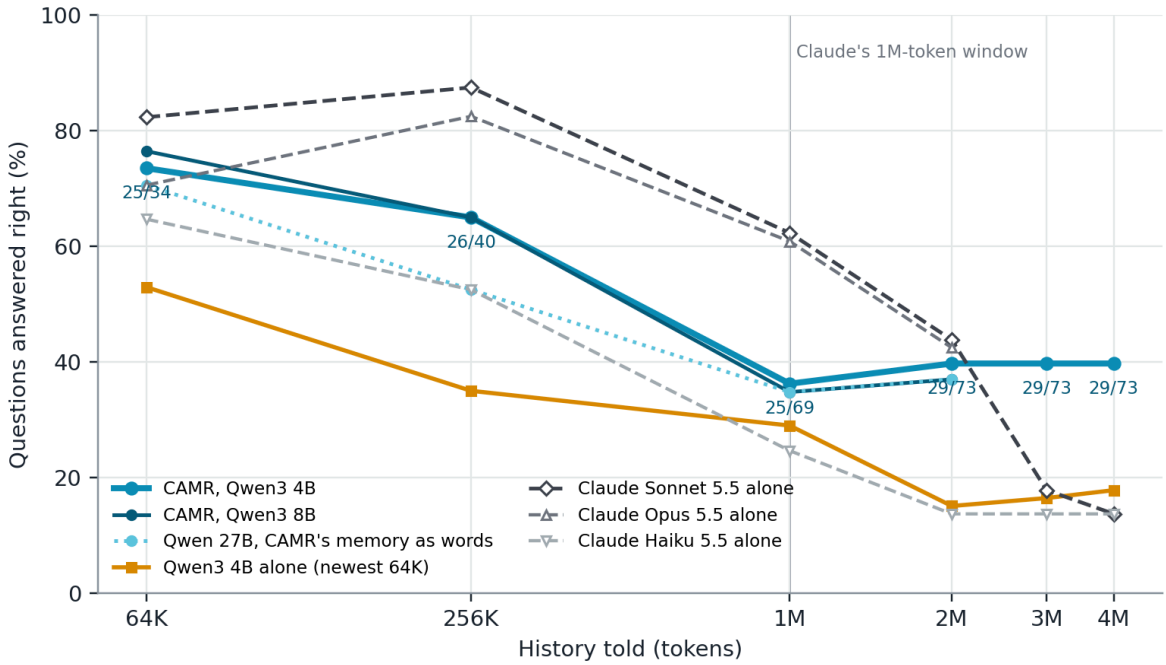


Figure 5.1: Share of questions answered right as the history grows. CAMR (cyan) holds its level past 1M tokens; the models alone (amber) and Claude alone (grey, dashed) fall once the history outgrows their windows

Four patterns stand out. First, memory roughly doubles what a local model answers once the history outgrows its window: at 2M, the 4B with CAMR answered 29 against 11 for the same model alone, and the 8B 27 against 11. Second, while most of the history fits in Claude's window, Sonnet and Opus answer more: up to 1M, Sonnet answered 28 to 43 and Opus 24 to 42, against CAMR's 25 to 26 at the same cuts, while Haiku (22, 21 and 17) trailed CAMR throughout. Already at 1M the history (counted with Claude's own tokenizer) no longer fitted whole, and Claude's request was cut from the front. Third, at 2M the gap between the 4B with CAMR and Sonnet alone is 3 answers of 73. Fourth, the larger readers did not do better with CAMR's memory than the 4B: the 8B answered 27 at 2M and the 27B, reading the memory as words, 27. On this test the memory, not the reader, sets the ceiling.

Where each answer's room came from matters for reading these results (Table 5.13). Up to 256K tokens almost every answer read its room as saved notes, from the SSD or from RAM. By 1M tokens the room files had reached their 40 GB disk budget, sleep had put most rooms away as words, and most answers read their room as words: 47 of 69 at 1M, 68 of 73 at 2M, and 71 and 72 of 73 at 3M and 4M. The engine's results past 1M are therefore the results of its memory's organisation (rooms chosen by similarity, the closest moments, dates on every moment) read mostly as words, with the saved notes doing the work only for the rooms that were still awake.

Table 5.13: Where the 4B's room came from at each cut (`summary.all.tiers` and `status` in the records)

Cut	Notes from RAM	Notes from the SSD	Asleep: read as words	Rooms awake / asleep
64K	6	28	0	237 / 0
256K	1	34	5	676 / 61
1M	2	20	47	703 / 1,994
2M	3	2	68	807 / 4,932
3M	0	2	71	987 / 8,368
4M	0	1	72	1,152 / 12,669

5.8.3 **Past the Window: 3M and 4M**

> **In plain words.** Pushed to 3 and 4 million tokens, Claude can only see the last days, which have nothing planted, and it answers little beyond the questions about things never said. CAMR answers exactly as it did at 2 million.

At 3M and 4M the same 73 questions were asked under one and two million more tokens of noise (E-39). The 4B with CAMR answered 29 both times, as at 2M, and declined all 10 never-said questions, with a median first word of 0.31 s. Claude Sonnet answered 13 and then 10, and Haiku 10 and 10: their windows held only the newest days, so they declined almost every question, and what they scored was mostly the never-said questions, which a decline answers right (Sonnet also answered 3 corrected-maths questions at 3M). The 4B alone answered 12 and 13, from the newest 64K tokens. A memory that lives outside the window therefore keeps its answers however long the history grows, while a window, however large, forgets the beginning.

5.8.4 **What Was Answered: Knowledge Against Reasoning**

> **In plain words.** CAMR is best at facts told long ago and at code that must use the project's own helpers. It does not join two facts told weeks apart, and nobody built the page that needed a design from week 1 and a colour change from day 61.

Table 5.14 and Figure 5.2 break the 2M checkpoint down by family.

Table 5.14: Right answers at 2M by family (10 questions each; for "never said", declining is right)

Family	CAMR 4B	CAMR 1.7B (own wording)	Claude Haiku	Claude Sonnet	Claude Opus
Facts across the gap	**6**	**6**	0	4	4
Changed facts	3	2	0	5	5
Look-alikes	3	3	0	5	5
Corrected maths	4	4	0	6	6
Code that must fit	**4**	2	0	2	1
Two hops	0	1	0	0	0
Never said	9	3	10	10	10
The page that grows (3)	0	0	0	0	0

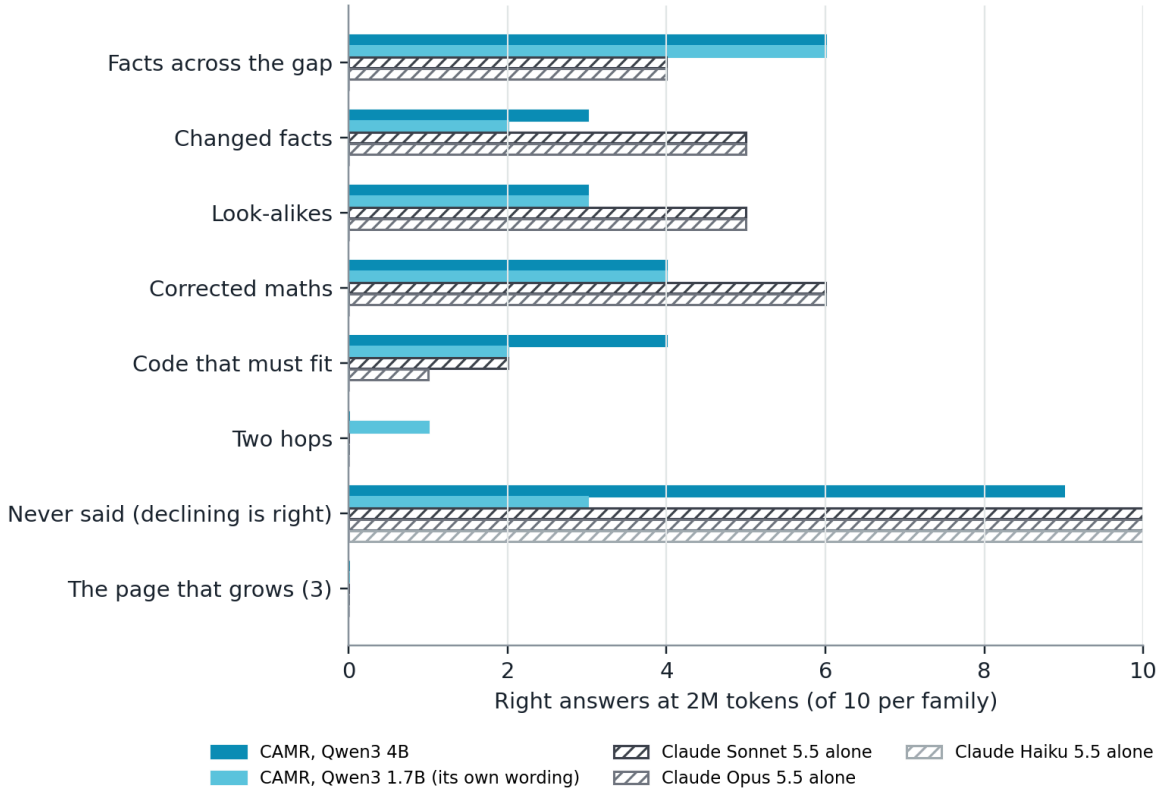


Figure 5.2: Right answers at 2M tokens by family: CAMR (cyan) against Claude alone (grey, hatched)

The pattern repeats the baseline's finding on public benchmarks (Section 5.8.16): memory recovers knowledge, not reasoning. CAMR led on facts planted far back (6 of 10 against Claude's 4), because Claude's window no longer held them, and on code that had to call the project's own helpers with their exact names and quirks (4 against 2). Claude led where the answer needs weighing several moments against each other: changed facts, where the newest of several values is right; look-alikes, where two near-identical names must be told apart; and corrected mathematics, where a correction must override a first answer. One room per question seldom holds every version of a changed fact, and a small reader takes the first it sees. Two hops were answered by nobody but once, by the 1.7B, and the

growing page was failed by every model, Claude included. It is the hardest probe: it needs the week-1 design system and the day-61 colour change together.

5.8.5 **Larger Readers: 8B and 27B**

The 8B with CAMR answered 26, 26, 24 and 27, level with the 4B, with a slower first word (0.32 to 0.54 s) and a larger process (8.1 to 8.8 GB). The 8B alone answered 15, 11, 15 and 11 from its newest 32K tokens, in a 13.6 GB process. The 27B cannot use saved notes, because 48 of its 64 layers keep a running state instead of keys and values (E-26); given the same rooms as words, it answered 24, 21, 24 and 27, at 2.2 to 2.4 s to the first word in a 16 to 17 GB process. A reader twice or about seven times the 4B's size, given the same memory, did not answer more. The memory sets the ceiling here, and the 4B is the best trade of answers against time and RAM on this Mac.

5.8.6 **The 1.7B: Its Wording, Not Its Memory**

> **In plain words.** The smallest model did badly at first. The cause was not its memory but the instructions: told to answer only from its notes and to say "I don't know" otherwise, it said "I don't know" even when the answer was in front of it.

With the yardstick's instructions, the 1.7B with CAMR scored on facts 2 of 10 at 64K, 5 at 256K, 1 at 1M and 0 at 2M; most of its total came from declining the never-said questions (E-37). Three checks on its own 2M memory found the cause (Table 5.15).

Table 5.15: Why the 1.7B missed facts at 2M (E-37)

Check	Finding	Conclusion
Was the right room chosen?	The room holding the planted fact was chosen for 6 of the 10 facts	Retrieval was not the cause
Did sleep lose it?	37 of 40 recall answers hit an asleep room; waking each room first gave 2 of 40 both ways	Sleep was not the cause
Was the fact on the desk?	Yes, sometimes twice (for example "Muthoni Wafula is working off a Lenovo Yoga 7"); the 1.7B still answered "I don't know", even with the same words as plain text in a fresh prompt	The wording was the cause

With the engine's own wording, written for the scripted lives before any Long Haul result, the same 1.7B answered 19, 20, 21 and 21 against 14, 20, 17 and 15, and 6 of 10 facts at 2M. It also invented more: it declined only 3 of the 10 never-said questions at 2M, against 7 with the yardstick's wording (and, on the same 2M desks read as words, 3 against 8; E-37). The 1.7B's honesty is a property of its wording, not of its memory. The 4B does not have this problem: it keeps the yardstick's wording and is both right and honest. No wording was picked after seeing it work on these questions; the 4B, 8B and 27B kept the yardstick's wording as planned.

5.8.7 **Notes Against Words: What the Tensors Buy**

The same 1.7B, with the same wording, can be given its memory two ways: as notes through the engine, or as words through Ollama and the pairing. As notes it answered 14, 20, 17 and 15; as words,

16, 17, 16 and 16. The first word came three times sooner as notes (0.10 to 0.14 s against 0.38 to 0.40 s), but the notes took 40 GB of disk against 0.1 GB for the pairing's words. On this test the tensors buy speed, not accuracy, though the comparison is clean only at 64K and 256K: at 1M and 2M most of the engine's rooms were asleep, and 41 and 69 of its answers read their room as words (Table 5.13). The rooms build had found the same within one room, where notes and words knew 164 and 163 facts (Section 5.8.15).

5.8.8 **Speed as Memory Grows**

> **In plain words.** The wait for the first word did not grow as the memory grew eighteenfold. RAM stayed flat. Only the disk filled, and sleep held it at its budget.

The 1.7B was told the whole Long Haul and measured at 1,000, 10,000 and 17,676 moments, with 5 questions at each point (Table 5.16, Figure 5.3).

Table 5.16: The wait as memory grows (`engine:qwen3-1.7b`; E-38)

Moments kept	History	First word, p50	Retrieval, p50	Peak RAM	Disk	Rooms awake / asleep
1,000	112K tokens	0.11 s	24 ms	4.26 GB	14.9 GB	358 / 0
10,000	1.15M tokens	0.15 s	17 ms	4.28 GB	40.0 GB	864 / 2,146
17,676 (the end)	2.01M tokens	0.08 s	18 ms	4.23 GB	40.1 GB	920 / 4,471

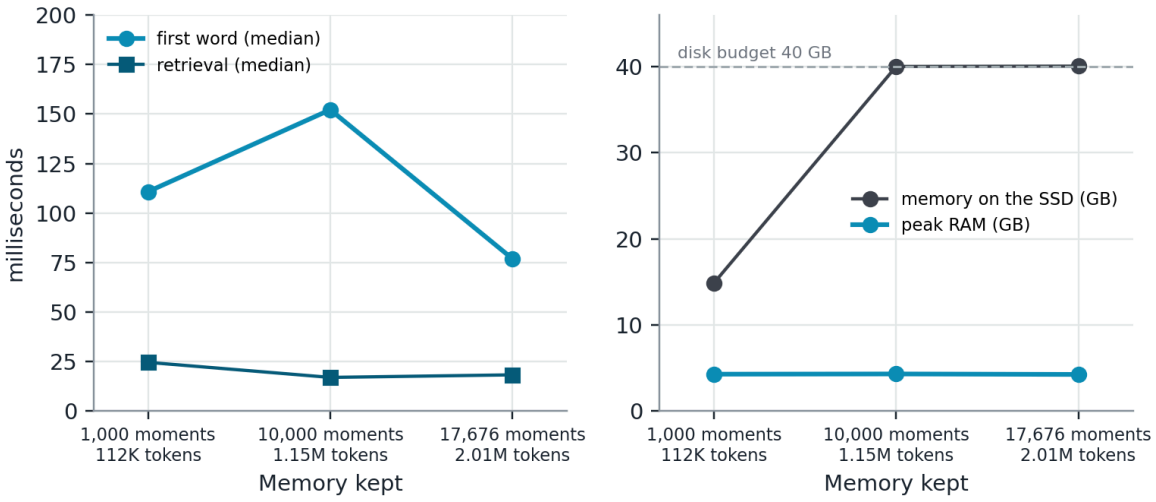


Figure 5.3: The wait, retrieval time, RAM and disk as the 1.7B's memory grows from 1,000 to 17,676 moments

The first word stays at about 0.1 s because the desk is always one room of at most 512 tokens plus four moments, however large the memory grows. Retrieval, one product of the question with every moment's embedding and one read of a room, took 17 to 24 ms. Reading the history in took about 1,100 tokens a second throughout. The 90th-percentile first word (2 to 5 s) is a question whose room

is asleep or whose answer is long, read as words. Rooms not hot were read off the SSD at 6 to 12 GB/s. At the 2M cut, Claude waited 16 to 105 s for its first token (Haiku 15.9 s, Sonnet 53.5 s, Opus 105.2 s).

5.8.9 **RAM: the Cap on the Buffer Cache**

> **In plain words.** The engine first seemed to use 5.7 GB. A closer look showed the whole process holding 19 GB, because the graphics library kept freed buffers. Capping that cache cut the process to 7.5 GB and made it almost twice as fast, with the same answers.

During the 4B's 4M-token run, the process's physical footprint (what Activity Monitor shows) was 19.2 GB, of which only 1.0 GB was resident; the rest sat compressed or in swap, which was full (8.8 of 9.2 GB). The records' "peak RAM" of 5.7 GB was MLX's peak of arrays in use during one answer, so it never saw this. The cause was MLX's cache of freed graphics buffers, which is unbounded up to the graphics limit (about 18 GB here), while a long ingest frees a desk's worth of buffers for every moment (E-40). The cache is now capped at 1 GB when a reader or embedder loads, which changes no computation, only how many freed buffers are kept for reuse. Table 5.17 and Figure 5.4 give the effect.

Table 5.17: The 4B at 3M, before and after the 1 GB cap (E-41)

Measure	**No cap**	**1 GB cap**
Right answers (of 73)	29	29, every verdict identical
Process footprint	19.2 GB	7.5 GB
Reading the 3M history	99.5 min (503 tokens/s)	53 min (938 tokens/s)
First word, p50	0.37 s	0.31 s
Retrieval, p50	20 ms	16 ms

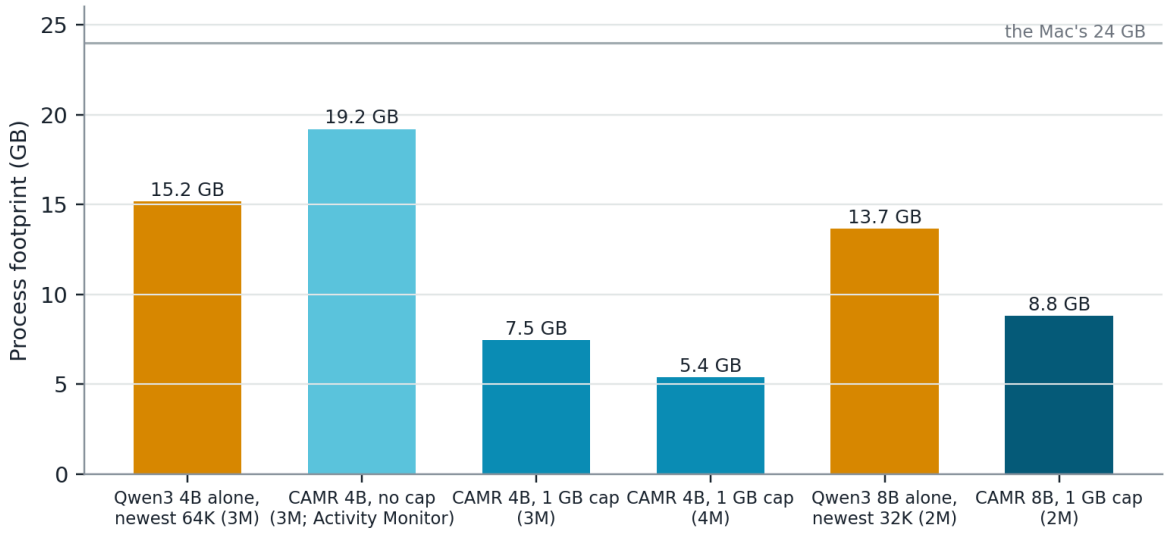


Figure 5.4: Process footprint: the models alone (amber) against CAMR (cyan), before and after the cap

The 7.5 GB is about what the parts add up to: the 4B's weights, the embedder, the 2 GB pool of hot rooms, the 1 GB cache and Python. At 4M the footprint was 5.4 GB. The 8B, the 27B and the Ollama suite runs were the first measured with the cap. Every run before them (the 1.7B and the 4B up to 2M) ran without it; their scores are unaffected, but their RAM figures are understated.

5.8.10 **Which Reach Works Best**

> **In plain words.** Keeping one memory per project is best. A memory per conversation forgets everything said in other weeks, and mixing two projects in one memory sometimes takes the wrong project's rule.

The 4B with CAMR heard the Long Haul's first 256K tokens four ways and was asked the same 40 questions; the other project was the Long Haul built from seed 2 (Table 5.18, Figure 5.5).

Table 5.18: Which reach works best (`engine:qwen3-4b`, 256K tokens, 40 questions; E-42)

Set-up	**Right (of 40)**	**Facts (of 10)**	**First word p50**	**Disk**
Session: each working week its own memory, asked in the last	11	0	0.24 s	44 GB
Project: every week into one project memory	**26**	**8**	0.19 s	44 GB
Global: both projects in one memory	24	8	0.19 s	85 GB
Project + global: both read, the closest room wins	24	7	0.19 s	85 GB

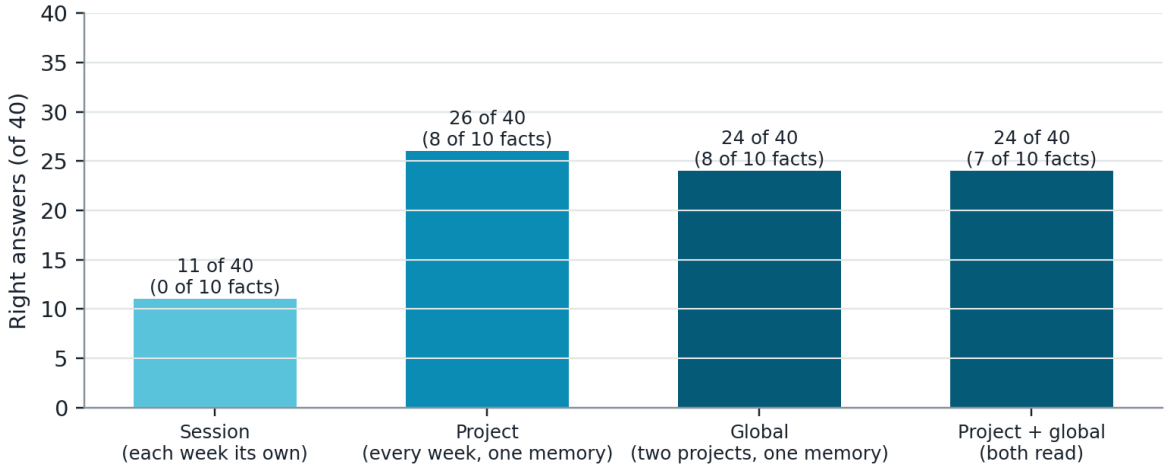


Figure 5.5: Right answers by reach

A session's memory holds only its own week, so 10 of its 11 right answers were never-said questions declined. In global memory, two mathematics answers went stale because they took the other project's look-alike rule. When both were read, the other project's room won 7 of the 40 questions and cost exactly 2 answers, a fact and a changed fact, which the 4B declined rather than answer with the other project's facts. Every set-up declined all 10 never-said questions. The app's default therefore stays project reach.

5.8.11 **The Claude Yardstick: Time and Cost**

Claude was only a yardstick: it answered the same questions alone, with no memory. Table 5.19 gives what it cost on the Long Haul.

Table 5.19: The Claude yardstick on the Long Haul (from the records)

Model	**Right, 64K–2M (of 216)**	**Right, 3M and 4M (of 146)**	**First token, per request**	**Cost on the Long Haul**
Claude Haiku 5.5	70	20	2.2–43.2 s	$1.13 + $0.97
Claude Sonnet 5.5	138	23	12.7–70.0 s	$4.92 + $3.87
Claude Opus 5.5	130	not run	19.3–105.2 s	$9.86

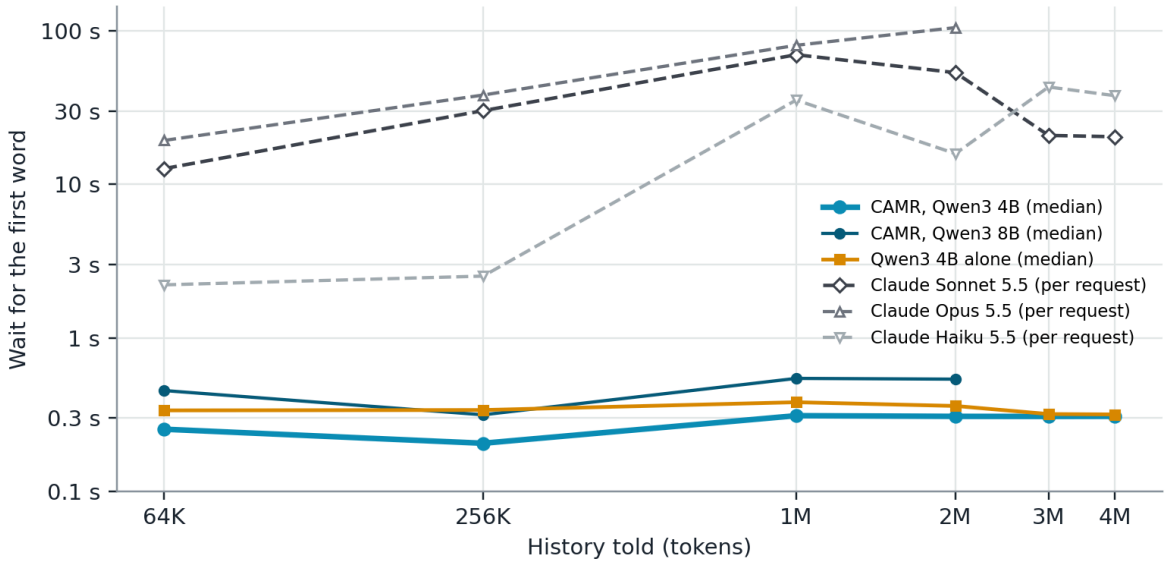


Figure 5.6: The wait for the first word as the history grows: CAMR's median (cyan) against the model alone (amber) and Claude's first token per request (grey, dashed); note the logarithmic scale

Figure 5.6 sets the waits side by side. For comparison, the 4B with CAMR answered 105 of the same 216 questions from 64K to 2M and 58 of 146 at 3M and 4M. Sonnet answered more within its window, and less beyond it. Opus scored no better than Sonnet on the Long Haul (130 against 138) at twice the price, so it sat out the suites (E-36) and the longer cuts (E-39). Claude was honest: it never invented a never-said fact. Sonnet's and Opus's first token came after 13 s at 64K and up to 105 s at 2M, because each request reads up to a million tokens; the engine's first word came in under a third of a second. Spending for the night, suites included, was $28.71 of the $60 cap, every call logged.

5.8.12 **The Public Suites**

Table 5.20 gives the public suites. On the three mathematics suites, which test reasoning with nothing to remember, both Claude models were strong; HARP shows how much a worked correction in the context helps a related new problem.

Table 5.20: Right answers on the public suites

Suite	Claude Haiku 5.5	Claude Sonnet 5.5	Ollama 1.7B alone	Ollama 1.7B with the pairing
MATH-500, levels 3 to 5 (50)	44	47		
AIME 2025 and 2026 (60)	54	56		
HARP pairs, correction told (60)	55	58	28	34
HARP pairs, nothing told (60)	48	52	21	21
BABILong qa1–qa5, 4k to 128k (100)	78		39	20

For Claude, the told correction raised HARP from 48 to 55 (Haiku) and from 52 to 58 (Sonnet): a worked solution in the context carries to a new problem of the same kind. Haiku answered 78 of 100 BABILong items from 4k to 128k tokens.

5.8.13 **The Ollama Pairing on the Suites**

The Ollama 1.7B answers with its thinking off, so on HARP it was given the same room to work in words that Claude has to think in (8,192 tokens; E-43). With that room, a told correction helped it, and the pairing carried the correction better than the model's own window did: with the correction told, the paired model answered 34 of 60 and the bare one 28; with nothing told, both answered 21. At the shared 400-token limit, 82 of the bare model's 120 answers and 95 of the paired one's had been cut off mid-working, and both had scored 10 or 11 either way; those records are kept beside the reruns (D-04).

On BABILong the pairing lost: 20 of 100 against 39 for the bare model reading the newest 32,768 tokens (Table 5.21). Its questions ask where something is now, which needs the latest of many mentions, and the pairing's 512 tokens of notes hold too few of them (E-43). At 4k and 16k tokens the bare model sees the whole story, and it answered 16 and 10 of 25 against the pairing's 6 and 6; at 64k and 128k both fell. Claude Haiku, reading every story whole, answered 18 to 21 of 25 at every length. The bare runs' 58 answers lost to the empty-thinking check (D-03) were resumed before these counts were taken.

Table 5.21: BABILong by story length (25 items each; qa1 to qa5)

Length	Ollama 1.7B alone (newest 32K)	Ollama 1.7B with the pairing	Claude Haiku 5.5
4k	16	6	20
16k	10	6	21
64k	5	3	19
128k	8	5	18
All (100)	39	20	78

Memory as words through the pairing therefore helps where one worked example carries to the next problem, and hurts where the answer is the newest of many similar mentions. The engine's own contenders, which read whole rooms with dates, were not run on these suites (E-36).

5.8.14 **Multimodal Memory**

The engine can also remember files (E-44). A PDF becomes one moment per page, an image its caption and the text written in it, and a video a frame every 2 s, each described with its time, by a vision model that turns the file into words once; the moments are then kept like any others, with the file copied into the vault. The multimodal benchmark tells 21 files of Mama Njeri's Kitchen (PDF invoices and menus, photographed receipts and notices, charts and short whiteboard videos) among about 100,000 tokens of the Long Haul's own events, all in the first 60,000 tokens, and then asks 37 questions graded by the Long Haul's rules: 29 about facts planted in the files, by kind of file, and 8 never-said questions where declining is right. The contenders are Qwen2.5-VL-3B with CAMR, alone with its newest 32K tokens, and with the file itself in the prompt as an upper bound; Qwen3 4B with CAMR and alone, reading the vision model's words; and Ollama's `qwen2.5vl:3b`, paired and bare.

Table 5.22 gives the results for seed 1 by kind of file, with each contender's total on a second draw of files and questions (seed 2), from the records `*-media-*.json` of 4 November.

Table 5.22: Multimodal memory: right answers by kind of file (seed 1: 29 planted facts and 8 never-said questions; seed 2: the same counts)

Contender	PDF (8)	Image (9)	Chart (6)	Video (6)	Never said (8)	Seed 1 (37)	Seed 2 (37)	First word p50	Process RAM
Qwen3 4B with CAMR, reading the vision model's words	**8**	**9**	**6**	**6**	**8**	**37**	**35**	0.34 s	7.3 GB
Qwen3 4B alone (newest 64K)	1	5	3	1	7	17	18	0.32 s	12.4 GB
Qwen2.5-VL-3B with CAMR	8	9	6	6	4	33	31	0.21 s	5.1 GB
Qwen2.5-VL-3B with CAMR, looking at the image again	8	9	6	6	3	32		0.80 s	8.7 GB
Qwen2.5-VL-3B with the file in the prompt (upper bound)	8	9	6	6	5	34	33	1.23 s	4.4 GB
Qwen2.5-VL-3B alone (newest 32K)	0	0	0	0	0	0	2	0.14 s	6.3 GB

Ollama `qwen2.5vl:3b` with the pairing (words)	8	9	6	6	5	34	33	0.22 s	1.9 GB†
Ollama `qwen2.5vl:3b` alone (newest 32K)	0	0	0	0	5	5	2		1.9 GB†

With CAMR, every planted fact from every kind of file was recalled on seed 1, at least 40,000 tokens after the file was told, by the vision model and by the text-only 4B: 29 of 29 each, and 27 to 28 of 29 on seed 2. The 4B with CAMR answered all 37 on seed 1 and 35 on seed 2, declining every never-said question, and beat the vision model holding the file in its prompt (34 and 33) at under a third of its wait. The models alone recalled almost nothing: by the questions, the files had left their windows. Ollama's vision model, given memory as words through the pairing, did as well as the engine's vision model (34 and 33), in the smallest process. Looking at the image again added nothing on this set and cost nearly four times the wait. The vision models' weakness was honesty, not recall: with memory they invented answers to 3 to 5 of the 8 never-said questions, where the 4B invented none. († Ollama's own peak resident memory, not a process footprint.)

5.8.15 **Development Evidence: the Rooms Build on Scripted Lives**

> **In plain words.** Before the final engine, its first build was tested on three invented twelve-week lives. Its notes answered as well as the words they came from, it stayed as fast in week 12 as in week 1, and it did errands three times as well as the baseline. It also made things up about what it was never told.

The rooms build stored one room of notes per question and was judged on the three held-out lives, with pass marks written before the runs (E-19, E-20). Its design came from a negative finding: notes written one moment at a time, then laid side by side, lost the thread; with 30 unrelated moments beside the right one, the reader found 2 of 46 facts from such notes against 32 from the words (E-15). Writing each room as one page fixed this within a room, and Table 5.23 shows the desk that won.

Table 5.23: Desks compared on the development lives at day 83, MacBook Air (E-16)

Desk	**Knows (of 255)**	**First word**
Words, 3 rooms	144	1.65 s
Notes, 3 rooms, plus 4 closest moments	142	0.35 s
4 closest moments as words, no room	135	0.45 s
Notes, 1 room	120	0.17 s
Notes, 1 room, plus 4 closest moments	**158**	**0.36 s**

On the proof lives at week 12 (Table 5.24) the rooms build was level with the baseline on the same 1.7B reader, at half the wait and 0.4 GB less RAM, and without the baseline's hand-written rules. Paired over the 255 told facts, 138 were known by both, 26 by the rooms build alone and 28 by the

baseline alone. Its notes gave the same answer as the words on 3,483 of 3,600 questions. Its wait stayed at 0.064 to 0.072 s in every week of every life while its notes on the SSD grew from 82 MB to 963 MB, and its peak RAM (as MLX reports it) stayed between 1.65 and 1.83 GB.

Table 5.24: The proof lives at week 12, three lives added (E-20)

Contender	Knows (of 255)	False memories (of 45)	First word	Peak RAM
1.7B alone	0	0	0.05 s	1.31 GB
Perfect reader (every note pasted in)	90	17	1.90 s, climbing	2.77 GB
CAMR v1 (baseline), 1.7B	166	29	0.15 s	2.24 GB
CAMR v1 (baseline), 4B	208	10	0.35 s	3.58 GB
Rooms build, 1.7B	**164**	**25**	**0.07 s**	**1.82 GB**
Rooms build, the same memory read as words	163	24	0.09 s	2.20 GB

The errands separated the memories most (Table 5.25). Each quiz question became an errand whose tool needs that exact fact; "Where does my vet live?" became "Book me a ride to my vet's home", which calls `book_ride(destination)`. The rooms build did three times as many errands right as the baseline, although the two were level on the quiz, because the baseline's gates were tuned for questions and a request falls past them, whereas the rooms build opens the closest room however the task is worded (E-25). No memory ever asked when it did not know: the rooms build made up 42 of the 45 never-told errands. That was the clearest weakness of the first build; the final engine's 4B declined 9 to 10 of 10 never-said questions on the Long Haul.

Table 5.25: Errands on the proof lives, last day, 1.7B reader (E-25)

Memory	Right (of 255)	Old value	Made up (of 45)	Asked	First word
None	1	0	44	0	0.15 s
CAMR v1 (baseline)	40	6	45	0	0.16 s
Rooms build	**122**	16	42	0	0.19 s

The speed bench measured how the wait grows with the memory on the desk (Table 5.26). Notes met the 0.5 s mark at 2,000 words, but were 4.6 times sooner than words rather than the 10 times required by NFR-02. The 10 times had been estimated on the MacBook Air, which read text about eight times more slowly than the Pro (about 800 against 6,600 tokens a second; E-17). Notes skip the reading but still pay for the SSD read and the copy into graphics memory, so their head start grows with the room.

Table 5.26: Time to the first word by memory on the desk, median of 5 (E-20)

Words in the room	Notes read	Notes	Words	Notes sooner by
253	75 MB	0.078 s	0.148 s	1.9×
1,002	294 MB	0.116 s	0.431 s	3.7×

2,006	588 MB	0.233 s	1.080 s	4.6×
3,007	894 MB	0.278 s	1.605 s	5.8×

The room size was tuned on the development lives for the most facts known (E-18): rooms of up to 512 tokens with a join threshold of 0.80 (208 tokens on average) and four moments re-read knew 164 of 255 facts; rooms of up to 1,024 and 2,048 tokens knew fewer at every setting tried (131 to 158). A small, fixed desk is best for a small reader, which agrees with the baseline, where 128 tokens was best for a 0.5B reader on facts.

5.8.16 **The Baseline, CAMR v1, on Public Benchmarks**

> **In plain words.** The first version of CAMR, which kept text notes, showed on public test sets that a small model with memory can match a cloud model on facts, but not on reasoning.

The baseline was evaluated before the engine existed, on CPU-only machines with Kimi K3 as the cloud ceiling. Its results still hold for the baseline and are the real-data complement to the scripted tests above; they are not results of the final engine. Table 5.27 summarises them.

Table 5.27: The baseline's main results on public benchmarks (CAMR v1)

Claim	Evidence	Result	Verdict
Long-tail facts: a small model with memory matches the cloud model	22-model sweep; PopQA	20 of 22 models reach or beat Kimi K3 closed-book (66.7%) with memory	Met
Multi-hop: memory closes part of the gap	HotpotQA, 200 held-out questions	3B + CAMR v1 54.0% against Kimi K3 51.8%; the 0.5B gains 23 points but stays far below	Met from 3B
The second hop found without a graph	2WikiMultiHopQA retrieval, 500 questions	Full-support recall 35.8% → 78.4% at the same budget	Met
Reasoning gaps closed by memory	GSM8K	No: a residual of 83.3 points; worked examples hurt the 0.5B	Not met
Few wrong facts stated	Held-out PopQA, 200, judged	3.0% wrong stated (Kimi K3 alone 31.5%), 60.5% right (Kimi 67.5%)	Met, at a cost in coverage
Memory beats fine-tuning on facts	The same documents	71.7% against 13.3%	Met
Retrieval at most 10% of answer time	Device profile	56% for the 0.5B under contention; met only from 1.5B	Not met for 0.5B

Three findings carried into the engine's design. Memory helps when the gap is knowledge and not when it is reasoning, which the Long Haul's families confirmed. More text distracts a small reader, so the budget should be small, which the room size confirmed. And recency and importance weighting lowered full-support recall on independent questions (by 1.2 points on 2WikiMultiHopQA and 5.4 on HotpotQA), so the engine chooses rooms by similarity alone.

5.8.17 **What Failed, and Why**

Table 5.28 lists every weakness the measurements found, including those that were fixed, because they show how the system failed before the fix.

Table 5.28: Weaknesses found

Weakness	Evidence	Cause	Status
Two hops are not joined	0 of 10 at 2M (4B)	One room per question seldom holds both halves, and the reader does not join rooms	Open: reading several rooms together
The growing page is never built	0 of 3 for every model at 2M	It needs the week-1 design and the day-61 change together	Open, for everyone
Changed facts and look-alikes trail Claude	3 against 5 at 2M	The newest of several values must be weighed against the others	Open
The 1.7B's honesty depends on its wording	Never said 3 of 10 at 2M (own wording) against 7 (yardstick's)	A small reader follows its instructions too literally either way	Bounded: both reported; the 4B is honest
RAM was understated	19.2 GB footprint against 5.7 GB reported (E-40)	MLX's unbounded buffer cache	Fixed: 1 GB cap, footprint logged
Notes are 10 times sooner than words	4.6 times (E-20)	The mark was set on a slower machine	Not met, kept
Notes are large	The room files reached the 40 GB budget by 1M tokens and were held there by sleep; 147,456 bytes per token for the 4B	Keys and values are about 29,000 times their text for the 1.7B, and more for the 4B	Bounded by sleep; distilling not built
Swarm was not measured	No benchmark	Built late, then removed from the app (E-50)	Removed

5.8.18 **Limitations, Delimitations and Threats to Validity**

Limitations are weaknesses of the evidence; each is stated with its likely effect.

i. **A scripted test.** The Long Haul is one invented project, fixed by its seed and built to be hard in known ways. It is not a sample of real work, and its results compare memories fairly on such a history but do not show how CAMR would serve a real user.

ii. **Small samples.** Each checkpoint asks 34 to 73 questions, 10 per family (3 for the page), so a difference of one or two answers is within noise; the 3-answer gap at 2M between CAMR and Sonnet is not a reliable ranking. No run was repeated for variance; only the 4B's 3M checkpoint was run twice, with identical verdicts.

iii. **Claude is a yardstick, not an opponent.** Claude answered with no memory, reading the newest part of the history that fits its window. A cloud model with its own memory would do better past its window; the comparison shows what a window alone does, not the best a cloud service could do.

iv. **RAM before the cap is understated.** The 1.7B's and the 4B's RAM figures up to 2M were measured as MLX's peak, before the buffer-cache cap; the process held

more (E-40). Their scores are unaffected.

v. **One machine.** Every Long Haul, suite and speed number was measured on one MacBook Pro M5 Pro with 24 GB (the desk design on the MacBook Air, and the baseline on CPU-only machines); answers also differ slightly between chips (158 facts on the Air and 164 on the Pro for the same design; E-17).

vi. **Grading by rule.** A rule can miss a right answer worded in an unexpected way. Every raw answer is kept in the records, so any verdict can be checked by hand, but no hand audit was done for this report.

vii. **What was not measured.** The engine's own contenders were not run on the public suites (E-36); LongMemEval and two further suites have loaders but no runs; the Code agent has no benchmark; no user study and no accessibility audit were done; the app's screenshots come from one session.

Delimitations are boundaries chosen for the study: Apple-silicon Macs only; English text; frozen, quantised open models; no fine-tuning; memory on the device, with the cloud only as a yardstick. Threats to validity: internally, every contender read the same instructions and the same questions, and the one wording change (the 1.7B's) is reported both ways; externally, the results hold for one scripted project, one Mac and the models named.

Chapter 6: **Discussion, Conclusions, Recommendations and Future Works**

6.1 **Introduction**

Small language models that run privately on a laptop know far less than large cloud models, and they forget whatever falls outside their context window. This study built CAMR, an external memory engine that sits beside a frozen model on an Apple-silicon Mac and keeps the model's own notes, its keys and values, in rooms on the SSD. It measured what CAMR closes, what it does not, and at what cost, against the same models without memory and against Claude Haiku, Sonnet and Opus 5.5 reading as much of the history as their windows hold. This chapter answers each objective of Section 1.3.2 with the evidence of Chapter 5 (Section 6.2), states the limitations of that evidence (Section 6.3), and sets out what should be done because of it (Section 6.4) and what remains open (Section 6.5). It adds no new results; every statement refers to the section of Chapter 5 that supports it.

6.2 **Conclusions by Objective**

6.2.1 **Objective i: Techniques and Their Limits on a Device**

Selective techniques that keep what the model reads small work for a small model; techniques that give it more text do not. In the baseline, entity bridging, a one-step hop that needs no graph, doubled full-support recall on 2WikiMultiHopQA (35.8% to 78.4%) at the same budget, while recency and importance weighting lowered it (Section 5.8.16). In the engine, a desk of one room plus four moments knew 158 facts on the development lives, against 144 for three rooms read as words, and larger rooms knew fewer facts at every setting tried (Section 5.8.15). The reuse of a model's own keys and values worked within one room, where notes answered as the words did (164 against 163 facts), but notes written separately did not combine for a small reader: 2 of 46 facts against 32 from the words (E-15). This limit is the one that the literature on cache blending works around (Yao et al., 2025), and it is why the engine reads one room per question. A long context window is not a substitute for memory: Claude's one-million-token window held only the newest days of a 3 or 4 million-token history, and it recalled nothing planted there (Section 5.8.3).

6.2.2 **Objective ii: The Design**

The design kept what it is told as the model's own notes, found what a question needs by one similarity search over every moment, and controlled what the model reads by a fixed desk of one room of at most 512 tokens plus four moments re-read as words. Three further layers made the memory usable as it grows: tiers that keep hot rooms in RAM and put weak rooms to sleep within a disk budget, a vault of linked plain-text notes that a person can read and edit, and reach, which decides whose memory a question reads (Chapter 4). On the Long Haul, most rooms were asleep past 1M tokens, so most answers there read their room as words (Table 5.13); the organisation of the memory, not the notes, carried those answers. The design was faithful where it was tested strictly: within one room, notes and words gave the same answer to 3,483 of 3,600 questions (Section 5.8.15). It has one structural limit: a question sees one room, so facts told in two places were almost never joined (0 of 10 two-hop questions at 2M for the 4B; Section 5.8.4). Of the four set-ups tested (session,

project, global, and project with global; agent reach was not run), project memory was best (26 of 40), and mixing in a second project cost 2 answers (Section 5.8.10).

6.2.3 **Objective iii: On the Device**

CAMR ran entirely on a MacBook Pro M5 Pro with no dedicated graphics card, with the model and its memory on the Mac, and as a native Mac app with Chat and Code (Section 5.4.5). The memory grew on the SSD without making the model slower or heavier: from 112 thousand to 2.01 million tokens of history, the 1.7B's first word stayed at 0.08 to 0.15 s, retrieval at 17 to 24 ms and RAM at about 4.3 GB, while sleep held the room files at their 40 GB budget (Section 5.8.8). After the graphics buffer cache was capped, the 4B's whole process needed 5.4 to 7.5 GB at 3 and 4 million tokens, against 15.2 GB for the same model reading only its newest 64,000 tokens (Section 5.8.9). Of the marks set for the device, two were met and one was not: the wait and RAM stayed flat as memory grew (NFR-03) and the first word at 2,000 words came in 0.233 s, under the 0.5 s mark (NFR-01), but notes were 4.6 times sooner than words, not the 10 times required (NFR-02), because the mark was estimated on a slower machine (Section 5.8.15). The proposal's embedded database was replaced by files on the SSD (Table 4.9), and the cost of memory moved to storage: the room files reached their 40 GB budget by about 1 million tokens, and sleep held them there by putting most rooms away as words.

6.2.4 **Objective iv: Evaluation of the Gap**

Within its window, the cloud model is better; beyond it, memory is. Up to 1M tokens, Claude Sonnet alone answered 28 to 43 questions per checkpoint against 25 to 26 for the 4B with CAMR. At 2M, the 4B with CAMR answered 29 of 73 against Sonnet's 32 and Opus's 31, nearly closing the gap; at 3M and 4M it answered 29 both times, while Sonnet fell to 13 and then 10 (Sections 5.8.2 and 5.8.3). Against the same model without memory the gain was large at every size of history past the window: 29 against 11 for the 4B at 2M, and 27 against 11 for the 8B. CAMR's first word came in 0.2 to 0.3 s, against 13 to 105 s for Sonnet and Opus (Section 5.8.11).

The engine helps knowledge-bound tasks more than reasoning-bound ones, as the proposal predicted. At 2M it led Claude on facts planted far back (6 against 4 of 10) and on code that had to use the project's own helpers (4 against 2), but trailed on changed facts, look-alikes and corrected mathematics, where several moments must be weighed against each other, and no model built the page that needed a design from week 1 and a change from day 61 (Section 5.8.4). The baseline found the same on public benchmarks: with memory, 20 of 22 small models matched the cloud model on long-tail facts, while memory left an 83-point gap on arithmetic (Section 5.8.16). The 4B with CAMR was also honest: it declined 9 of 10 never-said questions at 2M and all 10 beyond it. The same held for files: told 21 PDFs, photographs, charts and videos among 100,000 tokens of history, the 4B with CAMR recalled all 29 planted facts and declined all 8 never-said questions, while the vision model alone recalled none (Section 5.8.14).

6.2.5 **Objective v: How Much to Read, and What Each Part Contributes**

For a small reader, the best desk is small and fixed: one room of at most 512 tokens and four moments re-read (Section 5.8.15), which keeps the context at a few hundred tokens however large the memory grows. The comparisons isolated what each part contributes. Saving notes rather than words bought speed, not accuracy (a comparison that is clean only at 64K and 256K, since most rooms were asleep at 1M and 2M): the same 1.7B answered 14 to 20 with notes and 16 to 17 with the same memory as words, three times sooner, at 40 GB of disk against 0.1 GB (Section 5.8.7). A larger reader did not raise the ceiling: the 8B and the 27B answered 27 at 2M against the 4B's 29 (Section 5.8.5). The instructions mattered more than the memory for the smallest model: the 1.7B's own wording raised its score from 15 to 21 at 2M but cut its honest declines from 7 to 3 (Section 5.8.6). The buffer-cache cap cut the process's RAM by more than half and nearly doubled its reading speed, with identical answers (Section 5.8.9). Project reach was the right default (Section 5.8.10).

Table 6.1 summarises how each objective was answered.

Table 6.1: Alignment from objectives to evidence and conclusions

Objective (Section 1.3.2)	**Evidence (Chapter 5)**	**Conclusion**	**Status**
i. Investigate techniques and their limits	5.8.16, 5.8.15, 5.8.3	Selective, small desks work; separately written notes do not combine; a long window forgets the start	Met
ii. Design a lightweight memory engine	Chapter 4; 5.8.15, 5.8.10	One room of the model's own notes plus four moments, with tiers, a vault and reach; faithful within a room	Met, with one structural limit (one room per question)
iii. Implement it on a consumer laptop	5.4.5, 5.8.8, 5.8.9	Runs on the Mac, flat in time and RAM as memory grows; 5.4–7.5 GB at 3–4M tokens	Met; NFR-02 (10×) not met; files replace the database
iv. Evaluate the gap by task type	5.8.2, 5.8.3, 5.8.4, 5.8.11	Nearly level with Sonnet at 2M, ahead past its window; knowledge closed, reasoning not	Met on a scripted test
v. Determine how much to read and what each part adds	5.8.15, 5.8.7, 5.8.5, 5.8.6, 5.8.10	A small fixed desk; notes buy speed; the memory sets the ceiling; wording matters for the 1.7B; project reach	Met

6.2.6 **Contribution**

The study contributes:

i. An open memory engine that keeps a frozen model's own keys and values in rooms on the SSD, with a core in RAM, tiers and sleep under fixed budgets, a vault a person can read and reach, whose wait and RAM stayed flat as memory grew to 2 million tokens.

ii. The Long Haul, a scripted project of 2 to 4 million tokens with 216 questions graded by rule, in eight families that each isolate one way of failing, and a harness in which every answer records what it cost and every number names the code that made it.

iii. A measured comparison of a 1.7B to 27B local model with memory against Claude Haiku, Sonnet and Opus alone, with Haiku and Sonnet also past their window, and with time, RAM and dollars per answer.

iv. A native Mac application with Chat and Code that shows the engine at work, drawing only what the engine measures, and a pairing that gives Ollama and other local servers memory as words.

v. The baseline's findings on public benchmarks: on facts, memory lets a small model match a cloud model; on reasoning, it does not.

6.3 **Limitations of the Evidence**

The conclusions above hold within limits that the evidence itself sets.

i. **The benchmark is scripted.** The Long Haul is one invented project, fixed by its seed and built to be hard in known ways. It tests the mechanism fairly, every contender on identical questions, but it is not a sample of real work, and it does not show how CAMR would serve a real user.

ii. **Claude is a yardstick, not an opponent.** Claude answered alone, with no memory, from the newest part of the history that fits its window. A cloud service with a memory of its own would do better past its window. The comparison shows what a window alone does, not the best a cloud provider could build, and it was run once per model with the provider's service of that night.

iii. **The RAM figures before the cap are understated.** Runs before the buffer-cache cap (the 1.7B and the 4B up to 2M) report MLX's peak, not the process's footprint, which reached 19.2 GB on the 4B (E-40). Their scores are unaffected, and every later record logs the footprint.

iv. **The samples are small and single.** Each checkpoint asks 34 to 73 questions, 10 per family (3 for the page); differences of one or two answers, including the 3-answer gap at 2M, are within noise. No run was repeated for variance, apart from the 4B's 3M checkpoint, which gave identical verdicts.

v. **What was not measured.** The engine's own contenders were not run on the public suites; LongMemEval and two further suites have loaders but no runs; the Code agent and the core have no benchmark, and waking an asleep room has no speed benchmark; the app had no user study and no accessibility audit; and every number of the final engine comes from one Mac. The multimodal test is small: 21 files and 37 questions, with one seed reported.

6.4 **Recommendations**

Each recommendation follows from a result or limitation in Chapter 5; essential safeguards come first (Table 6.2).

Table 6.2: Recommendations

Priority	**Audience**	**Recommended action**	**Evidence and condition**
1. Essential	Anyone measuring memory on Apple silicon	Cap MLX's buffer cache and record the process's footprint, not only MLX's peak	19.2 GB footprint against 5.7 GB reported, then 7.5 GB with the cap and identical answers (Section 5.8.9)
2. Essential	Anyone letting a small model act from memory	Use a reader that declines what it was never told (here the 4B), and confirm every tool call	The 1.7B made up 42 of 45 never-told errands; the 4B declined 9 to 10 of 10 never-said questions (Sections 5.8.15, 5.8.4)
3. High	Developers of on-device assistants	Keep a growing memory outside the context window, in dated rooms on the SSD (as the model's notes while the disk allows, as words once they sleep), and read one small room per question	First word and RAM flat from 112K to 2.01M tokens; 29 of 73 at 2, 3 and 4M (Sections 5.8.8, 5.8.3)
4. High	The same	Prefer a 4B reader on a 24 GB Mac: an 8B or a 27B with the same memory did not answer more, and cost time and RAM	Section 5.8.5
5. High	Users and app builders	Keep one memory per project, not per conversation or for everything	Project 26 of 40; session 11; global 24 (Section 5.8.10)
6. Medium	Builders of memory engines	Fix the model's instructions before any test run and report a small model under more than one wording	The 1.7B scored 15 or 21 at 2M by wording alone (Section 5.8.6)
7. Medium	Researchers evaluating memory	Test past the cloud model's window, count declines on never-said questions apart from facts known, and log the cost of every answer	Sections 5.8.3, 5.8.11
8. Lower	The student and supervisor	Obtain ethics approval before any study with real users, and rotate the API key used for the yardstick	Section 3.8

6.5 **Future Works**

Each direction extends CAMR from a limitation in Chapter 5, ordered by expected value (Table 6.3).

Table 6.3: Prioritised future works

Priority	**Direction and rationale**	**How to evaluate**
1	**Several rooms read together.** One room per question joined 0 of 10 two-hop questions at 2M; notes written apart do not combine for a small reader. Re-encoding where rooms meet, in the manner of cache blending, could join them	The two-hop and changed-fact families, at a first word no higher than one room's
2	**Sleep that distils.** Notes are about 29,000 times their text, and sleep now only puts rooms away as words. Naps on the charger that distil what was learnt, checked by a quiz before it is kept, would shrink the memory	Disk per million tokens and Long Haul scores before and after a night's sleep

3	**Settling unsure facts at recall.** The smallest reader guessed when it did not know, and changed facts were often answered with an old value. A check before answering could resolve the doubt from the moments or ask the user	Never-said declines and stale answers on the Long Haul, with facts known not falling
4	**The engine on the public suites and LongMemEval.** The suites ran on Claude and Ollama only; LongMemEval has a loader and a judge but no run	Suite accuracy with the engine's own wording, LongMemEval by question type
5	**The native engine behind the pairing.** Through Ollama and the OpenAI-compatible endpoint, memory goes in as words	First word and accuracy through the endpoint, as notes against words
6	**Other Macs and real users.** One M5 Pro was measured; 8 GB Macs and older chips were not. No user has used the app over weeks	The Long Haul on two more Macs; with ethics approval, a study of the app over several weeks
7	**The page that grows.** No model, Claude included, built the page that needed a design from week 1 and a change from day 61	The three page probes, checked in a headless browser

6.6 **Closing Remarks**

The study set out to measure how much of the gap between an on-device and a cloud language model a lightweight memory engine bridges. On a long scripted project, the answer is: nearly all of it at 2 million tokens, all of it and more once the history outgrows the cloud model's window, and little of it where the answer must be reasoned across several facts: no model joined two hops or built the growing page, and on corrected mathematics Claude led 6 to 4. A 4-billion-parameter model on a laptop, given its memory back from the SSD, answered as many questions at 4 million tokens as at 2 million, in a third of a second and a process of 5 to 8 GB. Most of that memory was read as words by then; what carried the answers was the memory's organisation into dated rooms, chosen by similarity, and what the saved notes added was speed. What a window forgets, a memory outside it keeps.

References

Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., et al. (2024). Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., & Vechev, M. (2025). MathArena: Evaluating LLMs on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*.

Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., et al. (2022). Improving language models by retrieving from trillions of tokens. *Proceedings of the 39th International Conference on Machine Learning, PMLR 162*, 2206–2240.

Chen, Z., Xiang, Z., Xiao, C., Song, D., & Li, B. (2024). AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. *Advances in Neural Information Processing Systems 37*.

Chevalier, A., Wettig, A., Ajith, A., & Chen, D. (2023). Adapting language models to compress contexts. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3829–3846.

Chhikara, P., Khant, D., Aryan, S., Singh, T., & Yadav, D. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Collins, A. M., & Loftus, E. F. (1975). A spreading-activation theory of semantic processing. *Psychological Review, 82*(6), 407–428.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2978–2988.

Dong, S., et al. (2025). A practical memory injection attack against LLM agents. *arXiv preprint arXiv:2503.03704*.

Ge, T., Hu, J., Wang, L., Wang, X., Chen, S.-Q., & Wei, F. (2024). In-context autoencoder for context compression in a large language model. *Proceedings of the Twelfth International Conference on Learning Representations*.

Gemma Team. (2024). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., & Zhong, L. (2024). Prompt Cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems 6*.

Grattafiori, A., et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Gunter, T., et al. (2024). Apple Intelligence foundation language models. *arXiv preprint arXiv:2407.21075*.

Gutierrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems 37*.

Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on Apple silicon* [Computer software]. Apple. https://github.com/ml-explore/mlx

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1*.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105.

Ho, X., Nguyen, A.-K. D., Sugawara, S., & Aizawa, A. (2020). Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. *Proceedings of the 28th International Conference on Computational Linguistics*, 6609–6625.

ISO/IEC. (2023). *ISO/IEC 25010:2023 Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model*. International Organization for Standardization.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering*. International Organization for Standardization.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2023). Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research, 24*(251), 1–43.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., & Qiu, L. (2023). LLMLingua: Compressing prompts for accelerated inference of large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 13358–13376.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769–6781.

Kuratov, Y., Bulatov, A., Anokhin, P., Rodkin, I., Sorokin, D., Sorokin, A., & Burtsev, M. (2024). BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems 33*, 9459–9474.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2024). Let's verify step by step. *Proceedings of the Twelfth International Conference on Learning Representations*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the*

Association for Computational Linguistics, 12, 157–173.

Lu, Z., Li, X., Cai, D., Yi, R., Liu, F., Zhang, X., Lane, N. D., & Xu, M. (2024). Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*.

Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., & Fang, Y. (2024). Evaluating very long-term conversational memory of LLM agents. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 13851–13870.

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 9802–9822.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 1–22.

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing, 568*, 127063.

Teyler, T. J., & DiScenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience, 100*(2), 147–154.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems 30*, 5998–6008.

Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., & Yu, D. (2024). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.

Wu, Y., Rabe, M. N., Hutchins, D., & Szegedy, C. (2022). Memorizing transformers. *Proceedings of the Tenth International Conference on Learning Representations*.

Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). C-Pack: Packed resources for general Chinese embeddings. *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 641–649.

Yang, A., et al. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Yang, A., et al. (2025). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2369–2380.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., & Jiang, J. (2025). CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *Proceedings of the Twentieth European Conference on Computer Systems*, 94–109.

Yue, A. S., Madaan, L., Moskovitz, T., Strouse, D. J., & Singh, A. K. (2024). HARP: A challenging human-annotated math reasoning benchmark. *arXiv preprint*

arXiv:2412.08819.

Zhang, Y., et al. (2025). Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(17), 19724–19731.

Appendix A: Reproducing the Results

Every number in Chapter 5 comes from a record in `v2/records/bench/2026-11-04/` of the repository, and every record can be made again with the commands below, run from `v2/` on an Apple-silicon Mac after `sh scripts/install.sh`. One measuring job should run at a time (E-35). Claude runs need an Anthropic key in `~/.config/camr/anthropic.env`; every Claude call is held in the ledger against the cap.

Build and inspect the Long Haul, and grade any answers:

```
.venv/bin/python -m camr_engine.bench.haul build --seed 1 --out work/haul/s1
.venv/bin/python -m camr_engine.bench.longhaul show --dir work/haul/s1
.venv/bin/python -m camr_engine.bench.haul grade --dir work/haul/s1 --
answers answers.jsonl --cut 1M
```

Run the engine, a bare model, Ollama and Claude on the Long Haul:

```
.venv/bin/python -m camr_engine.bench.local longhaul --contender
engine:qwen3-4b --dir work/haul/s1
.venv/bin/python -m camr_engine.bench.local longhaul --contender bare:qwen3-
4b --dir work/haul/s1
.venv/bin/python -m camr_engine.bench.ollama_run longhaul --contender
ollama:qwen3-1.7b --dir work/haul/s1
.venv/bin/python -m camr_engine.bench.longhaul claude --model claude-sonnet-
5-5 --dir work/haul/s1
```

Measure the wait as memory grows, and how fast each model reads and writes on this Mac:

```
.venv/bin/python -m camr_engine.bench.local speed --contender engine:qwen3-
1.7b --dir work/haul/s1
.venv/bin/python -m camr_engine.bench.throughput --out records/bench
```

Run the public suites:

```
.venv/bin/python -m camr_engine.bench.suite_run babilong --contender ollama-
bare:qwen3:1.7b --n 20
.venv/bin/python -m camr_engine.bench.suite_run harp --contender
ollama:qwen3-1.7b --n 20
.venv/bin/python -m camr_engine.bench.suite_run math500 --contender claude-
haiku-5-5 --n 50 --max-usd 10
```

Table A.1 lists the records behind each result.

Rebuild this report's charts and diagrams, and the report itself, from the repository root:

```
python docs/thesis/figures/src/make_charts.py
python docs/thesis/diagrams/src/final_diagrams.py
python docs/thesis/build_thesis.py
```

Table A.1: The records behind Chapter 5 (`v2/records/bench/2026-11-04/`)

Result	Record files
The 4B with CAMR, 64K to 2M; 3M without and with the cap; 3M and 4M	`071525-longhaul-engine-qwen3-4b.json`; `100255-…`; `143706-longhaul-engine-qwen3-4b.json`
The 4B alone	`092854-longhaul-bare-qwen3-4b.json`; `155141-longhaul-bare-qwen3-4b.json`
The 8B with CAMR and alone	`161208-longhaul-engine-qwen3-8b.json`; `171043-longhaul-bare-qwen3-8b.json`
The 27B with the memory as words	`172930-longhaul-words-qwen-27b.json`
The 1.7B, both wordings, with CAMR and alone	`044359-…`, `062857-…`, `051530-…`, `070454-longhaul-*-qwen3-1.7b*.json`
Ollama on the Long Haul	`053819-longhaul-ollama-qwen3-1.7b.json`; `052439-longhaul-ollama-bare-qwen3-1.7b.json`
Claude on the Long Haul	`022518-…`, `022717-…`, `023106-…`, `094146-…`, `094325-claude-*-longhaul.json`
Speed as memory grows	`055344-speed-engine-qwen3-1.7b.json`
Reach	`195128-…`, `195633-…`, `200137-…`, `201043-reach-engine-qwen3-4b-*.json`
The suites	`*-math500-*`, `*-aime-*`, `*-harp-*`, `*-babilong-*.json`
Claude's spending	`v2/records/bench/claude-ledger.jsonl`

Appendix B: Decision Log Entries Cited

The project's decision log (`v2/DECISIONS.md`) records every technical decision with its reason and evidence. Table B.1 lists the entries this report cites.

Table B.1: Decision log entries cited in this report

Entry	Decision
E-02	One reader and one embedder for every contender, so differences between contenders are differences between memories
E-05	Three scripted twelve-week lives, generated from a persona and a seed
E-08	Growth measured as facts known out of those told; false memories counted apart
E-10	Deterministic answers: greedy decoding, thinking off
E-12	Every record carries the machine, the versions and a hash of the code
E-14	Honest SSD reads: room files read with the file cache bypassed
E-15	A room is written as one continuous page
E-16	One room on the desk, plus the closest moments as words
E-17	A new machine: the MacBook Pro M5 Pro, with the MacBook Air's results kept
E-18	Room size and how many moments to re-read, tuned on development lives
E-19	Step 1's pass marks, written before any held-out run
E-20	Step 1's results against its pass marks
E-21	What "used" means, and what watching attention costs
E-24	The first app: a local page with its security checks
E-25	The errands, measured
E-26	A model sixteen times bigger on the same Mac, reading the memory as words
E-27	A crash window in the room files, and its fix
E-29	The engine is the project: three layers, reach, local only, Claude only as a yardstick
E-30	The Long Haul, the study's own hard test
E-31	Asking never teaches
E-32	Everyone reads the same instructions
E-33	Ollama pairs through a proxy
E-34	The app's visual design, "Instrument"
E-35	One measuring job at a time
E-36	The Hugging Face suites run on Claude and Ollama, not yet on the engine
E-37	The 1.7B's wording, not its memory, sank its recall at 1M and 2M
E-38	Speed does not grow with memory
E-39	Past 2M: the same questions under one and two million more tokens
E-40	The engine's process held 19 GB; MLX's buffer cache capped at 1 GB
E-41	The cap, measured: a third of the RAM, almost twice the speed, the same answers
E-42	Project reach is the best reach
E-43	The Ollama suite runs redone where the runner, not the model, set the score
E-44	Files into memory: a vision model turns them into words once

E-47	An OpenAI-compatible endpoint with memory
E-48	One `camr` command and its installers
E-49	The landing page and the meaning of colour
E-50	Swarm removed: the app is Chat and Code